

Státnicové otázky

Poslední aktualizace: 23. května 2026

Obsah

I SOFI	3
Mobilní technologie	4
Vývoj mobilních aplikací	7
Inversion of Control (IoC)	12
Architektura informačních systémů	14
Webové služby a mikroslužby	21
NoSQL databáze a jejich základní principy	26
Data v distribuovaných systémech	31
Struktura a architektura systému UNIX / GNU / Linux	35
Řízení procesů UNIXu	44
Základy běžné uživatelské administrace systému UNIX	49
Ochrana dat a informací	53
Kryptografie	60
 II TIM	 67
Formální jazyky, gramatiky a automaty	68
Teorie vyčíslitelnosti	73
Teorie složitosti	76
Algoritmy a jejich složitosti	79
Orientované grafy	86
NP úplné problémy z oblasti diskrétní matematiky	94
Problematika numerických metod a řešení nelineárních rovnic	101
Aproximace funkcí, numerický výpočet derivace a integrálu funkcí	107
Numerické řešení soustav lineárních algebraických rovnic - přímé a nepřímé metody	113

Popis datového souboru	119
Princip inference	126
Obousměrné a jednosměrné vztahy kvantitativních veličin	130
Principy strojového učení	137
Neuronové sítě	142
Principy teorie her	146

Část I

SOFI

1 Mobilní technologie

historický vývoj, klasifikace mobilních zařízení – typy, OS, kategorie aplikací, specifické problémy mobilních platforem, senzory a jejich využití v kontextu mobilního uživatele

1.1 Historický vývoj

Vývoj mobilních technologií lze rozdělit do několika generací (tzv. “G”) a klíčových milníků:

1G (80. léta) Analogové sítě (NMT). Sloužily pouze pro přenos hlasu, nízká kvalita, žádná bezpečnost.

2G (90. léta) Přechod na digitální sítě (GSM). Zavedení SMS, MMS, lepší kvalita hovorů.

3G (počátek 2000+) Důraz na mobilní internet a multimédia (jednotky Mbps). Video hovory.

Zlomový rok 2007/2008:

2007 Apple představuje iPhone (iOS) – revoluce v UI (multitouch, ovládání prsty místo stylusu, absence fyzické klávesnice).

2008 Představen první telefon s OS Android (HTC Dream) a spuštěny obchody s aplikacemi (App Store, Android Market/Google Play).

4G/LTE (2010+) Vysokorychlostní internet. Masivní rozmach streamování videa, cloudových služeb a komplexních aplikací, VoIP.

5G (současnost) Extrémně nízká latence, vysoká propustnost. Rozvoj IoT (Internet věcí), autonomních vozidel a edge computingu. Nástup skládacích telefonů a integrace AI.

1.2 Klasifikace mobilních zařízení - typy

Personal Digital Assistants (PDA) Předchůdci smartphonů. Dotykové obrazovky, virtuální klávesnice, rozpoznávání písma. Organizační funkce: diář, adresář, poznámky. Synchronizace s PC. OS: Palm OS, Windows Mobile.

Smartphony Nejrozšířenější kategorie, kombinují telefon, počítač a multimediální zařízení. Vlastní operační systém.

Tablety Větší obrazovka, stejné OS jako smartphony.

Wearables Chytré hodinky, fitness náramky. Zaměřené na zdraví, notifikace a rychlé interakce.

IoT zařízení Chytré domácnosti, průmyslové senzory. Zaměřené na automatizaci a sběr dat. Normální operační systémy i RTOS.

1.3 Operační systémy

Operační systémy pro mobilní zařízení se liší v přístupu k uživatelskému rozhraní, bezpečnosti, vývojářským nástrojům a obchodům s aplikacemi:

iOS Uzavřený ekosystém, vysoká bezpečnost, optimalizace pro Apple hardware. App Store s přísnou kontrolou kvality. Programování v Swiftu a Objective-C.

Android Otevřený zdrojový kód, linuxové jádro, široká přizpůsobitelnost, velká fragmentace zařízení. Google Play s méně přísnými pravidly. Programování v Javě a Kotlinu.

Další starší OS Windows Mobile, BlackBerry OS, Symbian, Tizen.

1.4 Kategorie aplikací

Aplikace pro mobilní zařízení jsou různého typu, v závislosti na způsobu vývoje a cílové platformě:

Nativní aplikace Aplikace napsané pro konkrétní OS (iOS, Android). Vysoký výkon, přístup ke všem funkcím zařízení. Vyžadují vývoj pro každou platformu zvlášť.

Webové aplikace Spouštěné v prohlížeči, napsané v HTML, CSS, JavaScriptu. Platformově nezávislé, ale omezený přístup k funkcím zařízení a nižší výkon.

Hybridní aplikace Kombinace nativních a webových technologií (např. Apache Cordova, Ionic). Snaží se nabídnout kompromis mezi výkonem a multiplatformní kompatibilitou.

PWA (Progressive Web Apps) Webové aplikace s funkcemi blízkými nativním aplikacím (offline režim, push notifikace). Spouštěné v prohlížeči, ale mohou být "instalovány" na zařízení.

Cross-platformní frameworky Nástroje pro vývoj jedné kódové základny, která se překládá do nativních aplikací pro různé platformy (např. React Native, Flutter). Snaží se minimalizovat potřebu psaní specifického kódu pro každou platformu.

1.5 Specifické problémy mobilních platforem

Fragmentace Různé verze OS, různé hardwarové konfigurace. Ztížená optimalizace a testování aplikací.

Bezpečnost Neautorizovaný přístup k datům, malware (zejména u Androidu). Ochrana osobních údajů, sandboxing, oprávnění aplikací.

Omezené zdroje Nižší výkon, omezená baterie, menší úložiště. Nutnost optimalizace kódu a efektivního využití zdrojů.

Uživatelské rozhraní Menší obrazovka, dotykové ovládání. Nutnost přizpůsobení UI pro různé velikosti a rozlišení.

Síťová konektivita Nestabilní připojení, přechod mezi sítěmi (Wi-Fi, 4G/5G). Práce s offline režimem. Snížená spotřeba dat.

1.6 Senzory a jejich využití v kontextu mobilního uživatele

Oproti desktopovým počítačům mají mobilní zařízení širokou škálu integrovaných senzorů, které umožňují nové způsoby interakce s okolním světem:

Akcelerometr Měří zrychlení, detekce pohybu a orientace zařízení. Používá se pro krokoměry, detekci pádu, ovládání gesty.

Gyroskop Měří rychlost rotace zařízení. Používá se pro herní aplikace, navigaci a realitu.

GPS Poskytuje geografické informace. Používá se pro navigaci, mapy a lokální služby.

Proximity senzor Detekuje přítomnost předmětů v blízkosti zařízení. Používá se pro automatické vypnutí obrazovky při volání.

Světelný senzor Měří úroveň osvětlení v okolí zařízení. Používá se pro automatickou úpravu jasu obrazovky.

Biometrické senzory Čtečky otisků prstů, rozpoznávání obličeje. Používají se pro autentizaci a zabezpečení.

Magnetometr Měří magnetické pole, funguje jako kompas. Používá se pro navigaci a orientaci.

[Zpět](#) k využití specifického HW (Kapitola 2.)

2 Vývoj mobilních aplikací

specifika vývoje mobilního softwaru, nativní vs hybridní vývoj, Základní charakteristika OS Android (architektura, nástroje) nebo iOS. Vývojářské nástroje a jazyky/frameworky pro nativní vývoj. Obecná problematika tvorby UI (Views, Compose, Swift UI případně dalších). Využití specifického HW, způsoby napojení na IS

2.1 Specifika vývoje mobilního softwaru

Vývoj mobilních aplikací má svá specifika, která se liší od vývoje pro desktopové nebo webové platformy. Klíčové aspekty zahrnují:

Optimalizace pro omezené zdroje Mobilní zařízení mají omezenou výpočetní kapacitu, paměť a baterii. Vývojáři musí optimalizovat kód a minimalizovat spotřebu zdrojů.

Uživatelské rozhraní Menší obrazovka a dotykové ovládání vyžadují přizpůsobení UI pro různé velikosti a rozlišení. Důraz na jednoduchost a intuitivnost.

Fragmentace Různé verze OS, různé hardwarové konfigurace ztěžují optimalizaci a testování aplikací.

Bezpečnost Mobilní platformy jsou náchylné k bezpečnostním hrozbám, jako je malware a neoprávněný přístup k datům. Vývojáři musí implementovat bezpečnostní opatření.

Integrace s hardwarem Mobilní zařízení mají širokou škálu senzorů a funkcí (GPS, kamera, akcelerometr).

Distribuce a monetizace Mobilní aplikace jsou distribuovány prostřednictvím obchodů s aplikacemi (App Store, Google Play), které mají specifické požadavky a pravidla.

Sandboxing a oprávnění Mobilní platformy používají sandboxing pro izolaci aplikací a systém oprávnění pro kontrolu přístupu k funkcím zařízení.

2.2 Nativní vs. hybridní vývoj

Nativní vývoj znamená psaní aplikace specificky pro jeden operační systém (iOS, Android), zatímco hybridní vývoj využívá webové technologie (HTML, CSS, Javascript) a běží v rámci WebView nebo nativního kontejneru.

Nativní vývoj nabízí lepší výkon, přístup ke všem funkcím zařízení a lepší uživatelský zážitek, ale vyžaduje vývoj pro každou platformu zvlášť. Hybridní vývoj umožňuje multiplatformní kompatibilitu s jednou codebase, ale může mít nižší výkon a omezený přístup k funkcím zařízení.

Nativní vývoj iOS (Swift, Objective-C), Android (Java, Kotlin).

Hybridní vývoj Frameworky jako Apache Cordova, Ionic.

2.3 Základní charakteristika OS Android

Android je otevřený operační systém založený na linuxovém jádře, spravovaný společností Google. Nad jádrem je vrstva Android Runtime (dříve Dalvik) pro spouštění aplikací, knihovny pro různé funkce a aplikační rámec pro vývojáře. Aplikace jsou psány v Javě nebo Kotlinu a běží v sandboxu (izolovaně) pro bezpečnost. Komponenty Androidu:

Activity Uživatelské rozhraní a interakce.

Service Běží na pozadí pro dlouhodobé operace.

Broadcast Receiver Reaguje na systémové události. (např. změna sítě)

Content Provider Sdílení dat mezi aplikacemi.

Vývojové nástroje zahrnují Android Studio, SDK, emulátory. Způsoby tvorby UI:

XML views Tradiční přístup k tvorbě UI pomocí XML a Java/Kotlin.

Jetpack Compose Moderní deklarativní framework pro tvorbu UI v Kotlinu.

Struktura OS:

Jádro Linuxové jádro pro správu hardwaru a systémových zdrojů.

Hal Hardware Abstraction Layer pro komunikaci s hardwarem.

Android Runtime Spouštění aplikací, správa paměti.

Knihovny Nativní knihovny pro grafiku, databáze, síť, atd.

Java API Pro vývoj aplikací.

Aplikační rámec API pro vývojáře pro přístup k funkcím zařízení.

Aplikace Uživatelské aplikace a systémové aplikace.

2.4 Základní charakteristika OS iOS (Volitelné stačí znát pouze Android)

iOS je uzavřený operační systém vyvinutý společností Apple pro své mobilní zařízení. Je založen na unixovém jádře a používá architekturu MVC (Model -View-Controller). Aplikace jsou psány v Swiftu nebo Objective-C a běží v sandboxu pro bezpečnost. Vývojové nástroje zahrnují Xcode, iOS SDK, simulátory (Apple nazev pro emulátory). Způsoby tvorby UI:

Storyboardy Vizualní nástroj pro návrh UI a navigace mezi obrazovkami.

SwiftUI Moderní deklarativní framework pro tvorbu UI v Swiftu.

Struktura OS:

Jádro Unixové jádro pro správu hardwaru a systémových zdrojů.

Core OS Nízká úroveň služeb, jako správa paměti, bezpečnost, atd.

Core Services Služby pro síť, data, atd.

Media Knihovny pro grafiku, zvuk, video.

Application Framework API pro vývojáře pro přístup k funkcím zařízení.

Aplikace Uživatelské aplikace a systémové aplikace.

2.5 Vývojářské nástroje a jazyky/frameworky pro nativní vývoj

iOS Xcode IDE, Swift a Objective-C jazyky, iOS SDK, Interface Builder (součást Xcode) pro návrh UI. Framework SwiftUI pro deklarativní UI.

Android Android Studio IDE, Java a Kotlin jazyky, Android SDK, XML pro návrh UI, Framework Jetpack Compose pro deklarativní UI.

2.6 Obecná problematika tvorby UI

Tvorba uživatelského rozhraní pro mobilní zařízení zahrnuje několik klíčových aspektů:

Responsivita UI musí být přizpůsobeno různým velikostím obrazovek a rozlišením. Použití flexibilních layoutů a komponent.

Dotykové ovládání UI musí být optimalizováno pro dotykové ovládání, s dostatečně velkými tlačítky a intuitivními gesty.

Výkon UI by mělo být plynulé a rychlé, minimalizovat zpoždění a optimalizovat načítání dat.

Designové principy Dodržování platformových designových směrnic (Material Design pro Android, Human Interface Guidelines pro iOS) pro konzistentní a intuitivní uživatelský zážitek.

Testování Testování UI na různých zařízeních a verzích OS pro zajištění kompatibility a správné funkčnosti.

Views Tradiční přístup k tvorbě UI pomocí XML (Android) nebo Storyboardů (iOS) a programování v Java/Kotlin nebo Swift/Objective-C. Umožňuje detailní kontrolu nad vzhledem a chováním UI, ale je náročnější na údržbu a aktualizace. XML views v Androidu definují rozložení a v kódu se odkazuje pomocí ID. Komponenty jako: TextView, Button, ImageView, atd.

Compose a SwiftUI Moderní deklarativní frameworky pro tvorbu UI, které umožňují psaní UI kódu přímo v programovacím jazyce (Kotlin pro Jetpack Compose a Swift pro SwiftUI). Snadné přizpůsobení různým typům zařízení, lepší integrace s logikou aplikace a jednodušší aktualizace UI na základě změn stavu. Jetpack Compose tvoří UI pomocí funkcí s anotací @Composable, které vracejí UI komponenty.

Architektury:

MVVM Model-View-ViewModel, ViewModel obsahuje data pro UI. Model komunikuje s UI a dodává data pro zobrazení. ViewModel zpracovává logiku a aktualizuje UI na základě změn stavu.

MVI Model-View-Intent, UI pouze volá akce (intenty), které mění stav modelu, a UI se aktualizuje na základě změn stavu.

2.7 Využití specifického HW

Mobilní zařízení mají širokou škálu integrovaných senzorů a funkcí, které umožňují nové způsoby interakce s okolním světem. Vývojáři mohou využít tyto senzory pro různé účely, jako je detekce pohybu, navigace, autentizace a další.

Senzory Akcelerometr, gyroskop, GPS, proximity senzor, světelný senzor, biometrické senzory, magnetometr. Viz. [Senzory](#)

Kamera Využití pro fotografování, skenování QR kódů, rozšířenou realitu.

Mikrofon Využití pro hlasové ovládání, rozpoznávání řeči, nahrávání zvuku.

Bluetooth a NFC Využití pro komunikaci s jinými zařízeními, bezkontaktní platby, připojení periférií.

Přístup k těmto funkcím se obvykle realizuje prostřednictvím API poskytovaných operačním systémem, je nutné získat oprávnění od uživatele pro přístup k citlivým funkcím a datům. Od androidu 6.0 a iOS 10 je nutné žádat o oprávnění za běhu aplikace než pouze při instalaci.

2.8 Způsoby napojení na IS

Mobilní aplikace často potřebují komunikovat s informačními systémy (IS) pro získávání dat, autentizaci uživatelů, synchronizaci a další funkce. Způsoby napojení na IS zahrnují:

REST API Nejčastější způsob komunikace, využívá HTTP protokol pro výměnu dat ve formátu JSON nebo XML.

GraphQL Alternativa k REST, umožňuje klientům specifikovat přesně, jaká data potřebují. Protokol: HTTP

WebSocket Pro real-time komunikaci, umožňuje obousměrnou komunikaci mezi klientem a serverem.

Při napojení na IS je důležité zohlednit bezpečnostní aspekty, jako je autentizace, autorizace, šifrování dat a ochrana proti útokům (např. SQL injection, XSS).

Implementace na Androidu: Knihovny jako Retrofit a OkHttp (starší) pro REST API a WebSocket, Apollo pro GraphQL. Autentizace může být řešena pomocí OAuth 2.0, JWT, nebo API klíčů. Lokální ukládání dat může být realizováno pomocí SQLite (knihovna Room), SharedPreferences nebo DataStore. Ke komunikaci Jetpack Compose využívá Interceptors: zachytávají události, Authenticator specialní Interceptor pro autentizaci, Logging Interceptor pro logování síťové komunikace.

3 Inversion of Control (IoC)

principy, význam, a použití přístupu, vztah IoC a Dependency Injection. Příklady použití, nástroje podporující IoC

3.1 Principy

Inversion of Control (IoC) je princip návrhu softwaru, který obrací tradiční řízení toku programu. Místo toho, aby komponenty samy řídily své závislosti a životní cyklus, IoC umožňuje externímu kontejneru nebo frameworku řídit tyto aspekty. IoC je všeobecný koncept, který může být implementován různými způsoby, z nichž nejběžnější je Dependency Injection (DI).

3.2 Význam

Modularita Komponenty jsou méně závislé na konkrétních implementacích svých závislostí, což usnadňuje jejich výměnu a údržbu.

Testovatelnost Snadnější testování díky možnosti snadno nahradit závislosti mocky.

Flexibilita Snadnější změna implementace závislostí bez nutnosti měnit kód, který je používá.

Znovupoužitelnost Komponenty mohou být znovu použity v různých kontextech díky oddělení od konkrétních implementací závislostí.

3.3 Použití přístupu

IoC je možné implementovat několika přístupy.

Dependency Injection (DI) Závislosti jsou "injektovány" do komponenty zvenčí, obvykle prostřednictvím konstruktoru, setterů nebo rozhraní.

```
public ServiceA(ServiceB serviceB) {  
    this.serviceB = serviceB;  
}
```

Service Locator Komponenta získává své závislosti z centrálního registru nebo "služby", která je zodpovědná za poskytování instancí.

```
ServiceB serviceB = ServiceLocator.getService(ServiceB.class);
```

Event-driven Komponenty komunikují prostřednictvím událostí, které jsou zpracovávány externím systémem.

```
eventBus.post(() => {  
    // Handle event  
});
```

3.4 Vztah IoC a Dependency Injection

Dependency Injection (DI) je konkrétní implementace principu Inversion of Control (IoC). Zatímco IoC je obecný koncept, který může být realizován různými způsoby, DI je konkrétní způsob, jak dosáhnout IoC tím, že umožňuje externímu systému "injektovat" závislosti do komponenty. DI je nejběžnější a nejpoužívanější způsob implementace IoC.

3.5 Příklady použití a nástroje podporující IoC

IoC se používá v mnoha moderních frameworkách a knihovnách pro vývoj softwaru, zejména v oblasti webových a mobilních aplikací. Například:

Spring Framework (Java) Používá IoC kontejner pro správu závislostí a životního cyklu komponent. Využití DI pomocí anotací jako `@Autowired` pro automatické vkládání závislostí, `@Component` pro označení komponenty, `@Service` pro označení služby, `@Repository` pro označení repozitáře. Princip: Dependency Injection (DI) pomocí anotací.

Angular (TypeScript) Používá DI pro správu služeb a komponent. Princip: Dependency Injection (DI) pomocí dekorátorů jako `@Injectable`.

ASP.NET Core (C#) Podporuje DI pro správu závislostí v aplikacích. Princip: Dependency Injection (DI) pomocí vestavěného kontejneru a metod jako `AddTransient`, `AddScoped`, `AddSingleton`.

NestJS (Node.js) Používá DI pro správu závislostí v serverových aplikacích. Princip: Dependency Injection (DI) pomocí dekorátorů jako `@Injectable` a modul

Android Knihovny jako Dagger a Hilt pro implementaci DI v Android aplikacích. Princip: Dependency Injection (DI) pomocí anotací `@Inject` a generování kódu pro správu závislostí.

4 Architektura informačních systémů

vrstva business logiky, prezentační vrstva, datová vrstva - využití relačních a nere-lačních DB v kontextu IS, návrhové vzory pro IS, metody tvorby, komunikace mezi IS, automatizace, DevOps, využití do Cloudu

Informační systém je softwarová aplikace zpracovávající a poskytující data pro podporu plánování, rozhodování a řízení. Je to celek složený z počítačového hardwaru a souvisejícího softwaru, k němuž patří také lidé, kteří tento hardware a software využívají, a procesy (činnosti), které přitom vykonávají za účelem sběru, zpracování a šíření informací potřebných k plánování, rozhodování a řízení.

Charakteristika IS:

- Aplikace nebo integrovaná sada aplikací
- Slouží pro celou organizaci (ne jen pro jednoho uživatele)
- Sdílení dat mezi uživateli
- Běží mimo počítače uživatelů (na centrální infrastruktuře/v cloudu)
- Uživatelé využívají pro přístup k IS svých klientských zařízení

Typy IS:

Podnikové IS – Systémy využívané komerčními organizacemi a dostupné zejména pro jejich pracovníky. (Univerzální, Speciální, Na míru)

Veřejné IS – Veřejně či komunitně dostupné IS. (Encyklopedie (např. wikipedie), knihovní systémy, vyhledávače, mapové systémy, ...)

4.1 Vrstvy IS

Vrstva business logiky: Vrstva business logiky koordinuje tok dat mezi prezentační a datovou vrstvou. Obsahuje většinu algoritmů systému. Většinou reprezentovaná třídami v objektově orientovaném jazyce (Java, C#, JavaScript, ...). S okolím komunikuje pomocí rozhraní, např. REST API, SOAP, atd.

Vrstva prezentační: Prezenční vrstva zpracovává vstupy od uživatele a volá služby vrstvy business logiky. Sama neobsahuje žádnou business logiku, kromě validace, jednoduché výpočty (důraz na efektivitu a výkon). Často postavená na návrhovém vzoru Model-View-Controller a jeho variacích. Časté využití technologií založených na JavaScriptu. (Html, CSS, JS, Reacts, Flutter, WPF)

Vrstva datová: Datová vrstva obsahuje služby pro ukládání, načítání dat a dotazování. Pracuje s databázemi, cache, soubory, atd. Primárně zajišťuje persistentní uložení dat, jejich dostupnost a integritu. Pojmy:

- **CRUD** – Create, Read, Update, Delete.
- **ORM** – Mapování objektové struktury aplikace na relační strukturu databáze.
- **Scéma** – Schéma je datový model, který popisuje data a vztahy mezi nimi. (datové typy, domény - povolené/povinné hodnoty, relace - vazby mezi entitami)

Většinou reprezentovaná databází:

- **SQL:** Mají dané schéma. Jsou reprezentované relačním datovým modelem. Důraz na integritu dat. Princip ACID:
 - Atomicity – Pokud se transakce skládá z několika kroků, databáze provede všechny, nebo žádný. Pokud v průběhu dojde k chybě, celá transakce se vrátí zpět (tzv. rollback).
 - Consistency – Transakce musí převést databázi z jednoho platného stavu do druhého. Veškerá data musí splňovat definovaná pravidla, omezení (constraints) a integritu.
 - Isolation – Souběžně probíhající transakce se vzájemně neovlivňují. Výsledky rozpracované transakce nejsou viditelné pro ostatní procesy, dokud není transakce zcela dokončena.
 - Durability – Po úspěšném dokončení transakce (commit) jsou změny trvale zapsány a nevymaže je ani náhlý výpadek proudu či pád systému.

Vhodné pro IS s komplexními datovými vztahy (ERP, Účetní systémy apod). Možné pouze vertikální škálování (zvýšení výkonu jednoho stroje). Technologie například MySQL, PostgreSQL, MS SQL, Oracle, MariaDB.

- **NOSQL:** Mají flexibilní schema. Důraz na distribuci dat a škálovatelnost. Typy:
 - Dokumentové – Ukládání ve formátu JSON/BSON. Např. MongoDB, CouchDb.
 - Key-Value – Ukládání do mapy zvané Hash. Často jako cache. Např. Redis.
 - Sloupcové – Čtení po sloupcích což je efektivní pro velké datasety. Např. Cassandra.

- Grafové – Specializace na propojená data. Využití například u sociálních sítí. Např. Neo4j.

Vhodné pro BigData, aplikace s rychlým vývoje struktura, vysoká dostupnost, realtime (chat, senzory, IoT). Podpora transakcí je slabší nebo žádná. Nemají standardizovaný dotazovací jazyk jako SQL. Jsou vhodné na horizontální škálování (přidání dalších strojů).

Více k [NOSQL](#). (Kapitola 6.)

4.2 Architektury IS

Architektura je sadou zásadních rozhodnutí týkajících se struktury systému, návrhu komponent, jejich vzájemných vztahů a vztahů s okolím. Jde o nejvyšší (a nejdůležitější) úroveň SW návrhu.

Co řeší:

- Rozčlenění systému na samostatné celky (subsystémy /komponenty, mikroslužby)
- Hlavní komponenty IS a jejich rozhraní
- Hlavní vrstvy systému, způsob jejich realizace a vzájemné komunikace
- Doménový model
- Použité technologie
- Způsob integrace s okolními systémy a periferiemi

Typy:

Vrstvená architektura: Také označovaná jako „Vrstvený monolit“ nebo jen „Monolit“. Systém je vytvořen jako jeden nasaditelný celek, který je vnitřně rozčleněn na vrstvy, komponenty. Obsahuje všechny zmíněné [vrstvy](#). Snadné ladění a testování ale špatná údržba a škálovatelnost.

Modulární monolit: Architektura je rozdělená do modulů, což zlepšuje organizaci, ale stále běží jako jedna aplikace. (Modul = samostatná logická jednotka, např. fakturace, objednávky)

Architektura založená na mikroslužbách: Rozdělení jednoho systému na více samostatných a samostatně nasaditelných celků. (Mikroslužba = samostatně nasaditelná komponenta IS, často se samostatnou DB, která se zaměřuje na omezenou oblast business logiky) Dobrá škálovatelnost, dostupnost a podpora paralelního vývoje. Složitě nasazení, údržba a ladění. Služby jsou spravovány nástrojem pro orchestraci, například Kubernetes nebo Docker swarm. [Zpět](#) (Kapitola 5.)

Konkrétní návrhové vzory:

MVC (Model-View-Controller): Oddělení datového modelu (Model), uživatelského rozhraní (View) a řídicí logiky (Controller). Cílem je, aby změna v UI minimálně ovlivňovala datovou logiku a naopak.

MVVM (Model-View-ViewModel): Používaný ve webových a mobilních aplikacích (Angular, WPF). ViewModel připravuje data z modelu přímo pro potřeby zobrazení a využívá obousměrnou vazbu dat (data binding).

Hexagonální architektura (Ports and Adapters): Zaměřuje se na izolaci jádra business logiky od vnějších vlivů (databáze, API, UI). Jádro definuje "porty"(rozhraní) a okolí implementuje "adaptéry". Díky tomu lze snadno vyměnit např. typ databáze bez zásahu do logiky.

4.3 Metody tvorby IS

Metodiky se dnes dělí primárně podle přístupu k plánování a změnám na tradiční (prediktivní) a agilní. Většinou se kombinují.

Tradiční (prediktivní) přístupy

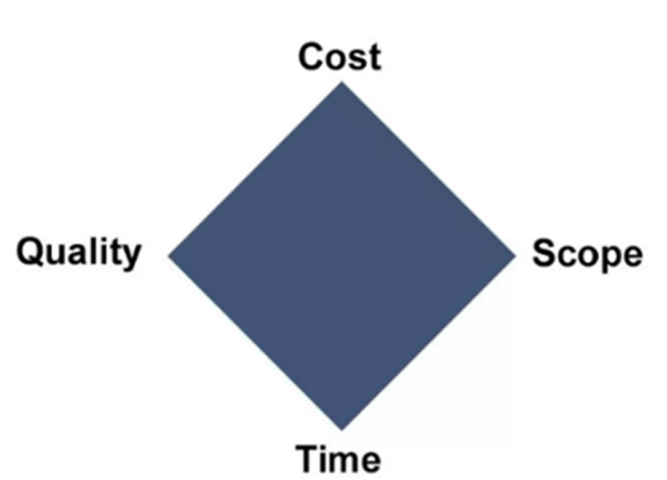
Tyto metody předpokládají, že lze na začátku projektu definovat všechny požadavky, které se pak v průběhu vývoje nemění.

Vodopádový model (Waterfall): Lineární proces, kde každá fáze (Požadavky, Návrh, Implementace, Testování, Nasazení, Údržba) následuje až po úplném dokončení té předchozí. Těžkopádný přístup.

RUP (Rational Unified Process): Robustní metodika, která je sice iterativní, ale stále velmi formální a orientovaná na detailní dokumentaci a UML modely. Iterativní metoda se čtyřmi fázemi:

- Zahájení (ujasnit vizi a cíle)
- Upřesnění (návrh systému)
- Konstrukce (vývoj, testování)
- Přechod do provozu

PRINCE2 (PProjects IN Controlled Environment): Jde o projektový rámec, který dává jasné role, fáze a rozhodovací mechanismy. Dá se použít na velký projekt i malou aplikaci. Projekt dělí na říditelné etapy s měřitelnými výstupy.



Obrázek 1: PRINCE2 - Aspekty řízení projektu

Agilní přístupy

Reagují na fakt, že požadavky zákazníka se v čase mění. Namísto velkého plánu na začátku sázejí na krátké cykly a průběžné dodávání funkčního softwaru.

SCRUM: Nejpoužívanější agilní rámec. Práce probíhá ve sprintech (2-4 týdny). Klíčové jsou role (Scrum Master, Product Owner) a pravidelné schůzky (Daily Stand-up, Retrospektiva). Velice flexibilní, snadné změny během vývoje a rychlá zpětná vazba.

KANBAN: Zaměřuje se na plynulost toku práce (flow) a vizualizaci úkolů na tabuli. Neřeší pevné sprinty, ale soustředí se na omezení rozdělané práce (WIP limit).

XP (Extrémní programování): Zaměřeno na kvalitu kódu a technické praktiky jako párové programování, TDD (vývoj řízený testy) a časté vydávání verzí.

4.4 Komunikace mezi IS

Způsob, jakým si systémy vyměňují data, zásadně ovlivňuje jejich propustnost, stabilitu a míru provázanosti (coupling).

1. Synchronní komunikace (Request-Response) Klient odešle požadavek a čeká na odpověď serveru. Pokud server neodpovídá, klient je blokován.

REST (Representational State Transfer): Dnes nejrozšířenější standard využívající protokol HTTP a formát JSON. Je bezstavový a využívá standardní metody (GET, POST, PUT, DELETE).

SOAP (Simple Object Access Protocol): Starší, robustnější protokol využívající XML. Často se používá v bankovníctví nebo enterprise sféře díky silnému typování a bezpečnosti (WS-Security).

gRPC: Moderní, vysoce výkonný framework od Google. Používá binární formát Protocol Buffers a HTTP/2. Ideální pro interní komunikaci mezi mikroslužbami díky nízké latenci.

2. Asynchronní komunikace (Messaging) Systémy spolu komunikují nepřímo skrze prostředníka (Message Broker). Odesílatel nečeká na zpracování, zpráva je uložena do fronty.

Message Queues (Bod-bod): Zpráva je doručena právě jednomu příjemci. Pokud je příjemců více, rozdělují si práci mezi sebou (např. RabbitMQ).

Publish/Subscribe (Pub/Sub): Odesílatel publikuje zprávu do tématu (Topic) a tu obdrží všichni odběratelé, které dané téma zajímá. Typickým zástupcem je Apache Kafka.

Výhody: Časové rozdělení (systémy nemusí běžet současně), odolnost proti špičkám v zátěži, snadné škálování.

3. Ostatní formy integrace

Webhooks: "Obrácené API". Server pošle HTTP požadavek na předem definovanou URL klienta, jakmile nastane určitá událost (např. proběhnutí platby).

Sdílená databáze: Více systémů přistupuje ke stejné DB. Považováno za antipattern, protože změna schématu v jednom systému rozbije ostatní.

Souborový přenos (Batch processing): Předávání dat v dávkách (např. CSV nebo XML soubory přes SFTP). Často u starších systémů (legacy) nebo pro přenos velkých objemů dat.

WebSockets: Trvalé spojené mezi klientem a serverem. Vhodné pro realtime app (chat, hry).

Enterprise Service Bus (ESB): Middleware platforma pro centralizovanou integraci. Podporuje různé protokoly. Např. Apache Camel, WSO2.

4.5 Automatizace a DevOps

DevOps (spojení slov Development a Operations) je metodika, která bourá bariéry mezi vývojáři a IT správci. Automatizace je zahrnuje build, testování, nasazení, monitoring atd.

CI/CD (Continuous Integration / Continuous Deployment): Odstraňuje manuální kroky při nasazování a zajišťuje tak automatizaci.

- **CI (Průběžná integrace):** Automatizovaný proces, kdy se kód od vývojářů pravidelně slučuje do hlavní větve. Každý "commit" spustí automatické sestavení (build), sadu testů, analýza kódu, a další úkony. Proces, který se snaží udržet všechny (vývojáře, testery . . .) synchronizovány,
- **CD (Průběžné doručování/nasazování):** Navazuje na CI. Automaticky připraví aplikaci k nasazení. U *Deploymentu* jde kód po úspěšných testech rovnou do prostředí bez lidského zásahu.

Klíčové praktiky a nástroje:

- **IaC (Infrastructure as Code):** Správa serverů a sítí pomocí konfiguračních souborů místo manuálního nastavování (např. Terraform, Ansible). Infrastruktura se pak chová jako software (lze ji verzovat).
- **Kontejnerizace:** Balení aplikace a všech jejích závislostí do jednoho balíčku (kontejneru), který běží stejně na počítači vývojáře i na serveru. Dominantním nástrojem je **Docker**.
- **Orchestrace:** Správa velkého množství kontejnerů, jejich škálování a restartování v případě výpadku. Standardem je **Kubernetes**.
- **Monitoring a Logging:** Neustálé sledování výkonu systému a sběr chybových logů (např. ELK stack - Elasticsearch, Logstash, Kibana), aby bylo možné reagovat na problémy dříve, než si jich všimne uživatel.

Výhody DevOps:

- **Rychlost:** Častější vydávání nových funkcí (místo jednou za půl roku klidně několikrát denně).
- **Spolehlivost:** Automatizované testy zachytí chyby dříve, než se dostanou k uživateli.
- **Škálovatelnost:** Díky cloudu a automatizaci lze snadno navyšovat výkon podle aktuální zátěže.

Využití do Cloudu:

- **IaaS (Infrastructure as a Service):** Poskytovatel nabízí základní výpočetní zdroje – virtuální servery, úložiště a síť. Uživatel si sám spravuje operační systém, middleware a aplikace (např. AWS EC2, Microsoft Azure VM).

- **PaaS (Platform as a Service):** Poskytovatel dodává kompletní platformu pro vývoj a běh aplikací. Vývojář se stará pouze o kód a data, nikoliv o správu OS nebo aktualizace runtime prostředí (např. Heroku, Google App Engine).
- **SaaS (Software as a Service):** Koncový software je poskytován přímo přes webový prohlížeč. Uživatel neřeší žádnou technickou stránku, pouze službu využívá (např. Office 365, Salesforce, Gmail).
- **FaaS (Function as a Service / Serverless):** Vývojář nasazuje pouze jednotlivé funkce (kousky kódu), které se spustí v reakci na určitou událost. Platí se pouze za čas běhu dané funkce (např. AWS Lambda, Azure Functions).
- **Modely nasazení (Deployment Models):**
 - *Veřejný cloud:* Sdílená infrastruktura u velkých poskytovatelů (nákladově efektivní).
 - *Privátní cloud:* Vyhrazená infrastruktura pro jednu organizaci (vyšší bezpečnost a kontrola).
 - *Hybridní cloud:* Kombinace obou přístupů – citlivá data v privátním cloudu, zbytek ve veřejném.
 - *Multi-cloud:* Využívání služeb od více poskytovatelů najednou (např. AWS i Azure), aby se předešlo závislosti na jednom dodavateli (vendor lock-in).

5 Webové služby a mikroslužby

význam SOA, WS a mikroslužeb. Principy SOAP vs REST, XML a JSON. Nástroje na tvorbu mikroslužeb

5.1 SOA (Service-Oriented Architecture)

Architektura orientovaná na služby (SOA) je přístup k návrhu podnikového softwaru, kde jsou systémy tvořeny nezávislými, znovupoužitelnými službami. Tyto služby poskytují specifickou business funkcionalitu (např. ověření bonity klienta, fakturace) a komunikují spolu přes síť pomocí standardizovaných protokolů.

Klíčové koncepty:

Služba (Service): Samostatný logický celek, který funguje jako "černá skříňka". Okolní systémy neznají její vnitřní implementaci, znají pouze její veřejné rozhraní (kontrakt).

ESB (Enterprise Service Bus): Centrální komunikační sběrnice. Služby v SOA typicky nekomunikují bod-bod (napřímo), ale všechny se připojují na centrální ESB. Sběrnice zajišťuje směrování zpráv, transformaci formátů (např. z XML starého systému do JSON moderního) a zabezpečení.

Volná vazba (Loose Coupling): Služby jsou navrženy tak, aby na sobě byly co nejméně závislé. Změna v jedné službě by neměla vyžadovat změny ve službách ostatních.

Výhody a nevýhody:

- **Výhody:** Vysoká míra znovupoužitelnosti napříč celým podnikem. Skvělé pro integraci historických (legacy) systémů s novými aplikacemi.
- **Nevýhody:** Sběrnice ESB se při velkém nárůstu služeb stává úzkým hrdlem (bottleneck) a centrálním bodem selhání (SPOF - Single Point Of Failure). Komunikace přes těžkopádné protokoly (tradičně SOAP a XML) generuje velkou režii.

Rozdíl mezi SOA a Mikroslužbami: Ačkoliv oba přístupy dělí systém na menší části, mají odlišnou filozofii:

- **Rozsah a cíl:** SOA se snaží integrovat různé systémy v rámci *celého podniku* (Enterprise level). Mikroslužby se naopak používají pro rozdělení *jedné konkrétní aplikace* (Application level).
- **Způsob komunikace:** SOA spoléhá na "chytrou sběrnici" (Smart ESB), která obsahuje spoustu směrovací logiky. Mikroslužby využívají "hloupé roury" (Dumb pipes – např. REST API nebo jednoduché fronty zpráv) a veškerou chytrou logiku si drží uvnitř samotné služby.
- **Datová vrstva:** Služby v SOA velmi často přistupují ke sdílené obří podnikové databázi. Správně navržená mikroslužba by měla mít vždy svou vlastní, izolovanou databázi.

5.2 WS (Web Services)

Webové služby představují standardizovaný způsob integrace aplikací nezávisle na jejich platformě a programovacím jazyce (např. systém v Javě komunikuje se systémem v .NET). Tradiční WS jsou postaveny na standardech organizace W3C a komunikují výhradně pomocí XML.

Základní stavební kameny:

SOAP (Simple Object Access Protocol): Komunikační protokol. Definuje formát zprávy zvaný *SOAP Envelope* (obálka), která obsahuje hlavičku pro metadata (bezpečnost, transakce) a tělo pro samotná byznysová data.

WSDL (Web Services Description Language): Detailní a striktní popis rozhraní (kontrakt) ve formátu XML. Přesně definuje, jaké operace (metody) služba nabízí a jaké datové typy přijímá a vrací.

UDDI (Universal Description, Discovery, and Integration): Adresář, do kterého mohou být služby registrovány, aby je ostatní systémy mohly dynamicky vyhledat. (V dnešní praxi se již téměř nepoužívá, ale je součástí definice).

Srovnání Web Services (SOAP) vs. REST API:

- **Formát dat:** WS používá striktně XML. REST může používat cokoliv, nejčastěji odlehčený JSON.
- **Přístup:** WS je orientováno na *akce/operace* (např. metoda `ZalozUcet()`). REST je orientován na *zdroje/entity* a využívá standardní HTTP metody (POST na entitu `/ucet`).
- **Kontrakt a typování:** WS má striktní smlouvu (WSDL). Kód se z WSDL generuje automaticky. REST je volnější (i když dnes využívá např. standard OpenAPI/Swagger pro dokumentaci).
- **Výkon a rezie:** Zprávy ve WS jsou kvůli struktuře XML mnohem větší a náročnější na zpracování. REST je mnohem rychlejší a vhodnější pro web a mobilní zařízení.

5.3 Mikroslužby

Předchozí kapitola: [Mikroslužby](#)

Každá služba zajišťuje jednu konkrétní funkcionalitu. Nejčastěji se využívá REST API ke komunikaci. Usnadňují škálování, vývoj, nasazení. Jsou odolnější vůči výpadku a dovolují využití různých technologií.

5.4 SOAP vs REST

Výhodou SOAP je robustnost a standardizace na druhé straně REST je jednoduchý a flexibilní. SOAP se typicky používá v rámci SOA pro složité a formálně definované webové služby. REST nepoužívá žádný specifický protokol, ale staví na principu http.

SOAP – Simple Object Access Protocol

REST – Representational State Transfer

Vlastnost	SOAP	REST
Protokol	Vlastní protokol (nad HTTP/SMTP)	Architektonický styl nad HTTP
Formát zpráv	Pouze XML	JSON, XML, HTML, text
Stavovost	Může být stavový i bezstavový	Striktně bezstavový (Stateless)
Bezpečnost	WS-Security (vysoká, na úrovni zprávy)	HTTPS (na úrovni transportu)
Výkon	Nižší (režie XML obálky)	Vysoký (minimalistický přenos)
Definice	WSDL (striktní kontrakt)	OpenAPI/Swagger (dokumentace)

Srovnání datových formátů (XML vs. JSON):

XML (eXtensible Markup Language): Jazyk založený na značkách (tagy). Je hierarchický a velmi ukecaný.

- **Silné stránky:** Podpora schémat (XSD) pro validaci dat, podpora jmenných prostorů (namespaces), samopopisné značky.
- **Slabé stránky:** Velký objem dat (nadbytečné tagy), složitější parsování pro prohlížeče a mobilní zařízení.

JSON (JavaScript Object Notation): Odlehčený formát založený na párech klíč–hodnota.

- **Silné stránky:** Minimální režie (malá velikost souborů), extrémně rychlé parsování v JavaScriptu, snadná čitelnost pro člověka.
- **Slabé stránky:** Chybí nativní podpora pro metadata a složité datové typy (např. datum je často jen řetězec), méně robustní validace oproti XSD.

Kdy co zvolit?

- **SOAP + XML:** Bankovní systémy, platební brány, státní správa. Tam, kde je prioritou bezpečnost, transakčnost a kde se rozhraní nemění desítky let.
- **REST + JSON:** Veřejná API, mobilní aplikace, webové fronty, mikroslužby. Tam, kde je prioritou rychlost vývoje, snadná konzumace dat a výkon.

5.5 Nástroje na tvorbu mikroslužeb

1. Frameworky:

- **Spring Boot (Java):** Nejpoužívanější framework v korporátní sféře. Nabízí moduly pro snadné vytvoření REST API a integraci s cloudovými službami (Spring Cloud).
- **Node.js / NestJS (JavaScript/TypeScript):** Populární díky asynchronnímu I/O modelu, což je ideální pro vysoce zatížená komunikační rozhraní.
- **Go (Golang):** Jazyk od Google navržený přímo pro cloudové služby. Vyniká rychlostí, nízkou spotřebou paměti a skvělou podporou souběžnosti.

2. Infrastruktura a orchestrace: Mikroslužby by bez automatizované správy nebylo možné provozovat v desítkách či stovkách instancí.

- **Docker:** Každá mikroslužba má svůj vlastní kontejner, který obsahuje vše potřebné k běhu.
- **Kubernetes (K8s):** Stará se o to, aby mikroslužby běžely, automaticky je škáluje při zátěži a restartuje při chybě.
- **Service Mesh (např. Istio):** Nadstavba nad Kubernetes, která řeší složitou síťovou komunikaci mezi službami (šifrování provozu, řízení toků, sledování latence).

3. Správa konfigurace a Service Discovery: V dynamickém prostředí se IP adresy služeb neustále mění, proto jsou potřeba nástroje pro jejich nalezení.

- **Service Registry (např. Consul, Netflix Eureka):** Telefonní seznam služeb. Každá nově spuštěná služba se zde nahlásí, aby ostatní věděli, kde ji najít.
- **API Gateway (např. Kong, NGINX):** Jediný vstupní bod pro klienty zvenčí. Směruje požadavky na správné vnitřní mikroslužby, řeší autentizaci a omezování zátěže (rate limiting).

4. Monitoring a distribuované trasování: Když nastane chyba v řetězci deseti služeb, je těžké ji najít bez specializovaných nástrojů.

- **Prometheus & Grafana:** Nástroje pro sběr a vizualizaci metrik (např. vytížení CPU, počet požadavků za sekundu).
- **ELK stack (ElasticSearch, Logstash, Kybana):** Sběr, transformace a vizualizace logů.
- **Jaeger / Zipkin:** Nástroje pro distribuované trasování. Umožňují sledovat cestu jednoho konkrétního požadavku napříč všemi mikroslužbami, kterými prošel.

6 NoSQL databáze a jejich základní principy

škálování, replikace, možnosti distribuce dat, Map&Reduce, sharding, perzistence dat, zabezpečení, CAP teorém, BASE model

[Zpět](#) (Kapitola 4.)

NoSQL (Not Only SQL) databáze vznikly jako reakce na limity tradičních relačních databází při zpracování obrovských objemů nestrukturovaných dat a potřebě horizontálního škálování. Upřednostňují výkon dostupnost a flexibilitu před integritou a transakčností. Zvládají rychlý zápis a čtení, které by klasické RDBMS nezvládalo.

6.1 Škálování

Škálování je schopnost systému zvládat rostoucí zátěž (počet uživatelů, objem dat) navýšením dostupných prostředků.

1. Vertikální škálování (Scale-Up) Znamená navýšení výkonu stávajícího serveru - přidání více CPU, RAM nebo rychlejších disků.

- **Výhody:** Jednoduchost (aplikace se nemusí měnit, stále běží na jednom stroji), žádná režie na síťovou komunikaci mezi uzly.
- **Nevýhody:** Existuje pevný hardwarový strop (limit základní desky), každé další navýšení výkonu je extrémně drahé a obvykle vyžaduje odstávku systému pro upgrade.

2. Horizontální škálování (Scale-Out) Znamená přidávání dalších, levnějších strojů (uzlů) do clusteru. Zátěž se pak rozděluje mezi ně.

- **Výhody:** Prakticky neomezený růst, vysoká dostupnost (když jeden uzel selže, ostatní pracují dál), možnost využít levnější hardware.
- **Nevýhody:** Vyšší komplexnost - systém musí být navržen jako bezstavový (stateless), je nutné řešit synchronizaci dat a použít **Load Balancer** (rozdělovač zátěže).

6.2 Replikace a sharding

Replikace zajišťuje redundanci dat, a tak existuje v případě ztráty nebo poškození záložní kopie. Sharding je rozdělení dat na více uzlů, což je základní princip horizontálního škálování.

1. Replikace Proces udržování stejných dat na více serverech současně.

- **Master-Slave (Leader-Follower):** Jeden uzel je hlavní (Master) a přijímá zápisy. Ostatní uzly (Slaves) kopírují jeho data a slouží pouze pro čtení.
 - *Výhoda:* Extrémní zrychlení čtení (typické pro weby - hodně se čte, málo píše).
 - *Nevýhoda:* Zapisovat může pouze Master.
 - Např. Mongo
- **Multi-Master:** Zápis je možný do libovolného uzlu.
 - *Výhoda:* Všechny uzly mohou číst i zapisovat.
 - *Nevýhoda:* Velmi složité řešení konfliktů (co když dva lidé změní stejný řádek na různých serverech ve stejnou vteřinu).
 - Např. Cassandra
- **Hlavní přínos:** Vysoká dostupnost (Availability) - když jeden server shoří, data máme jinde.
- **Synchronní/asynchronní:** Synchronní zápis je potvrzen až po zapsání do všech uzlů. Asynchronní zápis je potvrzen ihned, replikace proběhne později.

2. Sharding (Horizontální dělení) Rozdělení jedné obří tabulky na několik menších částí (shardy), které jsou fyzicky uloženy na různých serverech. Žádný server pak nemá kompletní kopii všech dat.

- **Shard Key:** Klíč (např. ID zákazníka nebo region), podle kterého se určuje, na kterém serveru data leží. Např. uživatelé z Evropy jsou na serveru A, uživatelé z USA na serveru B.
- **Výhody:** Umožňuje lineární růst databáze. Zápis se rozprostře mezi více strojů, takže neexistuje jeden bottleneck.
- **Nevýhody:** Velká komplexnost. Dotazy přes více shardů (JOIN) jsou velmi pomalé a náročné na režii.

Možnosti distribuce dat: Označuje způsob jakým jsou data uložena na více fyzických nebo logických uzlech.

- **HashBased** – Data jsou distribuována podle výstupu hashovací funkce nad klíčem což zajišťuje rovnoměrné rozložení.
- **RangeBased** – Data jsou rozdělena podle hodnoty klíče do určitých rozsahů.

- **Geografická distribuce** Každý uzel je v jiném datacentru nebo geografické oblasti. Ukládá data ze své oblasti.

6.3 Map&Reduce

MapReduce je programovací model pro distribuované zpracování obrovských objemů dat (Big-Data) na velkých clusterech. Tento koncept zpopularizoval Google a je základem technologií jako Apache Hadoop.

Princip fungování: Celý proces je rozdělen do dvou hlavních fází, mezi kterými probíhá automatické míchání dat:

1. **Map (Mapování):** Vstupní data jsou rozdělena na menší kusy (případně již jsou uložena na různých uzlech) a každý uzel zpracuje svou část a vygeneruje dvojice **klíč–hodnota** (key-value pairs).
2. **Shuffle and Sort (Míchání):** Systém automaticky seskupí všechny hodnoty se stejným klíčem k sobě a pošle je na stejný uzel pro závěrečné zpracování.
3. **Reduce (Redukce):** Uzly přijmou seskupená data pro daný klíč a provedou agregační operaci (např. součet, průměr, výběr maxima), čímž vznikne finální výsledek.

Příklad (Počítání slov - Word Count): Máme miliony dokumentů a chceme zjistit počet výskytů každého slova:

- **Map:** Každý stroj čte text a pro každé slovo "ahoj" vytvoří dvojici (ahoj, 1). (číslo je počet nálezů slova)
- **Shuffle:** Všechny dvojice (ahoj, *) se "sletí" na jeden konkrétní stroj.
- **Reduce:** Tento stroj sečte všechny jedničky u klíče "ahoj" a vypíše výsledek (ahoj, 1548).

Výhody a technologie:

- **Škálovatelnost:** Stačí přidat další servery a výkon roste lineárně.
- **Odolnost proti chybám:** Pokud jeden uzel během výpočtu selže, systém jeho práci automaticky přidělí jinému stroji.

6.4 Perzistence dat a zabezpečení

Perzistence dat a distribuovaný přístup: Zatímco relační databáze striktně dodržují pravidla ACID (Atomicita, Konzistence, Izolace, Trvalost), distribuované NoSQL databáze se často řídí **teorémem CAP** (Konzistence, Dostupnost, Odolnost proti rozdělení sítě) a principy **BASE** (Basically Available, Soft state, Eventual consistency). To znamená, že systém raději upřednostní dostupnost a rychlost před okamžitou konzistencí napříč všemi uzly. Mechanismy perzistence v NoSQL lze rozdělit do několika přístupů:

- **In-memory s asynchronním zápisem:** Databáze (např. Redis) drží data primárně v operační paměti (RAM) pro maximální rychlost. Perzistence je řešena občasným vytvořením snímku (Snapshotting) na pevný disk nebo průběžným logováním každé operace (Append-Only File).
- **Write-Ahead Logging (WAL):** Předtím, než se data zapíší do samotné datové struktury, jsou nejprve sekvenčně zaznamenána do transakčního logu na disku. Tím je zaručeno, že v případě výpadku proudu se data neztratí a systém je z logu obnoví (využívá např. Cassandra nebo MongoDB).

Zabezpečení: Historicky byly NoSQL databáze navrhovány s primárním důrazem na výkon a škálovatelnost. Předpokládalo se, že poběží v uzavřených a důvěryhodných vnitřních sítích, a proto v počátcích často postrádaly robustní zabezpečení (např. výchozí nastavení nevyžadovalo heslo). Moderní NoSQL systémy však již implementují komplexní bezpečnostní vrstvy:

- **Autentizace a Authorizace:** Moderní systémy využívají standardní mechanismy pro ověření identity (SCRAM, integrace s LDAP nebo Kerberos). Pro řízení přístupu se standardně používá **RBAC** (Role-Based Access Control) – uživatelům nebo aplikacím jsou přidělovány role s přesně definovanými oprávněními (např. pouze čtení z konkrétní kolekce).
- **Šifrování (Encryption):**
 - **In-transit (při přenosu):** Komunikace mezi klientem a databází, stejně jako mezi samotnými uzly v clusteru, je šifrována pomocí TLS/SSL protokolů, aby se zabránilo odposlechu (man-in-the-middle útokům).
 - **At-rest (v úložišti):** Data fyzicky uložená na pevných discích jsou šifrována (např. pomocí standardu AES-256). K datům nelze přistoupit pouhým zkopírováním databázových souborů bez šifrovacího klíče.
- **Síťová izolace a Auditing:** Databázové uzly jsou chráněny firewally a přístup k nim je povolen pouze z aplikačních serverů (nikoliv z veřejného internetu). Většina enterprise verzí NoSQL databází navíc podporuje podrobné auditování, které zaznamenává, kdo, kdy a k jakým datům přistupoval.

6.5 CAP teorém a BASE model

Tradiční relační databáze se spoléhají na vlastnosti ACID (Atomicita, Konzistence, Izolace, Trvalost), které zaručují absolutní spolehlivost transakcí. V prostředí obrovských distribuovaných systémů (BigData) je však striktní dodržování ACID velmi náročné na výkon a často nemožné. Proto NoSQL databáze využívají odlišné teoretické modely – CAP teorém a principy BASE.

CAP teorém (Brewerův teorém): Tento teorém definuje tři základní vlastnosti distribuovaných systémů. Podle něj může distribuované datové úložiště současně garantovat **nejvýše dvě** z těchto tří vlastností:

C – Consistency (Konzistence): Každý požadavek na čtení vrátí buď nejnovější zapsaná data, nebo chybu. Všichni uživatelé vidí v daný okamžik naprosto stejná data, nezávisle na tom, ke kterému uzlu (serveru) jsou připojeni.

A – Availability (Dostupnost): Každý požadavek, který systém přijme, dostane platnou odpověď (bez chybové hlášky). Není ale zaručeno, že tato odpověď obsahuje nejnovější verzi dat.

P – Partition Tolerance (Odolnost proti rozdělení sítě): Systém nadále funguje i v případě, že dojde k výpadku komunikace (rozdělení sítě) mezi jednotlivými uzly.

Volba v praxi: Ačkoliv se často říká, že si „vybíráme dvě ze tří“, v reálném světě jsou výpadky sítě (P) nevyhnutelné. Distribuovaný systém proto *musí* být odolný proti rozdělení (Partition Tolerant). Skutečná volba při návrhu NoSQL databáze tedy spočívá v tom, co systém udělá při výpadku sítě:

- **CP systémy (Konzistence + Odolnost):** Při výpadku sítě systém raději odmítne odpovědět (obětuje dostupnost), aby nevrátil zastaralá data (např. MongoDB, HBase).
- **AP systémy (Dostupnost + Odolnost):** Systém odpoví vždy, i za cenu, že klientům pošle starší data, protože uzly se nestihly synchronizovat (např. Cassandra, CouchDB).

Model BASE: Zatímco ACID je model pro relační databáze, NoSQL databáze (zejména ty typu AP) se řídí modelem BASE. Jde o akronym tří konceptů, které upřednostňují dostupnost před okamžitou konzistencí:

BA – Basically Available (Základní dostupnost): Systém garantuje vysokou dostupnost dat. I když selže jeden uzel nebo část sítě, databáze jako celek stále odpovídá na požadavky uživatelů.

S – Soft state (Měkký/proměnlivý stav): Vzhledem k chybějící garanci okamžité konzistence se stav systému může měnit v čase, a to i v případě, že nepřicházejí žádné nové požadavky na zápis (uzly se totiž synchronizují asynchronně na pozadí).

E – Eventual consistency (Konečná konzistence): Systém negarantuje, že data budou konzistentní okamžitě po zápisu. Garantuje však, že pokud do systému přestanou přicházet nové zápisy, po uplynutí určité doby se všechny uzly zkoordinují a data budou napříč systémem konzistentní.

7 Data v distribuovaných systémech

BigData a jejich zdroje, dataset a jeho životní cyklus, otevřená data, trend velkých datasetů, 5V, typy BigData a datové formáty, problémy BigData

7.1 BigData

Velké, rychle vznikající různorodé objemy dat, které není možné efektivně zpracovávat tradičními metodami a nástroji. Typicky několik TB až PB. Ve skutečnosti neexistuje pevná definice toho, co jsou BigData, jedná se o buzzword, pojem se stále vyvíjí, rychle se mění.

Gartner: „BigData“ jsou informační zdroje charakterizované velkým objemem, vysokou rychlostí a/nebo velkou rozmanitostí, které vyžadují nové způsoby zpracování, aby umožnily lepší rozhodování, získávání poznatků a optimalizaci procesů.

Zdroje:

- Senzory a IoT zařízení
- Web a mobilní aplikace – logy, pohyby myši, interakce, data o chování uživatelů
- Sociální sítě – texty, obrázky, videa
- Finanční a obchodní transakce – platby, objednávky, interakce v eshopu
- Veřejné zdroje a otevřená data
- Zdravotnictví a biomedicína
- Multimediální obsah

7.2 Dataset

Dataset (datová sada) je strukturovaná kolekce souvisejících dat, obvykle uspořádaná do tabulkové podoby (kde řádky reprezentují jednotlivé záznamy a sloupce jejich atributy), případně do podoby textových korpusů, grafů či složek s multimédií. Technicky jsou datasety uloženy v různých formátech, od jednoduchých souborů (CSV, JSON) až po komplexní relační či NoSQL databáze.

Životní cyklus datasetu

Práce s daty se řídí procesem, který pokrývá jejich existenci od vzniku po zánik. Hlavní fáze zahrnují:

- **Sběr (monitoring, scraping):** Získávání surových dat z různých zdrojů (uživatelské vstupy, senzory, web scraping, logy aplikací).
- **Organizace:** Organizace dat do datasetu. Řeší se například jmenné konvence, čištění, předspracování.
- **Ukládání:** Zabezpečené uložení dat do databází nebo jiným způsobem.
- **Prohledávání:** Průzkum dat například dotazy nad databází.
- **Sdílení:** Sdílení dat ostatním, např. registry otevřených dat.
- **Analýza a využití:** Vytěžování informací (Data Mining), podpora rozhodování (BI) nebo trénování modelů strojového učení (ML). Např. Grafana, Dataprep.
- **Vizualizace:** Prezentace výsledků získaných z dat. Např. dashboardy (Datastudio).
- **Archivace či zničení:** Odložení historických dat z důvodu úspory drahého úložiště nebo bezpečné smazání dat na základě legislativních požadavků (např. expirace souhlasu dle GDPR).

7.3 Otevřená data (Open Data)

Otevřená data jsou informace a datasety bezplatně zpřístupněné veřejnosti k volnému využití (bez licenčních či autorských omezení). Aby data byla skutečně „otevřená“, musí být publikována ve **strojově čitelném formátu** (např. CSV nebo XML, nikoliv skenovaný obrázek v PDF) a nesmí být vázána na proprietární software. Cílem otevřených dat je podpora transparentnosti (např. veřejné rozpočty, jízdní řády v Národním katalogu otevřených dat v ČR) a podpora inovací, neboť na nich mohou nezávislí vývojáři stavět nové služby a aplikace. [Web](#) na vysvětlení úrovně.

7.4 Trend velkých datasetů (BigData)

V současnosti čelíme exponenciálnímu nárůstu produkce dat. Zpracování velkých datasetů vyžaduje využití cloudu a distribuovaných výpočetních systémů (např. Apache Hadoop, Spark). Obrovské datasety jsou v současnosti naprosto klíčové pro trénování modelů umělé inteligence, což však přináší i nové výzvy v oblasti kvality dat, biasu (zaujatosti) a ochrany soukromí. Data se také stala komoditou.

- Spojování malých datasetů do velkých.
- Vyšší nároky na výkon DB.
- Objevení i zdánlivě neexistujících korelací.
- Killer-feature pro business intelligence!
- Klíčovými aspekty platformy BigData jsou integrace, analýzy, vizualizace a zabezpečená správa.

7.5 Koncept 5V

Původní charakteristika velkých dat (Volume, Velocity, Variety) se dnes standardně rozšiřuje o další dvě dimenze (Veracity, Value), které definují úspěšnost práce s BigData:

- **Volume (Objem):** Obrovské množství dat (terabyty až petabyty), které již nelze zpracovat na běžných počítačích či v tradičních databázích.
- **Velocity (Rychlost):** Nutnost data sbírat a analyticky zpracovávat v reálném čase (např. IoT senzory, detekce bankovních podvodů).
- **Variety (Rozmanitost):** Drtivá většina moderních dat je nestrukturovaná (obrázky, videa, příspěvky ze sociálních sítí, audio nahrávky).
- **Veracity (Věrohodnost/Pravdivost):** Týká se kvality a spolehlivosti dat. Při obrovských objemech data často obsahují šum, nekonzistence, duplicity nebo chyby. Pokud je špatný vstup, je špatný i výstup analýzy. Popisuje reálnou shodu s fakty a přesností.
- **Value (Hodnota):** Data sama o sobě nemají žádnou cenu. Hodnota představuje schopnost vytěžit z masivních datasetů užitečné znalosti, které přinesou byznysovou výhodu nebo zefektivní procesy.

7.6 Typy BigData a datové formáty

Z hlediska formálního uspořádání dělíme data do tří základních kategorií:

- **Strukturovaná data:** Mají přesně definovaný datový model a schéma. Lze je snadno ukládat a dotazovat pomocí SQL. *Formáty: CSV, tabulky v relačních databázích (MySQL, PostgreSQL).*
- **Polostrukturovaná data:** Nemají rigidní tabulkovou strukturu, ale obsahují hierarchii, klíče nebo značky (tagy) pro oddělení datových prvků. *Formáty: JSON, XML, YAML (typické pro webová API a NoSQL databáze).*
- **Nestrukturovaná data:** Tvoří zhruba 80 % všech moderních dat. Nemají předem definovaný model a nelze je snadno tabulkově prohledávat. Analýza často vyžaduje nástroje umělé inteligence. *Příklady: textové dokumenty (PDF), e-maily, obrázky (JPEG), video (MP4), audio stopy.*

7.7 Problémy BigData

- **Ukládání a škálování:** Není možné využívat centrální servery. Nutnost horizontálního škálování (distribuované systémy a cloudy), což razantně zvyšuje složitost architektury. Pro dotazování v databázi: cacheování, indexace
- **Integrace dat:** Obtížné propojování dat (Data Integration) z izolovaných systémů (tzv. datová sila), které navíc využívají zcela odlišné formáty a struktury.
- **Bezpečnost a ochrana soukromí:** Velká úložiště jsou lákavým cílem kyberútoků. Sběr osobních dat naráží na přísnou legislativu (GDPR), což vyžaduje spolehlivou anonymizaci či pseudonymizaci.
- **Výkon a latence:** IoT zařízení mají nízký výkon. Neustálý přenos dat po síti může síť přetížit. Nutná optimalizace přenosu.

8 Struktura a architektura systému UNIX / GNU / Linux

struktura systému, systém souborů, systém procesů, služby OS, shelly, příkazy shellu, popis jádra, datové struktury jádra, systém vyrovnávací paměti, subsystém V/V, správa paměti, operační systémy reálného času (základní charakteristika, hlavní faktury, definice, hard a soft RTOS, RMS, EDF, příklady RTOS)

- **UNIX:** Komerční operační systém vytvořený v 70. letech v Bellových laboratořích (AT&T). Přinesl revoluční myšlenky: víceúlohovost (multitasking), víceuživatelský přístup a filozofii „vše je soubor“. Původní kód UNIXu je uzavřený (proprietární) a dnes z něj vycházejí např. systémy macOS nebo FreeBSD.
- **GNU (GNU's Not Unix):** Projekt založený v roce 1983 (Richard Stallman) s cílem vytvořit svobodný (open-source) operační systém, který by se choval jako UNIX, ale neobsahoval jeho komerční kód. Projekt vytvořil klíčové uživatelské nástroje (příkazový řádek bash, překladáč GCC, základní utility), ale chybělo mu funkční jádro.
- **Linux:** Jádro (kernel) operačního systému, které v roce 1991 napsal Linus Torvalds. Spojením jádra **Linux** a uživatelských nástrojů **GNU** vznikl plně funkční a svobodný operační systém, který se dnes zkráceně, označuje jako „Linux“ (správně by mělo být GNU/Linux).

8.1 Struktura systému

Linux využívá monolitické jádro a striktně odděluje práva hardwaru a běžných uživatelů. Architektura se dělí do několika vrstev:

- **Hardware:** Fyzické komponenty (CPU, RAM, disky, síťové karty).
- **Jádro (Kernel Space):** Běží v privilegovaném režimu. Řídí přístup k hardwaru, spravuje paměť, plánuje procesorový čas pro aplikace a obsahuje ovladače zařízení. Jádro je program, který je zaveden po zapnutí počítače do operační paměti.
- **Systémová volání (System Calls):** Rozhraní (API) mezi jádrem a uživatelským prostorem. Pokud chce běžná aplikace zapsat soubor na disk, nesmí sáhnout přímo na HW, ale musí o to požádat jádro právě přes systémové volání.
- **Uživatelský prostor (User Space):** Neprivilegovaný režim, ve kterém běží knihovny (např. glibc), příkazový interpret (Shell), služby na pozadí (démoni), grafické prostředí (X11/Wayland) a samotné uživatelské aplikace.

8.2 Systém souborů (File System)

Základním principem je, že „**vše je soubor**“ - jako soubor se v Linuxu tváří textové dokumenty, složky, ale i hardwarová zařízení (např. pevný disk jako /dev/sda).

- **Hierarchie:** Vše vychází z jednoho kořenového adresáře označovaného lomítkem (/), další disky se připojují (mountují) do prázdných složek v rámci této jedné hierarchie.
- **I-uzly (I-node):** Datová struktura, která nese metadata o souboru (vlastník, velikost, přístupová práva, časy modifikace, typ souboru a hlavně ukazatele na fyzické bloky na disku, kde jsou data skutečně uložena). I-uzel neobsahuje název souboru - ten je uložen pouze v adresáři.
- **Odkazy (Linky):**
 - *Tvrdý odkaz (Hard link):* Více názvů souborů ukazuje na stejný i-uzel. Data se smažou z disku až po smazání posledního tvrdého odkazu.
 - *Symbolický odkaz (Symlink):* Obdoba zástupce ve Windows. Soubor, který obsahuje pouze cestu k jinému souboru.
- **Oprávnění:** Každý soubor má vlastníka (User) a skupinu (Group). Práva se dělí na čtení (r), zápis (w) a spouštění (x) pro vlastníka, skupinu a všechny ostatní (Others).

Základní typy souborů

- **Běžný soubor** (značí se pomlčkou -): Nejobvyklejší typ. Patří sem textové dokumenty, zdrojové kódy, obrázky, komprimované archivy i spustitelné binární programy.
- **Adresář** (značí se d - Directory): Z pohledu jádra jde o *speciální soubor*, který neobsahuje běžná data, ale pouze tabulku. V této tabulce jsou zapsány názvy souborů (které se v adresáři nacházejí) a k nim přiřazená čísla i-uzlů.
- **Symbolický odkaz** (značí se l - Link): Soubor obsahující pouze textovou cestu k jinému cílovému souboru (obdoba zástupce).
- **Soubory zařízení (Device files):** Slouží k přímé komunikaci s hardwarem. Nacházejí se typicky ve složce /dev. Dělí se na dva druhy:
 - *Znaková zařízení (značí se c - Character):* Přenáší data znak po znaku (proud bajtů) bez vyrovnávací paměti. Např. klávesnice, myš, terminál.
 - *Bloková zařízení (značí se b - Block):* Přenáší data ve větších blocích (např. 4 KB) přes vyrovnávací paměť. Např. pevné disky (/dev/sda) nebo USB flash disky.

- **Pojmenovaná roura** (značí se p - Pipe / FIFO): Speciální soubor sloužící k jednosměrné komunikaci mezi dvěma nezávislými procesy.
- **Soket** (značí se s - Socket): Speciální soubor umožňující obousměrnou pokročilou komunikaci mezi procesy, a to lokálně nebo přes síť.

Zpět na [subsystém V/V](#).

8.3 Systém procesů

Proces je běžící instance programu. Linux je plně víceúlohový, takže procesor přepíná mezi stovkami procesů (Context switching). Všechny procesy jsou zapsány v tabulce procesů.

- **Atributy procesu:** Každý proces má unikátní identifikátor **PID** (Process ID), identifikátor rodiče **PPID** (Parent PID) a práva uživatele (**UID** - uživatel, **GID** - skupina), který jej spustil.
- **Vznik procesu:** Vytvoření nového procesu probíhá ve dvou krocích. Voláním `fork()` vytvoří rodičovský proces svou přesnou kopii (potomka). Potomek následně použije volání `exec()`, kterým svůj obsah nahradí novým programem.
- **Init proces (systemd):** První proces spuštěný při startu jádra. Má vždy **PID 1** a je přímým nebo nepřímým rodičem všech ostatních procesů v systému.
- **Meziprocesová komunikace (IPC):** Procesy si mohou vyměňovat data nebo se synchronizovat. Využívají se k tomu *roury* (Pipes - znak |, přesměrování výstupu jednoho programu na vstup dalšího), signály (např. `SIGKILL` pro okamžité ukončení procesu) nebo sdílená paměť.
- **/proc souborový systém:** Pseudo-souborový systém poskytující rozhraní k interním údajům o běžících procesech a systému. (`/proc/cpuinfo` - informace o CPU, `/proc/meminfo`, `/proc/uptime`)

Stavy procesů

- Běžící nebo připravený běžet
- Spící připravený na probuzení signálem
- Čeká bez možnosti přerušení signálem
- Zastavený
- Zombie

Datová struktura procesu (`task_struct`)

V jádře Linuxu je každý proces (i vlákno) interně reprezentován strukturou zvanou **`task_struct`**. Jádro si tyto struktury udržuje v paměti a obsahuje v nich naprosto vše, co o procesu potřebuje vědět:

- **Stav procesu:** [Stavy procesu](#)
- **Identifikátory:** `pid` (číslo procesu), `tgid` (číslo skupiny vláken) a údaje o vlastníkovi (UID/GID) pro ověření oprávnění.
- **Rodokmen:** Ukazatele na rodičovský proces (`parent`) a seznam potomků (`children`).
- **Správa paměti (`mm_struct`):** Odkaz na datovou strukturu definující virtuální adresní prostor procesu (kde má proces kód, data a zásobník).
- **Otevřené soubory (`files_struct`):** Tabulka tzv. *file deskriptorů* (ukazatele na konkrétní soubory, roury či síťové sokety, které má proces zrovna otevřené).
- **Plánování a kontext CPU:** Priorita procesu a paměťový prostor, kam jádro uloží stavy registrů procesoru při přepínání úloh (Context Switch). Číselné statistiky např. kolik času strávil v uživatelském / jádru. Také `thread_info` s informacemi o kontextu při přepínání vláken.

Zpět na [Datové struktury jádra](#).

8.4 Služby OS

Služby operačního systému jsou základní softwarové funkce, které zajišťují chod počítače, správu hardwaru a poskytují prostředí pro spouštění aplikací.

- Správa procesů
- Správa paměti
- Správa souborového systému
- Správa zařízení
- Správa uživatelů a práv
- Síťové služby
- Synchronizace a komunikace mezi procesy

- Cachování diskového přístupu
- Virtuální paměť se stránkováním do tzv. odkládacího prostoru swap space
- Ukládání obsahu paměti na disk core dump
- Pravý hardwarový paralelismus
- Stránkování na vyžádání

8.5 Shelly

Shell (příkazový interpret, skutečný programovací jazyk) je textové uživatelské rozhraní k OS. Čte příkazy zadané uživatelem (nebo ze skriptu), interpretuje je a spouští příslušné programy.

Běžné typy shellů

- **sh (Bourne shell):** Historický základ, původní shell systému UNIX.
- **bash (Bourne Again shell):** Výchozí a nejrozšířenější shell ve většině linuxových distribucí.
- **zsh, fish, csh, a další:** Moderní shelly s pokročilým našeptáváním, obarvováním syntaxe a bohatými možnostmi přizpůsobení.

Typy příkazů

- **Interní (vestavěné) příkazy:** Jsou přímou součástí samotného shellu. Při jejich spuštění nevzniká nový proces. Příklad: `cd` (změna adresáře), `echo`, `export` (nastavení proměnné).
- **Externí příkazy:** Jsou to samostatné binární soubory uložené na disku. Shell je hledá v adresářích definovaných v systémové proměnné **\$PATH**. Při spuštění shell vytvoří nový proces (přes `fork` a `exec`). Příklad: `ls`, `grep`, `cat`.

Standardní proudy a přesměrování

Každý spuštěný proces v Linuxu má automaticky otevřené tři datové proudy (soubory): **stdin** (0 - standardní vstup, typicky klávesnice), **stdout** (1 - standardní výstup, obrazovka) a **stderr** (2 - chybový výstup, obrazovka). Shell s nimi umí manipulovat:

- **Přesměrování výstupu:** Znak `>` (uloží výstup do souboru, původní obsah přepíše), znaky `>>` (připojí výstup na konec souboru).

- **Roury (Pipes):** Znak `|`. Propojuje procesy – standardní výstup prvního příkazu se stává standardním vstupem druhého (např. `ls -l | grep "txt"` vypíše jen soubory obsahující "txt").

Přehled nejpoužívanějších příkazů

- **Práce se soubory/adresáři:** `ls` (výpis), `cd` (změna složky), `pwd` (aktuální cesta), `cp` (kopírování), `mv` (přesun/přejmenování), `rm` (mazání).
- **Zpracování textu:** `cat` (výpis obsahu souboru), `grep` (filtrování textu dle vzoru/regexu), `less` (stránkování textu).
- **Správa procesů:** `ps` (výpis běžících procesů), `top` / `htop` (interaktivní sledování zátěže), `kill` (zaslání signálu procesu, typicky pro ukončení).
- **Oprávnění:** `chmod` (změna práv r/w/x), `chown` (změna vlastníka).

8.6 Jádru OS

Jádru (neboli kernel) je programové vybavení, které pracuje na počítači bez jakékoliv další programové podpory, představuje množinu obslužných programů trvale umístěných v paměti, k nimž má každý proces přístup, základem operačního systému a odpovídá za správu všech hardwarových prostředků počítače, jako jsou disky, síťová rozhraní atd. Je velkým rozhraním mezi uživatelem a technickým vybavením počítače. Udržuje a podporuje v činnosti systém souborů a běžící procesy.

Popis jádra

- **Monolitická architektura:** Celé jádro (včetně většiny ovladačů a souborových systémů) běží v jednom velkém paměťovém bloku. To zajišťuje maximální výkon.
- **LKM (Loadable Kernel Modules):** Přestože je Linux monolitický, je *modulární*. Ovladače hardwaru lze do běžícího jádra dynamicky nahrávat a odebírat za běhu (příkazy `modprobe`, `lsmod`), aniž by bylo nutné systém restartovat.

Rozdíl mezi monolitickým jádrem a mikrojádrem

Základní rozdíl spočívá v tom, kolik služeb běží v privilegovaném režimu (Kernel Space).

- **Monolitické jádro (např. Linux):** Všechny služby OS (správa paměti, souborové systémy, ovladače HW) tvoří jeden velký celek a běží v kernel space.
 - *Výhoda:* Maximální výkon a rychlost (komunikace probíhá přímo v paměti bez režie).
 - *Nevýhoda:* Nižší stabilita (chyba nebo zásek v jediném ovladači může způsobit pád celého OS - tzv. Kernel Panic).
- **Mikrojádro (např. MINIX, QNX):** V jaderném prostoru běží jen naprosté minimum (plánování CPU, základní paměť a komunikace IPC). Vše ostatní (ovladače, filesystémy) běží odděleně jako běžné uživatelské procesy (v User Space).
 - *Výhoda:* Extrémní stabilita a bezpečnost (když spadne ovladač síťové karty, systém nepadne, ovladač se jednoduše restartuje).
 - *Nevýhoda:* Nižší výkon (velké zdržení a režie kvůli neustálému přepínání kontextu a posílání zpráv mezi procesy).

Datové struktury jádra

Jádro je napsáno převážně v jazyce C a ke správě systému využívá datové struktury:

- **task_struct:** Záznam o procesu (PCB), viz [předchozí kapitola](#).
- **signal_struct:** Datová struktura pro správu signálů.
- **inode:** Datová struktura nesoucí metadata o souboru a ukazatele na datové bloky na disku.
- **page:** Struktura reprezentující jednu fyzickou stránku operační paměti.

Systém vyrovnávací paměti (Cache / Buffers)

Operace čtení a zápisu na fyzický disk jsou extrémně pomalé. Jádro proto využívá volnou kapacitu RAM k dočasnému ukládání diskových dat.

- **Page Cache (Mezipaměť stránek):** Ukládá samotný obsah souborů, které byly nedávno čteny nebo zapisovány. Pokud uživatel otevře stejný soubor znovu, načte se z RAM.
- **Buffer Cache:** Slouží k ukládání surových bloků metadat (např. i-uzlů nebo superbloků) z disku. *V moderním Linuxu jsou Page a Buffer cache sjednoceny.*
- **Dirty pages (Špinavé stránky):** Pokud proces zapíše data do souboru, jádro je zapíše pouze do paměti (stránka se označí jako "špinavá"). Fyzický zápis na disk se zpozdí a provádí se hromadně na pozadí (démon `pdflush`) pro zvýšení výkonu. Vynutit zápis lze příkazem `sync`.

Subsystém V/V (Vstup / Výstup)

Komunikace s hardwarem je abstrahována tak, aby s ní mohly procesy pracovat jednotně. Zařízení jsou rozdělena na znaková a bloková viz. [typy souborů](#).

- **VFS (Virtual File System):** Abstraktní vrstva. Uživatelský program zavolá univerzální systémové volání `read()` nebo `write()`. VFS požadavek přijme a předá ho konkrétnímu ovladači (např. pro ext4, NTFS nebo USB flash disk).
- **Přerušení (Interrupts / SysRQ):** Hardwarová zařízení upozorňují procesor na událost (např. stisk klávesy nebo dokončení čtení z disku) pomocí asynchronního přerušení.
- **Struktura termios:** Terminálové rozhraní.

Charakteristika zařízení je obsažena v tabulce deskriptorů zařízení a zahrnuje:

- Identifikátor zařízení
- Instrukce zařízení
- Stav zařízení
- Identifikátor procesu, který se zařízením pracuje

Správa paměti

Linux nevyhrazuje procesům pevnou fyzickou paměť, ale využívá systém **virtuální paměti**. Každý proces má iluzi, že má k dispozici souvislý, obrovský blok vlastní paměti.

- **Stránkování (Paging):** Virtuální paměť i fyzická RAM jsou rozděleny na bloky o stejné velikosti, tzv. **stránky** (Pages).
- **MMU a tabulka stránek:** Hardwarová komponenta procesoru **MMU** (Memory Management Unit) za běhu překládá virtuální adresy na fyzické adresy pomocí Tabulky stránek, kterou spravuje jádro.
- **Výpadek stránky (Page Fault):** Pokud se proces pokusí přistoupit k paměti, která momentálně není nahrána ve fyzické RAM, vznikne výpadek. Jádro proces pozastaví, načte chybějící data z disku do RAM a proces obnoví.
- **Swapping:** Pokud začne docházet fyzická RAM, jádro přesune neaktivní stránky z RAM na pevný disk (do odkládacího oddílu – **Swap partition**). Jakmile je proces znovu potřebuje, přesunou se zpět. Toto zabraňuje pádu systému, ale výrazně jej zpomaluje.

8.7 Operační systémy reálného času (RTOS)

Definice a základní charakteristika

Definice: RTOS je systém, kde správnost výpočtu nezávisí pouze na logickém výsledku, ale striktně i na čase, kdy je výsledek doručen. V informatickém pojetí je RTOS takový OS, který zaručuje, že je každá správná informace ve správný čas na správném místě.

Charakteristika: Hlavním cílem RTOS není vysoká celková propustnost (jako u běžných OS typu Windows/Linux), ale absolutní predikovatelnost a determinismus. Systém zaručuje, že kritická operace proběhne do přesně stanoveného časového limitu (tzv. *deadline*).

Hard a Soft RTOS

Podle následků zmeškání časového limitu (*deadline*) se RTOS dělí na:

- **Hard RTOS (Tvrdý):** Zmeškání limitu představuje fatální selhání systému, které může ohrožit životy nebo majetek (např. vystřelení airbagu v autě, řízení kardiostimulátoru, brzdy vlaku). Ignoruje data která nepřišla včas.

Point-in-time - do určitého času (absolutní)

- **Soft RTOS (Měkký):** Zmeškání limitu nezpůsobí havárii, pouze se sníží kvalita poskytované služby (např. vynechání snímku při streamování videa, VoIP telefonie).

Delta-time - za určitý interval (relativní)

Hlavní faktory (metriky) RTOS

- Minimální latence při reakci na událost
- Minimální latence při přepínání vláken
- Někdy nutnost malých rozměrů (embedded systémy)
- Minimalizace časových okamžiků, kdy je vypnuto přerušování
- Preemptivní plánování založené na prioritách
- Dosažení paralelního běhu pokud možno všech zdrojů
- Podpora vysoce urgentních úloh
- Ochrana před uživateli ztrácí význam (je určen jen pro jednoho uživatele)

- Vyžadovaný mechanismy synchronizace a komunikace mezi úlohami

Plánování v RTOS (RMS a EDF)

Běžné OS (např. Linux) využívají plánovače zaměřené na spravedlnost (Fairness). RTOS místo toho využívá algoritmy založené na čase:

- **RMS (Rate Monotonic Scheduling):** Algoritmus se statickou prioritou. Pravidlo je jednoduché: čím kratší má úloha periodu opakování (čím častěji se musí spouštět), tím vyšší fixní prioritu od systému dostane.
- **EDF (Earliest Deadline First):** Algoritmus s dynamickou prioritou. Systém neustále přepočítává priority běžících procesů. Nejvyšší prioritu v daný okamžik dostane ta úloha, jejíž limit pro dokončení (*deadline*) vyprší nejdříve.

Příklady RTOS

- **QNX:** Komerční mikrokontrolerový OS, masivně využívaný v automobilovém průmyslu a medicíně.
- **VxWorks:** Komerční RTOS (např. řídil vozítka Curiosity a Perseverance na Marsu).
- **FreeRTOS:** Otevřený systém, dnes de facto standard pro malé mikrokontroléry a IoT zařízení.
- **RT-Linux:** Běžný Linux doplněný o tzv. PREEMPT_RT patch, který mu dodává vlastnosti RTOS.

9 Řízení procesů UNIXu

vytváření procesu, signály, ukončení procesu, vyvolání jiných procesů, reálné a efektivní UID, změna velikosti procesu, časování procesů, plánování procesů, SysRQ (použití, funkce)

Každý proces v systému (kromě procesu `init/systemd` s PID 1) vzniká jako potomek nějakého již existujícího rodičovského procesu.

9.1 Vytváření procesu (`fork`)

- **Systémové volání `fork()`:** Když běžící proces (rodič) zavolá `fork()`, jádro OS vytvoří jeho přesnou kopii (potomka).

- **Výsledek:** Potomek dostane vlastní unikátní PID, ale zdědí od rodiče otevřené soubory, proměnné i stav paměti. Od okamžiku volání běží oba procesy nezávisle a souběžně.
- **Návratová hodnota:** Aby program poznal, zda aktuálně běží jako rodič nebo potomek, vrací volání `fork()` různé hodnoty: 0 se vrátí potomkovi, zatímco rodiči se vrátí PID nového potomka.
- **Copy-on-Write (CoW):** Z důvodu výkonu se fyzická paměť při `fork()` ve skutečnosti nekopíruje ihned. Oba procesy sdílejí stejnou paměť, a ta se zkopíruje až v okamžiku, kdy se jeden z nich pokusí data změnit.

Vyvolání jiných procesů (`exec`)

Volání `fork()` vytvoří pouze identický klon. Pokud chceme spustit úplně nový program, musíme použít rodinu systémových volání `exec()` (např. `execvp`, `execl`).

- **Princip:** `exec()` nahradí kompletní paměťový prostor (kód, data, zásobník) aktuálního procesu novým programem nahráním.
- **Zachování PID:** Nový program se spouští v kontextu stávajícího procesu, takže jeho PID zůstává nezměněno.
- **Vzor `fork` + `exec`:** Toto je standardní UNIXový způsob spouštění programů. Rodič (např. shell) zavolá `fork()` (vznikne klon shellu) a tento klon okamžitě zavolá `exec()` (klon se přepíše na požadovaný program).

Ukončení procesu (`exit` a `wait`)

Proces může skončit normálně (dokončením úkolu) nebo abnormálně (chybou, signálem).

- **Systémové volání `exit()`:** Proces jím oznamuje jádru, že končí. Předává tzv. návratový kód (0 = úspěch, nenulová hodnota = chyba). Jádro následně uvolní paměť a zavře soubory procesu.
- **Systémové volání `wait()`:** Rodičovský proces musí zavolat `wait()`, aby si od jádra vyzvedl návratový kód svého potomka.
 - **Stav Zombie:** Pokud potomek skončí (`exit`), ale rodič ještě nezavolal `wait()`, potomek se stává *zombie*. Nezabírá už RAM ani CPU, ale stále blokuje jedno místo v tabulce procesů.
 - **Sirotek (Orphan):** Situace, kdy rodič zemře dříve než potomek. Sirotka převezme proces `init/systemd`, který po jeho skončení automaticky zavolá `wait()`.

9.2 Signály

Signály jsou formou softwarového přerušení. Slouží k asynchronní komunikaci a řízení procesů (např. upozornění na chybu nebo žádost o ukončení). Zachycují se pomocí trap.

- **Zpracování signálu:** Když proces obdrží signál, jádro přeruší jeho normální běh. Proces může signál *ignorovat*, *zachytit* (spustit vlastní obslužnou funkci - handler), nebo ponechat *výchozí akci* (typicky okamžité ukončení procesu).
- **Běžné signály:**
 - **SIGINT:** Přerušení z klávesnice (vyvoláno stiskem Ctrl+C). Lze ho zachytit a ignorovat.
 - **SIGTERM:** Slušná žádost o ukončení. Proces má čas si uložit data a korektně se ukončit. (Výchozí chování příkazu kill).
 - **SIGKILL:** Okamžité a brutální zabití procesu přímo jádrem (kill -9). Proces ho *nemůže* zachytit ani ignorovat.
 - **SIGSEGV:** Segmentation fault. Jádro zabíjí proces, protože se pokusil přistoupit do paměti, která mu nepatří.

9.3 Reálné a efektivní UID

V UNIXu/Linuxu je bezpečnost a přístup k souborům založen na identitě uživatele (User ID - UID). Každý běžící proces má s sebou spojeno několik takových identifikátorů:

- **Reálné UID (RUID):** Identifikuje skutečného uživatele, který proces spustil. Slouží především pro accounting a k určení, kdo může proces ukončit (např. běžný uživatel může zabít jen procesy se svým vlastním RUID).
- **Efektivní UID (EUID):** Jádro se na něj dívá při ověřování oprávnění (zda má proces právo číst/zapisovat do daného souboru). Většinou je EUID stejné jako RUID.
- **SUID (Set-User-ID) bit:** Speciální oprávnění u spustitelných souborů. Pokud běžný uživatel spustí program se SUID bitem (např. příkaz passwd pro změnu hesla), proces získá EUID vlastníka souboru (což je typicky root). Tím proces získá dočasná administrátorská práva pro zápis do systémových souborů, i když RUID zůstává uživatelské. Volá se setuid(uid).

Zpět na [Identita uživatele](#).

9.4 Změna velikosti procesu (Správa paměti)

Během svého života potřebuje proces často alokovat další paměť pro svá data (tzv. halda - Heap). Zvětšení nebo zmenšení paměťového prostoru procesu se děje na úrovni jádra pomocí systémových volání:

- **Systémová volání `brk()` a `sbrk()`:** Mění umístění tzv. *program break* (hranice definující konec datového segmentu/haldy procesu). Posunutím hranice nahoru se halda zvětší, posunutím dolů se paměť vrátí systému.
- **Systémové volání `mmap()`:** Modernější přístup pro alokaci velkých bloků paměti. Vytváří nová mapování ve virtuálním adresním prostoru procesu (často se používá i pro mapování souborů z disku přímo do RAM).

Poznámka: Běžný programátor tato volání nepoužívá přímo, ale využívá knihovní funkce C (např. `malloc` a `free`), které na pozadí chytře volají `brk` nebo `mmap` podle potřeby.

Paměťové segmenty procesu:

- Textový segment - obsahuje kód programu
- Datový segment - globální a statické proměnné
- Heap (halda) - dynamicky alokovaná paměť
- Stack (zásobník) - lokální proměnné

9.5 Časování procesů

Čas procesoru je rozdělen do epoch. V jedné epoše, každý proces má specifikováno časové kvantum vypočteno na jejím začátku. V jedné epoše, proces může využívat své časové kvantum po částech. Epoque končí, když všechny běhu schopné procesy vyčerpaly svá časová kvanta, kdy jsou přepočítána časová kvanta všech procesů (ne jenom běhu schopných) a začne nová epocha. Potomek zdědí základní časové kvantum rodiče.

9.6 Plánování procesů

Tři druhy plánování procesů (process scheduling):

- **Krátkodobé:** Výběr, kterému z připravených procesů bude přidělen procesor, ve všech více-úlohových systémech.
- **Střednědobé:** Výběr, který blokový nebo připravený proces bude odsunut z vnitřní paměti na disk, je-li vnitřní paměti nedostatek (swap out, roll out).
- **Dlouhodobé:** Výběr, která úloha bude spuštěna (aby byl počítač co nejvíce vytížen).

Protože procesů je více než jader procesoru, musí jádro (komponenta zvaná Plánovač - *Scheduler*) neustále rozhodovat, komu CPU přidělí.

- **CFS (Completely Fair Scheduler):** Výchozí plánovač moderního Linuxu (je preemptivní). Snaží se o maximální spravedlnost – modeluje ideální multitasking, kde každý proces dostává spravedlivý podíl výkonu procesoru na základě své váhy (priority).
- **Hodnota Nice:** V uživatelském prostoru se priorita neřídí přímo, ale tzv. *slušností* (hodnota nice).
 - Rozsah je od **-20** (nejvyšší priorita, nejméně slušný) do **+19** (nejnižší priorita, velmi slušný – pustí ostatní procesy před sebe).
 - Výchozí hodnota je 0. Běžný uživatel může hodnotu nice svému procesu pouze zvýšit. Negativní hodnoty může přidělovat pouze root. K úpravám slouží příkazy nice (při spuštění) a renice (za běhu).

9.7 SysRq

SysRq je kombinace kláves, která umožňuje uživateli posílat nízkoúrovňové příkazy přímo jádru OS. Funguje to i ve chvíli, kdy systém zamrzne, nereaguje nebo nefunguje grafické rozhraní, protože SysRq je obsluhováno přímo ovladačem klávesnice v jádře.

- **Použití:** Vyvolává se kombinací Alt + SysRq + <příkazové písmeno>. (V Linuxu musí být funkce nejprve povolena, např. zápisem do /proc/sys/kernel/sysrq). (Měl by se zapínat pouze pokud je to nutné hrozí napadení systému.)
- **Funkce a mnemotechnická pomůcka REISUB:** Sekvence pro bezpečné a čisté restartování zamrzlého systému (bez poškození souborového systému):
 - **R (Raw):** Převezme kontrolu nad klávesnicí od zmrzlého grafického serveru (X11/Wayland).
 - **E (tErminate):** Pošle všem procesům (kromě `init`) signál SIGTERM (žádost o bezpečné ukončení).

- **I** (kill): Pošle signál SIGKILL procesům, které na předchozí krok nereagovaly.
- **S** (Sync): Vynutí okamžitý zápis všech dat z RAM mezipaměti (špinavých stránek) na pevný disk.
- **U** (Unmount): Připojí všechny souborové systémy pouze pro čtení (Read-Only), aby nedošlo k poškození metadat při restartu.
- **B** (reBoot): Provede okamžitý restart systému (obdoba zmáčknutí reset tlačítka na case).

10 Základy běžné uživatelské administrace systému UNIX

vytváření souborů a adresářů, operace se soubory a adresáři, prohledávání systému souborů, identita uživatele, procesu, souboru a jejich změna, přístupová práva a jejich nastavení, přesměrování vstupu a výstupu, propojení příkazů, administrace uživatelů, zálohování, programy pro archivaci a kompresi dat, práce s příkazovými interprety, technologie SMART (význam, použití, vybrané údaje)

10.1 Vytváření a operace se soubory a adresáři

- **Vytváření:**

- `touch <soubor>`: Vytvoří nový prázdný soubor nebo zaktualizuje časové razítko existujícího.
- `mkdir <adresář>`: Vytvoří nový adresář (přepínač `-p` vytvoří celou cestu včetně chybějících nadřazených adresářů).

- **Operace:**

- `cp <zdroj> <cíl>`: Kopírování souborů. Pro adresáře nutno použít rekurzivně (`-r`).
- `mv <zdroj> <cíl>`: Přesun souborů nebo jejich přejmenování (v UNIXu jde o stejnou operaci).
- `rm <soubor>`: Smazání souboru. Pro adresáře nutno rekurzivně (`-r`), pro vynucení bez dotazů (`-f`).
- `rmdir <adresář>`: Smaže adresář, ale pouze pokud je prázdný.

10.2 Prohledávání systému souborů

- **find**: Komplexní vyhledávání v reálném čase podle různých kritérií (jméno, velikost, čas modifikace, vlastník). Příklad: `find /etc -name "*.conf"`.

- **locate:** Velmi rychlé hledání cesty k souboru pomocí předem indexované databáze (nutno aktualizovat pomocí updatedb).
- **grep:** Neprohledává samotný filesystem, ale *obsah* souborů. Hledá řádky odpovídající zadanému regulárnímu výrazu.
- **ls, cd**

10.3 Identita uživatele, procesu, souboru a jejich změna

- **Uživatel a proces:** Identifikován pomocí UID (User ID) a GID (Group ID). Proces po spuštění přebírá identitu uživatele (viz dříve [RUID/EUID](#)). Pro zjištění identity se používají příkazy `id` nebo `whoami`.
- **Soubor:** Každý soubor má svého vlastníka (uživatele) a vlastnickou skupinu.
- **Změna identity:**
 - `su <uživatel>`: Přepnutí uživatelského účtu v shellu.
 - `sudo <příkaz>`: Jednorázové spuštění příkazu s právy jiného uživatele (typicky root).
 - `chown <uživatel>:<skupina> <soubor>`: Změna vlastníka a skupiny souboru.
 - `chgrp <skupina> <soubor>`: Změna pouze skupiny.

10.4 Přístupová práva a jejich nastavení

Přístupová práva se definují pro tři kategorie: **u** (user/vlastník), **g** (group/skupina) a **o** (other-s/ostatní).

- **Základní práva:** **r** (read / čtení = 4), **w** (write / zápis = 2), **x** (execute / spuštění či vstup do adresáře = 1).
- **chmod:** Nástroj pro změnu práv.
 - *Oktalově (číselně):* Sečtením hodnot pro každou kategorii. Příklad: `chmod 755` (vlastník může vše (4+2+1=7), skupina a ostatní mohou jen číst a spouštět (4+1=5)).
 - *Symbolicky:* Příklad `chmod u+x` (přidá vlastníkovu právo spouštět), `chmod g-w` (odebere skupině právo zápisu).

10.5 Přesměrování vstupu/výstupu a propojení příkazů (Pipes)

Každý proces v UNIXu má automaticky otevřené 3 standardní proudy dat: **stdin** (0 - standardní vstup, typicky klávesnice), **stdout** (1 - standardní výstup, typicky monitor) a **stderr** (2 - chybový výstup).

- **Přesměrování výstupu:**
 - `>`: Přesměruje stdout do souboru (pokud existuje, přepíše ho).
 - `>>`: Přesměruje stdout a připojí jej na konec souboru (append).
 - `2 >`: Přesměruje chybový výstup (stderr) do souboru.
- **Přesměrování vstupu (<):** Program místo z klávesnice čte vstup ze specifikovaného souboru.
- **Roura / Pipe (|):** Vezme stdout prvního příkazu a přesměruje ho přímo do stdin druhého příkazu. Umožňuje řetězení jednoduchých nástrojů do složitých operací (např. `ls -la | grep "txt" | sort`).

10.6 Administrace uživatelů

Základní systémové soubory (dnes už je neupravujeme ručně, ale přes nástroje):

- `/etc/passwd`: Seznam uživatelů, jejich UID, GID, domovský adresář a výchozí shell.
- `/etc/shadow`: Bezpečně uložená (zahashovaná) hesla uživatelů.
- `/etc/group`: Seznam skupin a jejich členů.

Administrativní příkazy (vyžadují práva roota):

- `useradd <jméno>`: Vytvoření nového uživatele.
- `usermod <jméno>`: Modifikace existujícího uživatele (např. změna skupin, shellu).
- `userdel <jméno>`: Smazání uživatele.
- `passwd <jméno>`: Nastavení nebo změna hesla uživatele.

10.7 Zálohování, archivace a komprese dat

- **Archivace (tar):** Slouží ke spojení mnoha souborů/adresářů do jednoho velkého archivačního souboru (tzv. tarball). Sám o sobě nekomprimuje. (`tar -cvf` = vytvoření, `tar -xvf` = rozbalení).
- **Komprese (gzip, bzip2, xz):** Zmenšení velikosti souboru. V UNIXu se typicky kombinuje s `tar` (přepínače `-z` pro `gzip`, `-j` pro `bzip2`).
- **Zálohování (rsync):** Velmi mocný nástroj pro synchronizaci a zálohování. Kopíruje pouze změněné části souborů (inkrementální záloha), čímž šetří síťový provoz i čas.
- **dd:** Bitová kopie disků nebo diskových oddílů (low-level záloha).

10.8 Práce s příkazovými interprety (Shell)

Shell (např. `bash`, `zsh`, `sh`) je textové rozhraní tvořící vrstvu mezi uživatelem a jádrem OS.

- **Skriptování:** Shell obsahuje řídicí struktury (cykly, podmínky) a umožňuje automatizaci úloh pomocí shell skriptů.
- **Proměnné prostředí (Environment variables):** Konfigurují běhové prostředí aplikací. Významné jsou např. `$PATH` (seznam adresářů, kde systém hledá spustitelné soubory), `$USER`, `$HOME`. Zpřístupňují se pomocí příkazu `export`.
- **Konfigurace:** Každý uživatel má vlastní konfigurační soubory (např. `~/.bashrc` nebo `~/.profile`), kde lze definovat aliasy (vlastní zkratky příkazů) nebo upravit vzhled promptu.

10.9 Technologie SMART

S.M.A.R.T. (Self-Monitoring, Analysis and Reporting Technology) je monitorovací systém zabudovaný přímo v pevných discích (HDD/SSD).

- **Význam a použití:** Slouží k predikci selhání disku. Neustále sleduje hardwarové parametry a umožňuje administrátorovi včas vyměnit disk před ztrátou dat. V Linuxu se spravuje pomocí sady `smartmontools` (konkrétně příkaz `smartctl`).
- **Klíčové údaje (Atributy):**
 - *Reallocated Sector Count:* Počet poškozených sektorů, které disk nahradil záložními. Růst této hodnoty jasně značí odcházející disk.
 - *Power-On Hours:* Celková doba (v hodinách), po kterou byl disk v provozu.

- *Temperature*: Provozní teplota disku (přehřívání drasticky snižuje životnost).
- *Spin Retry Count (HDD)*: Počet marných pokusů o roztočení ploten (značí mechanický problém).
- *Wear Leveling Count (SSD)*: Indikátor opotřebení paměťových buněk u SSD.

11 Ochrana dat a informací

kategorie bezpečnostních přístupů (fyzická, informační, kybernetická, síťová, personál bezpečnost atp.), analýza aktiv, rizik a hrozeb, bezpečnostní politika, plán zvládání rizika a protipatření. Klíčová legislativa a mezinárodně uznávané standardy v oblasti informační a kybernetické bezpečnosti. Malware a ochrana proti škodlivému softwaru. Princip AAA v bezpečnosti, zabezpečení komunikace, zvýšení odolnosti systému a zálohování, možnosti ochrany HW

Ochrana dat představuje kompromis mezi mírou zabezpečení a použitelností.

11.1 Kategorie bezpečnostních přístupů

Ochrana dat a systémů není pouze záležitostí informačních technologií, ale vyžaduje komplexní přístup. Účinné zabezpečení se skládá z několika vzájemně se doplňujících vrstev (tzv. Defense in Depth). Hlavní kategorie bezpečnostních přístupů zahrnují:

- **Fyzická bezpečnost:** Zajišťuje ochranu samotného hardwaru, datových nosičů, serveroven a budov před neoprávněným fyzickým přístupem, krádeží, poškozením nebo živelnými pohromami. *Proky: bezpečnostní dveře, kamerové systémy (CCTV), biometrické vstupní systémy, UPS (záložní zdroje), protipožární systémy.*
- **Informační bezpečnost:** Široký koncept chránící data a informace bez ohledu na jejich formu (digitální data, ale i tištěné dokumenty či ústní předání). Její podstatou je udržení tzv. **CIA triády**:
 - *Confidentiality (Důvěrnost)*: K datům mají přístup jen oprávněné osoby.
 - *Integrity (Integrita)*: Data nesmí být neoprávněně změněna nebo poškozena.
 - *Availability (Dostupnost)*: Data musí být přístupná, když je uživatel potřebuje.
- **Kybernetická bezpečnost:** Podmnožina informační bezpečnosti, která se úzce zaměřuje na ochranu elektronických dat, počítačových systémů, mobilních zařízení a softwaru před digitálními hrozbami a útoky (např. malware, ransomware, DDoS).

- **Síťová bezpečnost:** Specifická část kybernetické bezpečnosti, jejímž cílem je chránit podnikovou síťovou infrastrukturu a data při jejich přenosu. *Proky: Firewally, VPN (Virtuální privátní sítě) pro šifrovaný přenos, systémy detekce a prevence průniku (IDS/IPS), oddělování sítí (segmentace a VLAN).*
- **Personální bezpečnost:** Řeší lidský faktor. Zaměřuje se na obranu proti hrozbám zevnitř (insider threats) a útokům založeným na sociálním inženýrství (např. phishing). *Proky: pravidelná školení uživatelů (Security Awareness), prověřování zaměstnanců, offboarding (odebírání přístupů při odchodu).*
- **Organizační (administrativní) bezpečnost:** Zastřešuje celkové řízení bezpečnosti v organizaci. Zahrnuje tvorbu bezpečnostních politik, směrnic, definování rolí, plány obnovy po havárii (Disaster Recovery Plan) a řízení rizik. Zajišťuje také soulad s legislativou a normami (např. GDPR, ISO 27001, Zákon o kybernetické bezpečnosti).
- **Aplikační bezpečnost:** Týká se zabezpečení dat přímo na úrovni softwaru a databází. Zahrnuje využívání bezpečných metod programování (Secure by Design), penetrační testování, pravidelné aktualizace (patch management) a ochranu proti zranitelnostem (např. SQL injection, XSS). Základním pravidlem zde je uplatňování **principu nejnižšího privilegia** (uživatel má do systému jen taková práva, která nezbytně potřebuje ke své práci).

Analýza aktiv, hrozeb a rizik

Řízení rizik (Risk Management) je základním stavebním kamenem efektivní ochrany dat. Nelze chránit vše na 100 %, proto organizace musí vědět, kam směřovat své omezené zdroje. Tento proces sestává z následujících kroků:

- **Analýza aktiv:** Identifikace a ohodnocení všeho, co má pro organizaci hodnotu a musí být chráněno. Aktivem nejsou jen data, databáze a servery (informační aktiva), ale také zaměstnanci, fyzické budovy, know-how nebo dobré jméno společnosti (reputace).
- **Analýza hrozeb a zranitelností:**
 - *Hrozba* je událost (nebo útočník), která může aktivum poškodit (např. hacker, ransomware, požár, lidská chyba).
 - *Zranitelnost* je slabina v systému (např. neaktualizovaný software, slabé heslo, chybějící zámeček). Hrozba využívá zranitelnost k útoku na aktivum.
- **Analýza rizik:** Riziko vzniká průnikem hrozby a zranitelnosti. Analýza riziko kvantifikuje nebo kvalifikuje na základě vzorce: $\text{Riziko} = \text{Pravděpodobnost hrozby} \times \text{Potenciální dopad (škoda na aktivu)}$.

Bezpečnostní politika organizace

Bezpečnostní politika je vrcholový, strategický dokument schválený nejvyšším vedením organizace. Zastřešuje veškeré snažení o ochranu dat a systémů.

- **Účel a rozsah:** Definuje základní cíle, pravidla a povinnosti pro zajištění bezpečnosti informací. Deklaruje podporu vedení.
- **Abstrakce:** Nezabíhá do detailních technických postupů (neřeší např. konfiguraci konkrétního routeru), ale stanovuje závazný rámec (např. „veškerá externí datová komunikace musí být šifrována“).
- **Návazná dokumentace:** Z bezpečnostní politiky vycházejí podrobné směrnice a standardy (např. směrnice pro tvorbu hesel, pravidla pro práci z domova, politika čistého stolu).
- **Závaznost:** Politika je závazná pro všechny zaměstnance a externí spolupracovníky, její porušení bývá sankcionováno na základě pracovněprávních předpisů.

Plán zvládání rizika a protiopatření

Na základě výsledků analýzy rizik rozhoduje vedení organizace o způsobu, jakým s jednotlivými riziky naloží. Existují čtyři základní strategie:

- **Snížení (Mitigace) rizika:** Nasazení protiopatření, která sníží pravděpodobnost výskytu hrozby nebo minimalizují její dopad.
- **Přenos (Transfer) rizika:** Přenesení finančních či provozních následků na třetí stranu (např. sjednání kybernetického pojištění nebo outsourcing rizikové služby do cloudu).
- **Vyhnutí se riziku:** Úplné ukončení činnosti nebo procesu, který riziko generuje (např. zákaz používání USB flash disků ve firmě, vypnutí zastaralého systému).
- **Akceptace rizika:** Náklady na protiopatření by byly vyšší než hodnota samotného aktiva nebo potenciální škoda. Vedení riziko formálně přijme a je připraveno nést případné následky.

Protiopatření: Jsou konkrétní mechanismy (bezpečnostní kontroly) k mitigaci rizik. Dělí se dle účelu na *preventivní* (zabraňují incidentu, např. firewall), *detekční* (upozorní na probíhající incident, např. alarm) a *korekční/reaktivní* (obnova dat ze zálohy po havárii).

11.2 Legislativa a standardy

Oblast ochrany dat a kybernetické bezpečnosti je přísně regulována. Zatímco legislativa definuje **co** musí být chráněno (závazné právní mantinely), mezinárodní standardy poskytují osvědčené postupy (Best Practices), **jak** toho technicky a procesně dosáhnout.

Klíčová legislativa Evropské unie

- **GDPR (Obecné nařízení o ochraně osobních údajů)**
- **Směrnice NIS 2 (Network and Information Security)**
- **DORA (Digital Operational Resilience Act)**

Klíčová legislativa České republiky

- **Zákon o kybernetické bezpečnosti (ZKB):** Implementuje evropskou směrnici NIS do českého práva. Vymezuje práva a povinnosti pro orgány státní správy, provozovatele kritické informační infrastruktury (KII) a poskytovatele základních služeb. Kontrolním a metodickým orgánem je **NÚKIB** (Národní úřad pro kybernetickou a informační bezpečnost).
- **Zákon o zpracování osobních údajů:** Adaptační zákon, který doplňuje a upřesňuje pravidla GDPR pro prostředí ČR. Dozorovým orgánem pro tuto oblast je **ÚOOÚ** (Úřad pro ochranu osobních údajů).

Mezinárodně uznávané standardy

Kromě legislativy se organizace řídí normami, které umožňují systematické řízení bezpečnosti. Často slouží jako podklad pro získání certifikace, kterou firmy vyžadují v rámci obchodních smluv. Organizace vytvářené normy: ISO, IEC, ITU (v ČR: ČSN, pro EU: ENISA)

- **Rodina norem ISO/IEC 27000:** Nejznámější globální standardy pro informační bezpečnost.
 - **ISO/IEC 27001:** Specifikuje požadavky na zavedení a udržování **ISMS** (Information Security Management System - Systém řízení bezpečnosti informací). Je založen na neustálém zlepšování pomocí Demingova cyklu **PDCA** (Plan - Do - Check - Act). Organizace se dle této normy mohou nechat certifikovat.
 - **ISO/IEC 27002:** Obsahuje podrobný katalog konkrétních bezpečnostních opatření (kontrol), které pomáhají naplnit požadavky normy 27001.

- **NIST Cybersecurity Framework (CSF):** Velmi populární rámec vyvinutý americkým Národním institutem standardů a technologie. Dělí obranu do pěti logických fází, které se dobře komunikují vedení firmy: **Identify** (Identifikuj aktiva a rizika) → **Protect** (Chraň je) → **Detect** (Detekuj útok) → **Respond** (Reaguj na incident) → **Recover** (Obnov provoz).

11.3 Malware

Malware (z anglického *malicious software* – škodlivý software) je zastřešující pojem pro jakýkoliv program nebo kód vytvořený s cílem poškodit počítačový systém, odcizit data, získat neoprávněný přístup nebo vydírat oběť.

Základní typy malwaru

Škodlivý kód se klasifikuje především podle způsobu šíření a svého chování:

- **Počítačový virus:** Připojuje se k legitimním spustitelným souborům. K aktivaci a šíření vyžaduje interakci uživatele (např. spuštění infikovaného souboru).
- **Červ (Worm):** Na rozdíl od viru je samostatný a nepotřebuje hostitelský soubor ani interakci uživatele. K masivnímu šíření sítí využívá bezpečnostní zranitelnosti OS a aplikací.
- **Trojský kůň (Trojan):** Maskuje se jako užitečný nebo legitimní program. Sám se nereplikuje, ale po instalaci uživatelem často otevírá zadní vrátka do systému (Backdoor) pro stažení dalšího malwaru.
- **Ransomware:** Zašifruje data na disku (nebo uzamkne zařízení) a za dešifrovací klíč požaduje výkupné. Moderní varianty data před zašifrováním navíc ukradnou a hrozí jejich zveřejněním.
- **Spyware a Keyloggery:** Skrytě sledují činnost uživatele, kradou přihlašovací údaje, historii prohlížení a zaznamenávají stisky kláves.
- **Rootkit:** Sofistikovaný malware operující hluboko v systému (často na úrovni jádra OS). Jeho cílem je utajit svou přítomnost a přítomnost dalšího škodlivého kódu před antivirem.
- **Botnet:** Sítí infikovaných zařízení (zvaných zombie), která útočník ovládá na dálku. Slouží k rozesílání spamu nebo k hromadným **DDoS** útokům.

Ochrana a protipatření

Obrana proti malwaru musí být vícevrstvá, nelze spoléhat pouze na jeden nástroj:

- **Antivirové systémy a EDR:** Běžný antivirus funguje na bázi detekce *virových signatur* (známých vzorů v kódu). Moderní firemní ochrana využívá **EDR** (Endpoint Detection and Response), která analyzuje nestandardní *chování* procesů v reálném čase pomocí AI a dokáže zastavit i dosud neznámý malware (Zero-day hrozby).
- **Patch Management (Aktualizace):** Pravidelné a včasné instalování bezpečnostních záplat operačního systému a aplikací, což eliminuje zranitelnosti, které využívají červi.
- **Filtrování sítě a e-mailů:** Většina malwaru (zejména ransomware a trojské koně) přichází formou **phishingu** v e-mailu. Nutností je blokování nebezpečných příloh a zakázání spouštění maker v dokumentech MS Office.
- **Zálohování:** Jediná 100% spolehlivá obrana proti ransomwaru. Ideálně dle pravidla **3-2-1** (3 kopie dat, na 2 různých médiích, z toho 1 geograficky oddělena/offline). Zálohy navíc musí být chráněny proti smazání (tzv. immutable backups).
- **Vzdělávání uživatelů:** Školení zaměstnanců např. proti phishingu.

11.4 Princip AAA v bezpečnosti

Proces řízení přístupu k datům a systémům se opírá o bezpečnostní model **AAA**, který se skládá ze tří na sebe navazujících kroků (předchází jim *Identifikace* – uživatel řekne, kdo je, např. zadá login):

- **Autentizace (Authentication – Ověření totožnosti):** Uživatel dokazuje, že je skutečně tím, za koho se vydává. Provádí se pomocí něčeho, co uživatel *zná* (heslo, PIN), co *má* (čipová karta, token, mobil) nebo čím *je* (biometrie – otisk, Face ID). Dnes je standardem **MFA** (Vícefaktorová autentizace).
- **Autorizace (Authorization – Oprávnění):** Systém na základě ověřené identity přidělí uživateli přístupová práva. Využívá se zde **RBAC** (Role-Based Access Control) – práva se nepřidělují jednotlivcům, ale pracovním rolím. Platí princip **nejnižšího privilegia** (přístup jen k tomu, co je nezbytné pro práci).
- **Auditování (Accounting – Záznam):** Monitorování a logování toho, co uživatel v systému dělal (kdy se přihlásil, jaké soubory smazal). Slouží k vyšetřování incidentů a zajištění principu *nepopiratelnosti* (Non-repudiation).

11.5 Zabezpečení komunikace

Cílem je ochrana dat při jejich přenosu (Data in Transit) před odposlechem a modifikací. Zajišťuje se pomocí kryptografie a síťových protokolů:

- **Šifrování sítě:** Pro bezpečný přístup do firemní sítě přes nezabezpečený internet se využívá VPN (Virtuální privátní síť), která vytváří šifrovaný tunel (protokoly IPsec nebo OpenVPN).
- **Zabezpečení aplikační vrstvy:** Přenos webových stránek a API komunikace musí probíhat přes HTTPS/TLS. Pro e-maily či messengery se využívá E2EE (End-to-End šifrování), kdy data dokáže dešifrovat pouze odesílatel a příjemce (ani provozovatel služby k nim nemá přístup).
- **Digitální podpisy a haše:** Pro ověření integrity zpráv (zda nebyly po cestě změněny) a ověření identity odesílatele (asymetrická kryptografie, infrastruktura veřejných klíčů – PKI).

11.6 Zvýšení odolnosti systému a zálohování

Odolnost znamená schopnost systému pokračovat v provozu i při selhání hardwaru nebo útoku.

- **Vysoká dostupnost (High Availability) a Redundance:** Kritické prvky (servery, disky, switche, zdroje napájení) jsou zdvojeny. Při výpadku primárního prvku automaticky přebírá zátěž záložní (Failover). Využívají se disková pole RAID a clustering serverů.
- **Zálohování (Backup):** Vytváření kopií dat pro případ ztráty. Definuje se dvěma klíčovými metrikami:
 - **RPO (Recovery Point Objective):** O kolik dat si může firma dovolit přijít (např. zálohujeme každou hodinu = RPO je 1 hodina).
 - **RTO (Recovery Time Objective):** Za jak dlouho po havárii musí být systém opět plně funkční.
- **Typy záloh:** *Plná* (všechna data, pomalá), *Inkrementální* (pouze změny od poslední jakékoliv zálohy, rychlá tvorba, pomalá obnova), *Diferenciální* (změny od poslední plné zálohy).

11.7 Možnosti ochrany hardwaru

Kromě klasické fyzické bezpečnosti (zámky, mříže, kamery, UPS, generátory, biometrický přístup do serveroven) se HW chrání i na úrovni samotného zařízení:

- **Šifrování celého disku (FDE - Full Disk Encryption):** Nástroje jako BitLocker (Windows) nebo FileVault (macOS) šifrují pevný disk. Pokud je notebook nebo disk fyzicky ukraden, data jsou bez dešifrovacího klíče nečitelná.
- **Čip TPM (Trusted Platform Module):** Specializovaný kryptografický čip na základní desce zařízení. Bezpečně uchovává šifrovací klíče a zajišťuje proces tzv. *Secure Boot* (ověřuje, že se při startu počítače nespouští škodlivý kód, např. rootkit).
- **Ochrana portů a BIOS/UEFI:** Ochrana vstupu do BIOSu heslem, zamezení bootování z externích médií a softwarové či fyzické uzamčení USB portů proti vložení neoprávněných flash disků (ochrana před únikem dat – DLP, i před zavlečením malwaru).

12 Kryptografie

základní principy a pojmy v oblasti kryptografie, klasifikace šifer (substituční, monoalfabetické, homofonní atp.), kryptografický systém, klíč, časová a paměťová náročnost, kryptoanalýza. Hashovací funkce, steganografie, symetrické a asymetrické šifrování, infrastruktura veřejných klíčů, certifikáty, certifikační autority

12.1 Základní principy a pojmy v oblasti kryptografie

Kryptografie je vědní disciplína (součást kryptologie), která se zabývá tvorbou a zkoumáním metod pro bezpečný přenos a ukládání dat za přítomnosti protivníka (útočníka). Její podstatou je transformace srozumitelných dat do nesrozumitelné podoby a naopak s využitím matematických aparátů.

Základní pojmy

- **Otevřený text (Plaintext):** Původní, srozumitelná zpráva nebo data určená k zašifrování.
- **Šifrový text (Ciphertext):** Zašifrovaná, pro neoprávněnou osobu nesrozumitelná zpráva.
- **Šifrování (Encryption):** Proces transformace otevřeného textu na šifrový text pomocí šifrovacího algoritmu a klíče.
- **Dešifrování (Decryption):** Opačný proces, kdy oprávněný příjemce transformuje šifrový text zpět na otevřený text.
- **Klíč (Key):** Tajný parametr (sekvence bitů), který řídí proces šifrování a dešifrování. Zajišťuje, že i při znalosti šifrovacího algoritmu nelze zprávu bez správného klíče přecíst.

- **Šifra / Šifrovací algoritmus:** Matematická funkce (nebo sada instrukcí), která definuje, jak přesně probíhá transformace dat.

Cíle kryptografie

Kryptografické mechanismy neslouží pouze k samotnému utajení obsahu zprávy, ale plní čtyři základní funkce moderní informační bezpečnosti:

- **Důvěrnost (Confidentiality):** Zajištění, že data si může přečíst pouze ten uživatel, který vlastní správný dešifrovací klíč.
- **Integrita (Integrity):** Schopnost spolehlivě detekovat, zda data nebyla během přenosu nebo uložení neoprávněně (či náhodně) změněna. K tomuto účelu slouží tzv. hašovací funkce.
- **Autentizace (Authentication):** Prokázání identity komunikující strany, tedy bezpečné ověření, že data skutečně pocházejí od deklarovaného odesílatele.
- **Nepopiratelnost (Non-repudiation):** Odesílatel zprávy nemůže v budoucnu lživě popřít, že zprávu odeslal nebo vytvořil. Typicky se realizuje pomocí asymetrické kryptografie a digitálních podpisů.

Kerckhoffsův princip

Zásadní teoretické pravidlo moderní kryptografie (formulované v 19. století), které říká, že **bezpečnost šifrovacího systému nesmí záviset na utajení samotného algoritmu, ale pouze na utajení klíče**.

12.2 Klasifikace šifer (Základní operace a klasické šifry)

Z hlediska základního principu fungování dělíme šifry do dvou hlavních kategorií: **transpoziční** a **substituční**.

1. Transpoziční (permutační) šifry

Znaky otevřeného textu se nenahrazují jinými, ale pouze se mění jejich pořadí (přeskupují se) podle určitého pravidla. Z textu se stává přesmyčka (anagram). Cílem je difúze informačního obsahu zprávy po celé její délce. Kryptoanalýza je namáhavější než u substitucních šifer. Nevýhodou použití jsou pametové a časové nároky při šifrování.

- *Příklad:* Skytalé (starověké Řecko - navíjení pergamenu na válec určitého průměru), Rail Fence (psaní textu do "plotu").

2. Substituční šifry

Každý znak (nebo skupina znaků) otevřeného textu je nahrazen jiným znakem, symbolem nebo číslem podle stanoveného klíče. Původní pořadí znaků zůstává zachováno. Dále se dělí na několik typů:

- **Monoalfabetické šifry:** K šifrování se používá pouze jedna šifrovací abeceda. Jeden konkrétní znak otevřeného textu je vždy nahrazen stejným znakem šifrového textu (např. každé "A" se změní na "D").
 - *Příklad:* Caesarova šifra (posun abecedy o pevný počet míst).
 - *Zranitelnost:* Velmi snadno prolomitelné pomocí **frekvenční analýzy** (porovnání četnosti znaků v šifrovém textu s průměrnou četností znaků v daném přirozeném jazyce).
- **Homofonní šifry:** Byly vytvořeny jako obrana proti frekvenční analýze. Jeden znak otevřeného textu může být nahrazen několika různými znaky (nebo čísly) šifrového textu. Znakům, které se v jazyce vyskytují často (např. samohláska "E"), je přiřazeno více možných šifrových variant, čímž se vyrovná rozložení četnosti v šifrovém textu.
- **Polygramové šifry:** Nenahrazují znaky po jednom (po písmenech), ale šifrují skupiny znaků najednou (např. dvojice písmen - bigramy, nebo trojice - trigramy).
 - *Příklad:* Playfairova šifra (šifruje dvojice písmen pomocí matice 5x5).
- **Polyalfabetické šifry:** Používají více šifrovacích abeced současně. To, jakou abecedou se daný znak zašifruje, závisí na jeho pozici v textu a na použitém klíči (hesle). Jedno písmeno "A" tak může být v šifrovém textu reprezentováno pokaždé jiným znakem.
 - *Příklad:* Vigenèrova šifra. Odolnost proti běžné frekvenční analýze je zde mnohem vyšší.

12.3 Kryptografický systém (Kryptosystém)

Kryptosystém je komplexní soubor prvků, algoritmů a postupů, které společně umožňují bezpečné šifrování a dešifrování dat. Podle způsobu šifrování a přenášení klíče se rozděluje na varianty:

- Symetrické
- Asymetrické
- Hybridní

12.4 Klíč

Klíč je tajný parametr, který vstupuje do šifrovacího či dešifrovacího algoritmu a mění jeho chování. V souladu s Kerckhoffsovým principem je bezpečnost celého systému závislá výhradně na utajení tohoto klíče.

- **Délka klíče:** Udává se v bitech a určuje velikost prostoru klíčů (např. klíč o délce n bitů má 2^n možných kombinací). Čím delší klíč je, tím obtížnější je jeho prolomení.

12.5 Kryptoanalýza

Kryptoanalýza je vědní disciplína zabývající se hledáním slabin v kryptografických systémech a jejich prolamováním bez předchozí znalosti klíče. Podle informací, které má útočník k dispozici, rozlišujeme základní modely útoků:

- **Útok pouze se znalostí šifrovaného textu (Ciphertext-only):** Útočník má k dispozici pouze zašifrované zprávy a snaží se z nich odvodit otevřený text nebo klíč.
- **Útok se znalostí otevřeného textu (Known-plaintext):** Útočník zná (nebo uhodne) textovou podobu části zprávy a zároveň má k dispozici její šifrovaný ekvivalent.
- **Útok s volbou otevřeného textu (Chosen-plaintext):** Útočník má možnost nechat zašifrovat libovolný vlastní text a analyzovat vzniklý šifrový text.
- **Útok hrubou silou (Brute-force):** Systematické zkoušení všech možných kombinací klíčů z prostoru K , dokud se neobjeví smysluplný otevřený text.
- **Další:** Slovníkový útok, Frekvenční analýza

12.6 Výpočetní náročnost

Bezpečnost moderních šifer se opírá o tzv. výpočetní bezpečnost — útok je teoreticky možný, ale prakticky neproveditelný z důvodu limitovaných zdrojů (času a paměti). Hodnotí se proto asymetricky:

- **Časová náročnost:** Vyjadřuje množství operací (procesorového času) potřebných k provedení algoritmu. Pro legitimní uživatele musí být šifrování a dešifrování rychlé (polynomiální čas $O(n^k)$). Pro útočníka bez klíče musí být kryptoanalýza časově nezvládnutelná (exponenciální čas $O(2^n)$), trávající například miliony let i na superpočítačích.

- **Paměťová náročnost:** Vyjadřuje množství operační paměti (RAM) nebo úložného prostoru, které algoritmus během svého běhu vyžaduje. Většina běžných šifer je navržena s ohledem na minimální paměťovou náročnost, aby fungovaly na vestavěných zařízeních či čipových kartách. Naopak u specifických funkcí (např. hashování hesel jako Argon2) se paměťová náročnost záměrně zvyšuje, což slouží jako obrana proti hardwarové akceleraci útoků pomocí GPU a ASIC čipů.

12.7 Hashovací funkce

Matematické algoritmy, které přijímají vstupní data libovolné délky a převádějí je na výstup pevné, předem dané délky (tzv. hash, otisk zprávy nebo fingerprint).

- **Základní vlastnosti:**
 - **Jednosměrnost:** Ze získaného hashe nelze matematicky zkonstruovat původní zprávu.
 - **Bezkoliznost:** Je výpočetně neproveditelné najít dvě různé zprávy, které by měly stejný hash.
 - **Citlivost na vstup:** I nepatrná změna na vstupu (např. změna jednoho znaku) vede ke zcela odlišnému výslednému hashi.
- **Využití:** Kontrola integrity dat (zda nebyl soubor poškozen či pozměněn), bezpečné ukládání hesel (neukládá se heslo, ale pouze jeho hash) a tvorba digitálních podpisů. Příklady: rodina SHA-2, SHA-3.

12.8 Steganografie

Na rozdíl od kryptografie, která zprávu činí nesrozumitelnou (ale je zjevné, že jde o šifru), steganografie skrývá samotnou existenci zprávy.

- **Princip:** Tajná data jsou vložena do jiného, na první pohled neškodného nosiče (tzv. cover medium), aby nevzbudila pozornost útočníka.
- **Využití:** Úprava nejméně významných bitů (LSB) v obrazových (JPEG, PNG) nebo zvukových souborech. Často se kombinuje s kryptografií — zpráva se nejprve bezpečně zašifruje a následně se vzniklý šifrový text steganograficky skryje.

12.9 Symetrické a asymetrické šifrování

Liší přístupem ke kryptografickým klíčům:

- **Symetrické šifrování:** Používá stejný klíč pro šifrování i dešifrování.
 - *Výhody:* Velmi rychlé a výpočetně nenáročné, ideální pro šifrování velkých objemů dat.
 - *Nevýhody:* Zásadní problém s distribucí klíče — obě komunikující strany si musí před zahájením komunikace bezpečně předat tajný klíč tak, aby jej nikdo neodposlechl.
 - *Příklady:*
 - * blokové – AES (Advanced Encryption Standard), DES, ChaCha20, RC5, Blowfish
 - * Proudové – RC4
- **Asymetrické šifrování:** Používá matematicky provázaný pár klíčů — veřejný klíč (Public Key) a soukromý klíč (Private Key). Co jedním klíčem zašifrujeme, to lze dešifrovat pouze druhým klíčem z páru.
 - *Princip:* Veřejný klíč odesílatel volně zveřejní. Kdokoliv jím může zašifrovat zprávu, ale dešifrovat ji může pouze majitel patřičného soukromého klíče, který drží v tajnosti.
 - *Výhody:* Řeší problém bezpečné distribuce klíčů a umožňuje realizaci digitálních podpisů (zajišťuje nepopiratelnost a autentizaci).
 - *Nevýhody:* Výpočetně velmi náročné a pomalé (běžně se proto používá k bezpečné výměně symetrického klíče, kterým se pak šifrují samotná data — tzv. hybridní šifrování).
 - *Příklady:* Diffie-Hellman, El Gamal, RSA, kryptografie eliptických křivek (ECC), DSA pro elektronické podpisy.

12.10 Infrastruktura veřejných klíčů (PKI) a certifikáty

Aby asymetrická kryptografie fungovala v praxi (např. na internetu), je nutné zajistit, že daný veřejný klíč skutečně patří tomu, kdo to o sobě tvrdí (obrana proti útoku Man-in-the-Middle). K tomu slouží systém PKI.

- **PKI (Public Key Infrastructure):** Ucelený systém hardwaru, softwaru, lidí, politik a postupů potřebných pro vytváření, správu, distribuci a revokaci digitálních certifikátů.
- **Digitální certifikát:** Elektronický dokument (typicky ve formátu X.509), který pevně svazuje veřejný klíč s identitou jeho majitele (např. s doménou webového serveru nebo jménem osoby).

Obsahuje:

- Veřejný klíč
- Informace o držiteli
- Platnost certifikátu

- Název a podpis certifikační autority
- **Certifikační autorita (CA):** Důvěryhodná třetí strana, která funguje jako digitální notář. CA ověří identitu žadatele a pokud je vše v pořádku, žadatelův certifikát (obsahující jeho veřejný klíč) elektronicky podepíše svým vlastním soukromým klíčem. Pokud uživatelský počítač důvěřuje této CA, bude automaticky důvěřovat i všem certifikátům, které vydala. (např. Lets encrypt, DigiCert, Globalsign, Comodo)

Část II

TIM

1 Formální jazyky, gramatiky a automaty

Základní pojmy teorie formálních jazyků, definice gramatiky, Chomského hierarchie gramatik. Regulární výrazy, konečné automaty, možnosti a využití regulárních jazyků. Bezkontextové gramatiky, zásobníkové automaty, vzájemný vztah bezkontextových jazyků a zásobníkových automatů.

1.1 Základní pojmy teorie formálních jazyků

Formální jazyk je množina řetězců nad nějakou abecedou Σ (někdy značena A). Formální jazyk se obvykle označuje velkým písmenem, např. L . $L \subseteq \Sigma^*$, kde Σ^* je formální jazyk nad abecedou Σ .

Řetězec je konečná posloupnost symbolů z abecedy Σ . Například, pokud $\Sigma = \{a, b\}$, pak $w = aab$ je řetězec nad Σ .

Abeceda je konečná množina symbolů, které se používí k vytváření řetězců. Například $\Sigma = \{0, 1\}$ je binární abeceda. **Poznámka:** Abeceda může být prázdná (neobsahuje žádné řetězce) nebo může obsahovat pouze prázdný řetězec ϵ (který nemá žádné symboly).

Délka řetězce je počet symbolů v řetězci. Například délka řetězce $w = aab$ je 3.

Prázdný řetězec je řetězec, který neobsahuje žádné symboly, a označuje se ϵ . Délka prázdného řetězce je 0.

Prefix řetězce w je jakýkoli řetězec, který lze získat odstraněním nula nebo více symbolů z konce w . Například, pokud $w = aab$, pak prefixy jsou ϵ , a , aa , a aab .

Podřetězec řetězce w je jakýkoli řetězec, který lze získat odstraněním nula nebo více symbolů ze začátku či konce w . Například, pokud $w = aab$, pak podřetězce jsou ϵ , a , b , aa , ab , a aab .

Postfix řetězce w je jakýkoli řetězec, který lze získat odstraněním nula nebo více symbolů z začátku w . Například, pokud $w = aab$, pak postfixy jsou ϵ , b , ab , a aab .

Zřetěžením dvou řetězců x a y vznikne řetězec, který je spojením x a y . Například, pokud $x = aa$ a $y = b$, pak zřetěžení xy je aab .

Překladače a interprety jsou programy, které zpracovávají formální jazyky. Překladač převádí zdrojový kód napsaný v jednom jazyce (např. C++) do jiného jazyka (např. strojový kód), zatímco interpret přímo vykonává instrukce napsané v daném jazyce bez předchozího překladu.

Proces překladu:

1. Lexikální analýza: Kontroluje jednotlivé znaky a vytváří z nich vyšší jednotky. Každé slovo musí být v souladu s gramatikou jazyka.
Např. $0.262.154 \times 0.262154 \checkmark$
2. Syntaktická analýza: Kontroluje správnost vyšších jednotek jazyka.
Např. *if(podmínka) then(příkaz)*
3. Sémantická analýza: Kontroluje význam vět.
Např. *if(pole.length=2) then... (je pole typu array)*
4. Generování kódu: Překlad syntakticky a sémanticky správného kódu do cílového jazyka (např. strojový kód).

1.2 Definice gramatiky

Gramatika je je čtveřice $G = (N, T, P, S)$, kde:

- N je konečná množina neterminálních symbolů (neboli proměnných).
- T je konečná množina terminálních symbolů (neboli abecedy), přičemž $N \cap T = \emptyset$.
- P je konečná množina produkčních pravidel, kde každé pravidlo má tvar $\alpha \rightarrow \beta$.
 - levá strana α musí obsahovat alespoň jeden neterminální symbol z N .
 - pravá strana β může být řetězec obsahující terminální i neterminální symboly, nebo může být prázdný řetězec ϵ .
 - $(N \cup T)^* N (N \cup T)^* \times (N \cup T)^*$
- $S \in N$ je startovací symbol.

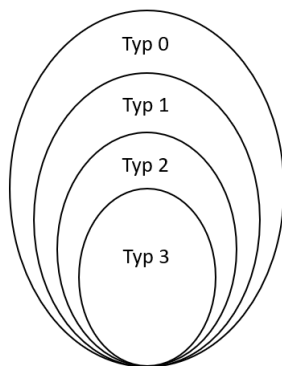
Gramatika G generuje jazyk $L(G)$, který je množinou všech řetězců nad abecedou T , které lze odvodit ze startovacího symbolu S pomocí produkčních pravidel v P . Formálně, řetězec $w \in T^*$ patří do jazyka $L(G)$, pokud existuje posloupnost odvození $S \Rightarrow^* w$, kde $\Rightarrow^*$ znamená, že w lze získat z S aplikací produkčních pravidel z P .

1.3 Chomského hierarchie gramatik

Chomského hierarchie gramatik je klasifikace formálních jazyků do čtyř tříd podle složitosti gramatik, které je generují:

- Typ 0: Rekurzivně spočetné jazyky, generované obecnými gramatikami bez omezení. (Turingovy stroje)
- Typ 1: Kontextové jazyky, generované kontextovými gramatikami, kde pravidla mají tvar $\alpha \rightarrow \beta$, kde α je alespoň 1 neterminální symbol a β není prázdný řetězec a zároveň platí $|\alpha| \leq |\beta|$. (Lineárně omezující automaty)
- Typ 2: Bezkontextové jazyky, generované bezkontextovými gramatikami, kde pravidla mají tvar $A \rightarrow \alpha$, kde $A \in N$ a $\alpha \in (N \cup T)^*$. (Zásobníkové automaty)
- Typ 3: Regulární jazyky, generované regulárními gramatikami, kde pravidla mají tvar $A \rightarrow aB$ nebo $A \rightarrow b$, kde $A, B \in N$, $a \in T^*$ a $b \in T$. (Deterministické konečné automaty)

Pumping lemma: Pro regulární a bezkontextové jazyky existují tzv. pumping lemmata, která poskytují nutnou podmínku pro to, aby jazyk byl regulární nebo bezkontextový.



Obrázek 2: Chomského hierarchie gramatik

1.4 Regulární výrazy, konečné automaty, možnosti a využití regulárních jazyků.

Regulární jazyk je jazyk, který lze generovat regulární gramatikou (typ 3) nebo rozpoznat konečným automatem. Regulární jazyky jsou nejjednodušší třídou jazyků v Chomského hierarchii. Jazyky definované regulárními výrazy jsou regulární jazyky, tj. jazyky rozpoznávané konečnými automaty.

Tři možnosti popisu regulárních jazyků:

- Regulární gramatiky
- Regulární výrazy
- Konečné automaty

Konečné automaty: Konečný automat je matematický model výpočetního zařízení, které může být v jednom z konečného počtu stavů a přechází mezi těmito stavy na základě vstupních symbolů. Vstup je přijat nebo zamítnut na základě toho, zda konečný automat skončí v akceptujícím stavu po zpracování celého vstupu. Nemá paměť pouze aktuální stav. Konečné automaty se dělí na dva typy:

- Deterministické konečné automaty (DFA): Pro každý stav a každý vstupní symbol existuje přesně jeden přechod do dalšího stavu.
- Nedeterministické konečné automaty (NFA): Pro každý stav a každý vstupní symbol může existovat více přechodů do různých stavů.

Formálně lze konečný automat definovat jako pětici $M = (Q, T, \delta, q_0, F)$, kde:

- Q je konečná množina stavů.
- T je konečná množina vstupních symbolů (abeceda).
- $\delta : Q \times T \rightarrow Q$ je přechodová funkce, která určuje, do kterých stavů může automat přejít z daného stavu při zpracování daného vstupního symbolu. V případě nedeterministického automatu představuje δ zobrazní, $Q \times T \rightarrow 2^Q$.
- $q_0 \in Q$ je počáteční stav.
- $F \subseteq Q$ je množina akceptujících (konečných) stavů.

Poznámka: δ je delta, 2^Q je množina všech podmnožin Q .

Konečné automaty jsou ekvivalentní regulárním gramatikám, což znamená, že každý jazyk, který lze generovat regulární gramatikou, lze také rozpoznat konečným automatem, a naopak. Regulární jazyky jsou uzavřené vůči operacím sjednocení, průniku, doplňku a zřetězení, což z nich činí velmi užitečný nástroj pro práci s řetězci a textem.

Využití regulárních jazyků: Regulární jazyky se široce používají v různých oblastech informatiky, včetně:

- Lexikální analýza v překladačích, kde se používají k rozpoznávání tokenů v zdrojovém kódu.
- Vyhledávání a manipulace s textem pomocí regulárních výrazů (regex).
- Jednoduché stroje pro zpracování a validaci vstupů, jako jsou automaty.

1.5 Bezkontextové gramatiky, zásobníkové automaty, vzájemný vztah bezkontextových jazyků a zásobníkových automatů

Bezkontextové gramatiky jsou gramatiky, kde každé produkční pravidlo má tvar $A \rightarrow \alpha$, kde A je neterminální symbol a α je řetězec obsahující terminální i neterminální symboly a zároveň platí $|\alpha| \leq |\beta|$ (tj. délka pravé strany pravidla není delší než délka levé strany). Bezkontextové jazyky jsou jazyky, které lze generovat bezkontextovými gramatikami. Bezkontextové gramatiky jsou nejednoznačné, což znamená, že není jednoznačné které pravidlo aplikovat pro odvození řetězce. Například, pokud máme pravidla $S \rightarrow aSb$ a $S \rightarrow \epsilon$.

Zásobníkové automaty: Zásobníkové automaty jsou rozšířením konečných automatů, které mají navíc zásobník pro ukládání a načítání symbolů. Zásobníkové automaty jsou schopné rozpoznávat bezkontextové jazyky. Formálně lze zásobníkový automat definovat jako sedmici $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, kde:

- Q je konečná množina stavů.
- Σ je konečná množina vstupních symbolů (abeceda).
- Γ je konečná množina symbolů zásobníku.
- $\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow Q \times \Gamma^*$ je přechodová funkce, která určuje, do kterých stavů může automat přejít z daného stavu při zpracování daného vstupního symbolu a vrcholu zásobníku.
- $q_0 \in Q$ je počáteční stav.
- $Z_0 \in \Gamma$ je počáteční symbol zásobníku.
- $F \subseteq Q$ je množina akceptujících (konečných) stavů.

Vztah mezi bezkontextovými jazyky a zásobníkovými automaty je takový, že každý bezkontextový jazyk může být rozpoznán nějakým zásobníkovým automatem, a naopak, každý jazyk rozpoznaný zásobníkovým automatem je bezkontextový. Tento vztah je formálně vyjádřen následujícími větami:

- Pro každý bezkontextový jazyk L existuje zásobníkový automat M takový, že $L = L(M)$, kde $L(M)$ je jazyk rozpoznaný automatem M .
- Pro každý zásobníkový automat M existuje bezkontextová gramatika G taková, že $L(M) = L(G)$, kde $L(G)$ je jazyk generovaný gramatikou G .

Pumping lemma: Pokud je jazyk např. bezkontextový, tak každé dost dlouhé slovo v něm musí jít rozdělit a část toho slova se dá opakovat víckrát a pořád zůstane v daném jazyce. Pokud se nám pro nějaký jazyk ukáže, že takové dělení neexistuje, pak jazyk není bezkontextový. Typický příklad je jazyk $a^n b^n c^n$, který pomocí pumping lemma dokážeme, že není bezkontextový.

2 Teorie vyčíslitelnosti

Turingův stroj, varianty Turingových strojů, univerzální Turingův stroj. Rozhodnutelnost vs rozpoznatelnost jazyka. Church-Turingova teze, problém zastavení Turingova stroje a další algoritmicky nerozhodnutelné problémy.

2.1 Turingův stroj

Turingův stroj je abstraktní model výpočetního zařízení (vytvořen Alanem Turingem), které se skládá z nekonečné pásky rozdělené na buňky, hlavy, která může číst a zapisovat symboly na pásku, a řídicí jednotky, která určuje chování stroje na základě aktuálního stavu a symbolu pod hlavou. Turingův stroj je schopen simulovat jakýkoli algoritmus a je považován za univerzální model výpočtu. Rozpoznává rekurzivně spočetné jazyky, což znamená, že může rozpoznat jakýkoli jazyk, pro který existuje algoritmus, ale nemusí být schopen rozhodnout, zda řetězec patří do jazyka nebo ne (může se dostat do nekonečné smyčky).

Turingův stroj se formálně definuje jako sedmice $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$, kde:

- Q je konečná množina stavů.
- Σ je konečná množina vstupních symbolů (abeceda). Neobsahuje prázdný znak $\emptyset$.
- Γ je konečná množina symbolů pásky, pro kterou platí $\emptyset \in \Gamma$ a $\Sigma \subseteq \Gamma$.
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ je přechodová funkce, která určuje, kam se má hlava posunout po zpracování daného vstupního symbolu na pásce a jaký symbol má být zapsán na pásku.
- $q_0 \in Q$ je počáteční stav.
- $q_{accept} \in Q$ je akceptující stav.
- $q_{reject} \in Q$ je zamítající stav, přičemž $q_{accept} \neq q_{reject}$.

Variety Turingových strojů: Existují různé varianty Turingových strojů, které se liší v některých aspektech, ale všechny jsou ekvivalentní z hlediska výpočetní síly. Některé z těchto variant zahrnují:

- Vícepáskový Turingův stroj má na rozdíl od zde již definovaného TS k dispozici hned několik pásek. Každá páska má vlastní čtecí/zapisovací hlavu.
- Nedeterministický Turingův stroj nemá jednoznačně definovanou přechodovou funkci, ale pracuje se zobrazením. V některých krocích výpočtu se tento výpočet dělí do více větví a pokud alespoň jedna větev dojde do stavu q_{accept} , je výpočet úspěšný. Lze k němu sestavit ekvivalentní deterministický Turingův stroj.
- TS s oboustrannou páskou má pásku, která je nekonečná na obě strany, což umožňuje posouvat hlavu i vlevo od počáteční pozice.
- Univerzální Turingův stroj.

Univerzální Turingův stroj: Univerzální Turingův stroj je Turingův stroj (UTS), který může simulovat jakýkoli jiný Turingův stroj na základě jeho popisu a vstupního řetězce. UTS dostane na pásce Gödelovo číslo (kód) jiného Turingova stroje a vstupní řetězec oddělený oddělovačem, a poté simuluje tento stroj na daném vstupu. Univerzální stroj U potom simuluje činnost stroje M a pokud M slovo w přijme, akceptuje toto slovo i stroj U. Ne každý binární řetězec je Gödelovým číslem nějakého Turingova stroje M (101, 0101110, ...)

2.2 Rozhodnutelnost vs rozpoznatelnost jazyka

Rozpoznatelnost jazyka znamená, že existuje Turingův stroj, který přijme všechny řetězce v jazyce a buď zamítne nebo se nikdy nezastaví pro řetězce, které nejsou v jazyce. Jinými slovy, jazyk je rozpoznatelný, pokud existuje algoritmus, který může potvrdit, že řetězec patří do jazyka, ale nemusí být schopen potvrdit, že řetězec nepatří do jazyka (může se dostat do nekonečné smyčky). Takový jazyk se také nazývá rekurzivně spočetný jazyk.

Rozhodnutelnost jazyka znamená, že existuje Turingův stroj, který přijme všechny řetězce v jazyce a zamítne všechny řetězce, které nejsou v jazyce. Jinými slovy, jazyk je rozhodnutelný, pokud existuje algoritmus, který může potvrdit, že řetězec patří do jazyka i že řetězec nepatří do jazyka (vždy se zastaví s odpovědí "ano" nebo "ne"). Takový jazyk se také nazývá rekurzivní jazyk.

2.3 Church-Turingova teze

Church-Turingova teze je tvrzení, že všechny algoritmy (nebo výpočty) lze provést pomocí Turingova stroje. Jinými slovy, pokud nějaký problém může být řešen algoritmem, pak existuje Turingův stroj, který tento problém může vyřešit. Každý jazyk, který lze popsat konečným výrazem, je rekurzivně spočetný. Tato teze není matematicky dokázána, ale je široce přijímána v informatice a matematice jako základní princip pro pochopení výpočtů a algoritmů.

2.4 Problém zastavení Turingova stroje a další algoritmicky nerozhodnutelné problémy

Problém zastavení Turingova stroje je otázka, zda existuje algoritmus, který může rozhodnout, zda daný Turingův stroj zastaví na daném vstupním řetězci. Alan Turing dokázal, že tento problém je nerozhodnutelný, což znamená, že neexistuje žádný algoritmus, který by mohl univerzálně rozhodnout, zda jakýkoli Turingův stroj zastaví.

Důkaz diagonalizační metodou: Diagonalizační metodou snadno sestavíme TS X , který se ode všech strojů v tabulce bude lišit chováním na diagonále. Takový stroj X potom ovšem v tabulce zdánlivě chybí, ale jelikož zde máme všechny TS, dostáváme spor s předpokladem, že tabulku lze vyplnit s pomocí stroje K , který umí rozhodnout problém zastavení.

	e	o	1	00	01	...
M_0	H→ L	H	H	L	L	...
M_1	L	H→ L	L	H	H	...
M_2	H	L	L→ H	L	H	...
M_3	H	H	H	L→ H	L	...
...	...	...	...	...	...	...

Obrázek 3: Diagonalizační metoda pro důkaz nerozhodnutelnosti problému zastavení Turingova stroje

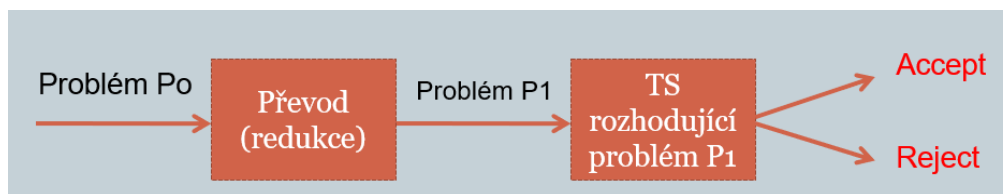
Další algoritmicky nerozhodnutelné problémy:

Problém ekvivalence dvou Turingových strojů

Problém neprázdnosti jazyka

Postův korespondenční problém: Lze ze zadané sady dominových kostek složit posloupnost, ve které je výsledný horní řetězec stejný jako řetězec dolní?

Užitečnou technikou při dokazování je redukce jednoho problému na problém jiný. Máme-li rozhodnout, zda nějaký problém P_1 je či není rozhodnutelný a víme-li, že problém P_0 rozhodnutelný není, potom obvykle postupujeme následovně:



Obrázek 4: Redukce jednoho problému na problém jiný

Redukovat musíme problém, o kterém víme, že není rozhodnutelný na problém, o kterém to chceme dokázat. Podaří-li se nám to opačně, ještě to vůbec neznamena, že náš problém je skutečně nerozhodnutelný. Redukci je třeba vždy dělat ze známého problému na náš.

Kromě nerozhodnutelných problémů existují i problémy Turingovsky nerozpoznatelné, což znamená, že neexistuje žádný Turingův stroj, který by mohl rozpoznat jazyk spojený s tímto problémem. Příkladem takového problému je Problém prázdnosti jazyka. Tento jazyk není rozpoznatelný, protože pokud by existoval Turingův stroj, který by ho rozpoznával, mohl by být použit k řešení (rozhodutelnosti) problému neprázdnosti jazyka, což je již dokázáno jako nemožné.

3 Teorie složitosti

Časová složitost, analýza algoritmu, asymptotická složitost algoritmu. Definice tříd P a NP, polynomiální převoditelnost problémů, pojem NP úplnosti, příklady NP úplných úloh, praktické důsledky.

Složitost algoritmu se obvykle měří v závislosti na velikosti vstupu, která se označuje jako n . Dva nejčastější typy složitosti jsou časová složitost, která měří, jak dlouho algoritmus běží, a prostorová složitost, která měří, kolik paměti algoritmus potřebuje.

3.1 Časová složitost, analýza algoritmu, asymptotická složitost algoritmu:

Časová složitost algoritmu je měřítkem toho, jak se čas potřebný k provedení algoritmu zvyšuje s velikostí vstupu. Nejčastěji se používá asymptotická notace, která popisuje chování algoritmu pro velké hodnoty n . Nejčastější asymptotické notace jsou:

- $O(f(n))$: Horní mez, která říká, že algoritmus běží nejvýše v čase $f(n)$ pro dostatečně velké n .
(O)

- $\Omega(f(n))$: Dolní mez, která říká, že algoritmus běží nejméně v čase $f(n)$ pro dostatečně velké n . (Omega)
- $\Theta(f(n))$: Přesná asymptotická složitost, která říká, že algoritmus běží v čase $f(n)$ pro dostatečně velké n . (Theta)

Nejedná se o přesné měření času, ale spíše o způsob, jak popsat růst složitosti algoritmu v závislosti na velikosti vstupu. Například algoritmus s časovou složitostí $O(n^2)$ bude běžet nejvýše v čase, který je úměrný druhé mocnině velikosti vstupu.

Analýza algoritmu zkoumá jak je algoritmus efektivní z hlediska času a prostoru. Provádí se nezávisle na konkrétní implementaci a zaměřuje se na obecné chování algoritmu.

Typy časové složitosti zahrnují:

- Konstantní čas $O(1)$: Algoritmus běží v konstantním čase, bez ohledu na velikost vstupu.
- Logaritmický čas $O(\log n)$: Algoritmus běží v čase, který roste logaritmicky s velikostí vstupu.
- Lineární čas $O(n)$: Algoritmus běží v čase, který roste lineárně s velikostí vstupu.
- Kvadratický čas $O(n^2)$: Algoritmus běží v čase, který roste kvadraticky s velikostí vstupu.
- Polynomiální čas $O(n^k)$: Algoritmus běží v čase, který roste polynomiálně s velikostí vstupu, kde k je konstanta.
- Exponenciální čas $O(2^n)$: Algoritmus běží v čase, který roste exponenciálně s velikostí vstupu. (NP)
- Faktoriální čas $O(n!)$: Algoritmus běží v čase, který roste faktoriálně s velikostí vstupu. (NP)
- Superexponenciální čas $O(n^n)$: Algoritmus běží v čase, který roste superexponenciálně s velikostí vstupu. (NP)

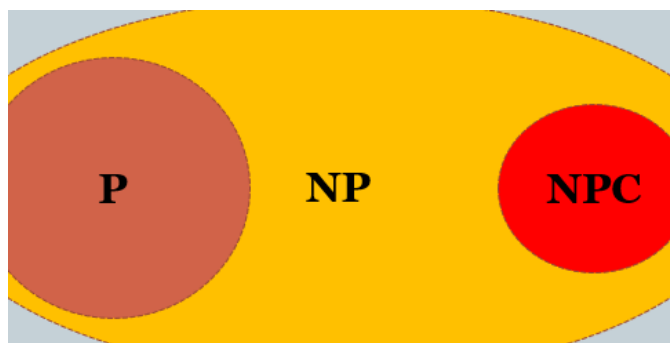
3.2 Definice tříd P a NP

Třída P: Třída P je třída všech jazyků, které jsou rozhodnutelné deterministickým Turingovým strojem v polynomiálním čase. V podstatě odpovídá třídě úloh, které jsou realisticky řešitelné na počítači. Příklady úloh v P zahrnují: třídění seznamu čísel, aritmetické operace (sčítání, násobení, dělení), násobení matic, nejkratší cesta v grafu (Dijkstraův algoritmus), průměr, medián a modus, atd.

Třída NP: Třída NP (NPTIME) je třída jazyků, pro které existuje verifikační algoritmus pracující v polynomiálně omezeném čase. Jazyk patří do třídy NP (NPTIME) právě tehdy, když je rozhodnutelný v polynomiálně omezeném čase nějakým nedeterministickým Turingovým strojem. Příklady úloh v NP zahrnují: izomorfismus grafů, diskrétní logaritmus, faktorování velkých čísel, atd.

Všechny úlohy v P jsou také v NP, protože deterministický Turingův stroj je speciálním případem nedeterministického Turingova stroje. Nicméně, není známo, zda všechny úlohy v NP jsou také v P, což je známé jako problém P vs NP.

Zatímco problémy z třídy P jsou „rychle“ rozhodnutelné, problémy z NP jsou pouze „rychle“ verifikovatelné.



Obrázek 5: Vztah mezi třídami P, NP, NP-úplné

3.3 Polynomiální převoditelnost problémů

Polynomiální převoditelnost problémů je koncept, který se používá k porovnání složitosti různých problémů. Říkáme, že problém A je polynomiálně převoditelný na problém B (označujeme jako $A \leq_p B$), pokud existuje polynomiální algoritmus, který může převést instance problému A na instance problému B . Polynomiální převoditelnost se používá k určení, zda je problém A stejně těžký jako problém B . Pokud $A \leq_p B$ a B je v NP, pak A je také v NP. Pokud navíc B je NP-úplný, pak A je také NP-úplný.

3.4 NP-úplnost

Obtížnost některých problémů z třídy NP je stejná v tom smyslu, že nalezení „rychlého“ algoritmu pro řešení kterékoliv z těchto úloh, by znamenalo nalezení „rychlého“ algoritmu pro všechny úlohy z této specifické třídy. Problém je NP-úplný, pokud je v NP a každý problém v NP lze polynomiálně převést na tento problém. Pokud by byl nalezen algoritmus, který by řešil problém z třídy NP-úplných úloh v polynomiálním čase, pak by všechny problémy v NP mohly být řešeny

v polynomiálním čase, což by znamenalo, že $P = NP$. Příklady NP-úplných úloh zahrnují: problém splnitelnosti logických formulí (SAT), existence kliky, problém barvení grafu, problém nezávislé množiny, problém obchodního cestujícího, atd.

Praktické důsledky: Pokud $P = NP$, znamenalo by to, že všechny problémy v NP, včetně těch, které jsou považovány za velmi obtížné (jako je faktorování velkých čísel), by mohly být řešeny efektivně. To by mělo obrovské důsledky pro kryptografii, optimalizaci a mnoho dalších oblastí informatiky a matematiky. Na druhou stranu, pokud $P \neq NP$, znamenalo by to, že existují problémy, které jsou inherentně obtížné a nelze je řešit efektivně.

4 Algoritmy a jejich složitosti

Hledání minimální kostry v grafu, artikulace, mosty, bloky v grafu, algoritmus na vyhledávání artikulací a bloků v grafu. Bipartitní grafy, párování v grafu, přiřazovací problém. Úloha čínského pošťáka.

4.1 Hledání minimální kostry v grafu

Minimální kostra (nebo minimální kostrový strom) v grafu je podgraf, který je stromem (tj. je acyklický a souvislý) a má minimální možnou cenu (součet cen hran) mezi všemi možnými kostrami grafu. Pro souvislé neorientované grafy s ohodnocenými hranami. Využívá se například v síťových návrzích, kde je potřeba minimalizovat náklady na propojení všech uzlů. Podmínky kostry:

- Graf musí být souvislý.
- Graf neobsahuje cykly.
- Graf obsahuje všechny vrcholy původního grafu.
- Součet cen hran v kostře je minimální.
- Počet hran v kostře je vždy $|V| - 1$, kde $|V|$ je počet vrcholů v grafu.

Algoritmy pro hledání minimální kostry:

- **Jarníkův algoritmus (Primův algoritmus)** – využívá prioritní frontu pro efektivní výběr nejlevnější hrany. Složitost:

- $O(E \log V)$ s použitím binární haldy.
- $O(E + V \log V)$ s použitím Fibonacciho haldy
- $O(V^2)$ s použitím jednoduché matice sousednosti.

E je počet hran a V je počet vrcholů v grafu.

1. Začneme s prázdnou kostrou a zvolíme libovolný počáteční vrchol.
 2. Opakovaně přidáváme nejlevnější hranu, která spojuje vrchol v kostře s vrcholem mimo kostru, dokud nejsou všechny vrcholy zahrnuty.
- **Borůvkův algoritmus** – Složitost: $O(E \log V)$, kde E je počet hran a V je počet vrcholů v grafu.
 1. Každý vrchol začíná jako samostatná komponenta.
 2. V každém kroku pro každou komponentu najdeme nejlevnější hranu, která ji spojuje s jinou komponentou. Všechny tyto hrany přidáme do kostry (duplicitu se odstraní).
 3. Pokračujeme, dokud není kostra kompletní (všechny vrcholy jsou v jedné komponentě).
 - **Kruskalův algoritmus** – Složitost: $O(E \log E)$, kde E je počet hran v grafu. Pokud jsou hrany již seřazeny, složitost je $O(E \alpha(V))$, kde α je inverzní Ackermannova funkce, která roste velmi pomalu a pro všechny praktické účely lze považovat za konstantu. (téměř lineární čas)
 1. Seřadíme všechny hrany grafu vzestupně podle jejich cen.
 2. Postupně procházíme hrany a každou přidáme do kostry, pokud její přidání nevytvoří cyklus (používáme strukturu *Disjoint Set Union* pro sledování komponent).
 3. Pokračujeme, dokud kostra neobsahuje $|V| - 1$ hran.

4.2 Artikulace

Artikulace (neboli artikulační body) v grafu jsou vrcholy, které, pokud jsou odstraněny, způsobí, že graf se stane nesouvislým. Jinými slovy, artikulační bod je vrchol, jehož odstranění zvýší počet komponent grafu. Artikulace jsou důležité pro analýzu struktury grafu a pro identifikaci kritických bodů v síti.

Algoritmus pro hledání artikulací:

- **Tarjanův algoritmus pro artikulace** – Složitost: $O(V + E)$, kde V je počet vrcholů a E je počet hran.
 1. Provedeme prohledávání do hloubky (DFS). Každému vrcholu u přiřadíme:

- $disc[u]$: čas (pořadí) objevení vrcholu v rámci DFS.
 - $low[u]$: nejmenší $disc$ dosažitelný z u nebo jeho potomků pomocí nejvýše jedné zpětné hrany.
2. Vrchol u je artikulací, pokud splňuje jednu z podmínek:
- u je kořenem DFS stromu a má v tomto stromě **více než jednoho přímého potomka**.
 - u není kořenem a má přímého potomka v takového, že $low[v] \geq disc[u]$. (To znamená, že z podstromu vrcholu v nevede žádná zpětná hrana do u nebo výše.)

4.3 Mosty v grafu

Mosty (neboli hrany, které jsou mosty) v grafu jsou hrany, které, pokud jsou odstraněny, způsobí, že graf se stane nespojitým. Jinými slovy, most je hrana, jejíž odstranění zvýší počet komponent grafu. Mosty jsou důležité pro analýzu struktury grafu a pro identifikaci kritických spojení v síti.

Algoritmus pro hledání mostů:

- **Tarjanův algoritmus pro mosty** – Složitost: $O(V + E)$, kde V je počet vrcholů a E je počet hran.
 1. Provedeme prohledávání do hloubky (DFS). Každému vrcholu u přiřadíme:
 - $disc[u]$: čas objevení vrcholu v rámci DFS.
 - $low[u]$: nejmenší $disc$ dosažitelný z u nebo jeho potomků pomocí zpětných hran (**přitom ignorujeme hranu, po které jsme do u přišli z rodiče**).
 2. Hrana (u, v) , kde v je potomek u v DFS stromu, je mostem, pokud platí: $low[v] > disc[u]$
 3. Algoritmus takto projde celý graf a identifikuje všechny mosty.

4.4 Bloky v grafu

Bloky v grafu jsou maximální souvislé podgrafy, které po odstranění libovolného vrcholu zůstanou souvislé. Jinými slovy, blok je buď maximální podgraf bez artikulačních bodů, nebo jedna samostatná hrana (most). Bloky jsou klíčové pro analýzu robustnosti neorientovaných grafů, protože identifikují části sítě, které drží pohromadě i při výpadku jednoho uzlu.

Algoritmus pro hledání bloků:

- **Tarjanův algoritmus pro bloky** – Složitost: $O(V + E)$, kde V je počet vrcholů a E je počet hran.

1. Provedeme prohledávání do hloubky (DFS). Každému vrcholu u přiřadíme hodnoty $disc[u]$ a $low[u]$ (stejně jako u hledání [artikulací](#)).
2. Během průchodu DFS ukládáme každou prošlou hranu na **zásobník**.
3. Když se v rekurzi vracíme z vrcholu v do u a zjistíme, že je splněna podmínka artikulace ($low[v] \geq disc[u]$):
 - Právě jsme našli nový blok.
 - Ze zásobníku hran postupně vybíráme (pop) všechny hrany až po hranu (u, v) . Tyto hrany tvoří jeden blok (2-souvislou komponentu).
4. Po skončení DFS jsou všechny hrany rozděleny do příslušných bloků.

Poznámka k implementaci Tarjanových algoritmů: Při DFS se hodnoty $disc$ (čas objevení) nastavují při cestě dolů (vstup do vrcholu). Hodnoty low a vyhodnocení podmínek (pro artikulace, mosty i bloky) se provádějí při návratu z rekurze (backtracking).

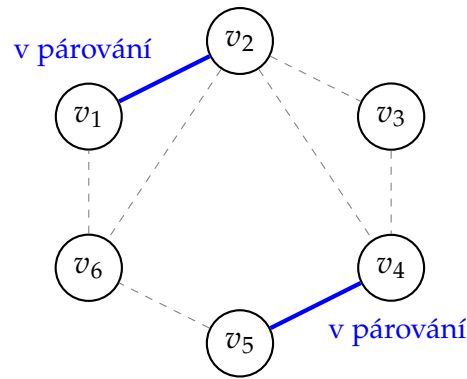
4.5 Bipartitní grafy

Bipartitní graf je graf, jehož vrcholy lze rozdělit do dvou disjunktních množin U a V tak, že každá hrana spojuje vrchol z U s vrcholem z V . Jinými slovy, neexistují hrany mezi vrcholy v rámci stejné množiny. Bipartitní grafy se často používají k modelování vztahů mezi dvěma různými skupinami objektů, například studentů a kurzů, zaměstnanců a projektů, atd.

4.6 Párování v grafu

Párování v grafu (matching) je množina nezávislých hran, tedy hran, které nemají žádný společný vrchol. V bipartitních grafech se párování často používá k řešení problémů přiřazení. Například, pokud máme bipartitní graf, kde jedna množina vrcholů reprezentuje zaměstnance a druhá projekty, párování přiřazuje zaměstnance k projektům tak, aby každý zaměstnanec pracoval maximálně na jednom projektu a každý projekt měl maximálně jednoho zaměstnance.

- **Maximální párování (Maximal matching):** Párování, ke kterému již není možné přidat žádnou další hranu, aniž by se porušila podmínka, že žádné dvě hrany nesmí sdílet vrchol.
- **Největší párování (Maximum matching):** Párování, které obsahuje absolutně největší možný počet hran v daném grafu.
- **Perfektní (úplné) párování:** Speciální případ největšího párování, které pokrývá všechny vrcholy grafu (každý vrchol je incidentní s přesně jednou hranou z párování).



Obrázek 6: Princip párování na obecném grafu. Modré hrany (v_1, v_2) a (v_4, v_5) tvoří párování, protože nemají žádný společný vrchol. Vrcholy v_3 a v_6 jsou volné. Toto párování je **maximální** (žádnou další hranu nelze přidat, aniž by sdílela vrchol s modrými), ale není **největší** (šlo by vybrat např. hrany (v_6, v_1) , (v_2, v_3) a (v_4, v_5) , čímž by vzniklo perfektní párování velikosti 3).

Algoritmy pro hledání párování v bipartitním grafu:

- **Hopcroft-Karpův algoritmus** – Složitost: $O(\sqrt{VE})$, kde V je počet vrcholů a E je počet hran. Efektivní algoritmus pro hledání **největšího párování**.
 1. Provedeme prohledávání do šířky (BFS) ze všech aktuálně volných (nespárovaných) vrcholů v množině U . Cílem je vytvořit vrstevnatý graf a najít nejkratší cesty k volným vrcholům v množině V .
 2. Pokud BFS nenajde žádný dosažitelný volný vrchol ve V , algoritmus končí a aktuální párování je **největší**.
 3. Pro každý volný vrchol z U provedeme prohledávání do hloubky (DFS) podél vrstev nalezených pomocí BFS. Hledáme *zlepšující (augmentující) cestu* – cestu, která střídá hrany mimo párování a v párování a začíná i končí ve volném vrcholu.
 4. Pokud najdeme zlepšující cestu, aktualizujeme párování přepnutím (symetrickou diferencí). Hrany na cestě, které byly v párování, z něj vyjmemme, a ty, které v něm nebyly, do něj přidáme. Tím se velikost párování zvětší o 1.
 5. Opakujeme tento proces od bodu 1, dokud lze nacházet zlepšující cesty.
- **Hladový (Greedy) algoritmus** – Složitost: $O(V + E)$. Tento algoritmus slouží jako rychlá heuristika. (Nezaručuje největší párování, ale je velmi rychlý a jednoduchý.)
 1. Projdeme všechny hrany grafu. Pro každou hranu (u, v) zkontrolujeme, zda jsou oba vrcholy u a v volné (nespárované).
 2. Pokud ano, přidáme hranu (u, v) do párování a oba vrcholy označíme jako spárované.
 3. Pokračujeme, dokud neprojdeme všechny hrany.

4. Výsledkem je vždy **maximální** párování, ale ne nutně **největší**. Nemusí být optimální, protože brzkým výběrem hrany můžeme zablokovat dlouhé zlepšující cesty. Je ovšem garantováno, že velikost tohoto párování je alespoň poloviční oproti největšímu možnému párování.

4.7 Přiřazovací problém

Přiřazovací problém je speciální případ problému párování v bipartitním grafu, kde máme dvě množiny vrcholů (například pracovníci a úkoly) a každá hrana mezi nimi má přiřazenou cenu (náklad). Cílem je najít perfektní párování, které minimalizuje celkovou cenu přiřazení.

Algoritmus pro řešení přiřazovacího problému:

- **Maďarský algoritmus** (také známý jako Kuhn-Munkresův algoritmus) – Složitost: $O(V^3)$, kde V je počet vrcholů v jedné partitě (předpokládáme, že obě množiny mají stejný počet vrcholů, reprezentováno maticí $V \times V$). Tento algoritmus efektivně řeší problém přiřazení a zaručuje nalezení optimálního řešení.
 1. Vytvoříme nákladovou matici, kde řádky reprezentují jednu množinu (např. pracovníky) a sloupce druhou množinu (např. úkoly). Každá buňka obsahuje cenu přiřazení.
 2. **Redukce řádků:** Pro každý řádek najdeme nejmenší hodnotu a odečteme ji od všech hodnot v tomto řádku.
 3. **Redukce sloupců:** Pro každý sloupec najdeme nejmenší hodnotu a odečteme ji od všech hodnot v tomto sloupci.
 4. Pokusíme se najít perfektní párování výhradně pomocí nulových položek v redukované matici. Hledáme nezávislé nuly, takové, které nesdílejí řádek ani sloupec s jinou nulou v párování. Pokud je takové párování nalezeno, algoritmus končí a je optimální.
 5. Pokud perfektní párování pomocí nul nelze sestavit, upravíme matici:
 - Pokryjeme všechny nuly v matici minimálním možným počtem horizontálních a vertikálních čar.
 - Najdeme nejmenší hodnotu mezi čísly, která *nejsou* pokryta žádnou čarou.
 - Tuto hodnotu odečteme od všech nepokrytých čísel a přičteme ji k číslům, která leží na průsečíku dvou čar (ostatní pokrytá čísla zůstávají beze změny).
 6. Po této úpravě matice se vrátíme zpět ke kroku 4.

4.8 Úloha čínského poštáka

Úloha čínského poštáka (Chinese Postman Problem) je problém hledání nejkratšího možného sledu (uzavřené cesty), který projde každou hranou grafu alespoň jednou a vrátí se zpět do výchozího bodu. Problém zformuloval čínský matematik Kwan Mei-Ko.

Na rozdíl od problému obchodního cestujícího je úloha čínského poštáka na neorientovaných i orientovaných grafech řešitelná v **polynomiálním čase**. NP-těžká je pouze pro smíšené grafy (obsahující orientované i neorientované hrany).

Algoritmus pro řešení úlohy čínského poštáka (neorientovaný graf):

- **Edmonds-Johnsonův algoritmus** – Složitost: $O(V^3)$.
 1. **Kontrola eulerovskosti:** Zkontrolujeme stupně všech vrcholů v grafu. Pokud mají všechny vrcholy sudý stupeň, graf je Eulerovský. Řešením je přímo Eulerův tah (lze najít např. Hierholzerovým algoritmem v čase $O(E)$), jehož délka se rovná součtu vah všech hran. Není třeba nic přidávat.
 2. **Identifikace lichých vrcholů:** Pokud graf není Eulerovský, najdeme všechny vrcholy s lichým stupněm. Podle [lemmatu o podání ruky](#) víme, že jich musí být sudý počet.
 3. **Nejkratší cesty:** Spočítáme nejkratší cesty (a jejich vzdálenosti) mezi všemi dvojicemi těchto lichých vrcholů (např. pomocí Dijkstrova nebo Floyd-Warshallova algoritmu).
 4. **Pomocný úplný graf:** Vytvoříme nový, pomocný úplný graf, jehož vrcholy jsou pouze naše identifikované liché vrcholy. Váha hrany mezi libovolnými dvěma vrcholy bude odpovídat vzdálenosti jejich nejkratší cesty v původním grafu.
 5. **Minimální perfektní párování:** V tomto pomocném grafu najdeme *perfektní párování s minimální cenou*. Tím liché vrcholy optimálně spárujeme tak, aby součet vzdáleností mezi páry byl co nejmenší.
 6. **Zdvojení hran (Eulerizace):** V původním grafu vezmeme nejkratší cesty odpovídající nalezenému párování a všechny hrany na těchto cestách "zdvojíme" (přidáme k nim paralelní hrany). Tím se zvýší stupeň spárovaných vrcholů o 1 (stanou se sudými), aniž by se porušila sudost mezilehlých vrcholů.
 7. **Nalezení tahu:** Modifikovaný multigraf má nyní všechny vrcholy sudého stupně. Nalezneme v něm Eulerův tah. Tento tah představuje optimální trasu čínského poštáka a jeho celková cena je součtem vah všech původních hran plus vah zdvojených hran.

Lemma o podání ruky: V každém neorientovaném grafu je součet stupňů všech vrcholů roven přesně dvojnásobku počtu hran.

Pro orientované grafy je postup podobný, ale místo párování lichých vrcholů hledáme párování vrcholů s nerovným počtem příchozích a odchozích hran. Pro smíšené grafy je problém NP-těžký, protože kombinace orientovaných a neorientovaných hran komplikuje strukturu problému a nelze jej řešit pomocí výše uvedených metod.

5 Orientované grafy

Silná souvislost a silně souvislé komponenty v grafu, acyklické grafy, topologické uspořádání grafu, externální cesty v acyklickém grafu. Metody síťové analýzy (CPM, PERT), toky a maximální tok v sítích.

Orientované grafy (digrafy) jsou grafy, ve kterých jsou hrany orientovány, tj. mají směr. Každá hrana je reprezentována jako uspořádaná dvojice vrcholů (u, v) , což znamená, že hrana směřuje od vrcholu u k vrcholu v . Orientované grafy se používají k modelování asymetrických vztahů mezi objekty, například v sociálních sítích, dopravních sítích, závislostech mezi úkoly atd.

5.1 Silná souvislost a silně souvislé komponenty v grafu

Silná souvislost v orientovaném grafu znamená, že pro každou dvojici vrcholů u a v existuje orientovaná cesta z u do v a zároveň z v do u . Jinými slovy, každý vrchol je dosažitelný z každého jiného vrcholu. Silně souvislé komponenty jsou maximální podgrafy, které jsou silně souvislé.

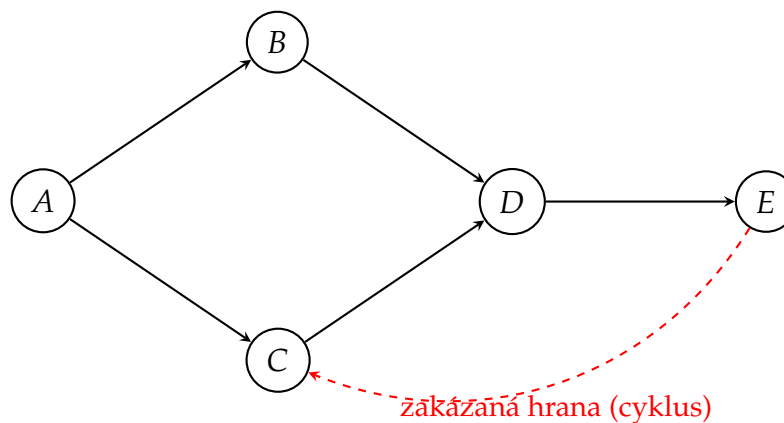
Algoritmy pro hledání silně souvislých komponent:

- **Tarjanův algoritmus pro SCC** – Složitost: $O(V + E)$, kde V je počet vrcholů a E je počet hran.
 1. Provedeme prohledávání do hloubky (DFS) a přiřadíme každému vrcholu u dvě hodnoty:
 - $disc[u]$: čas (pořadí) objevení vrcholu v rámci DFS.
 - $low[u]$: nejmenší $disc$ dosažitelný z u nebo jeho potomků pomocí hran vedoucích do vrcholů, **kteří jsou aktuálně na zásobníku**.
 2. Během průchodu DFS ukládáme navštívené vrcholy na zásobník.
 3. Když se vracíme z rekurze vrcholu u , zkontrolujeme, zda $low[u] == disc[u]$. Pokud ano, u je *kořenem* silně souvislé komponenty.
 4. Následně odebíráme vrcholy ze zásobníku (pop) tak dlouho, dokud neodebereme vrchol u . Všechny tyto odebrané vrcholy (včetně u) tvoří jednu silně souvislou komponentu.

- **Kosarajuův algoritmus** – Složitost: $O(V + E)$.
 1. Provedeme první prohledávání do hloubky (DFS) na původním grafu. Vždy, když dokončíme zpracování vrcholu (návrat z rekurze), **vložíme jej na zásobník**.
 2. Vytvoříme transponovaný (reverzní) graf, ve kterém obrátíme směr všech hran.
 3. Provedeme druhé DFS na transponovaném grafu. Kde začít, určíme tak, že odebíráme vrcholy z vrcholu zásobníku (čímž zajistíme, že začínáme u vrcholů s nejpозdějším časem dokončení z prvního DFS).
 4. Každý strom prohledávání vytvořený v tomto druhém DFS tvoří právě jednu silně souvislou komponentu. Zpracované vrcholy označujeme, abychom je neprocházeli vícekrát.

5.2 Acyklické grafy

Acyklické grafy jsou orientované grafy, které neobsahují žádné orientované cykly. To znamená, že v nich neexistuje žádná orientovaná cesta, která by začínala a končila ve stejném vrcholu. Díky absenci cyklů lze vrcholy těchto grafů vždy seřadit do tzv. **topologického uspořádání**. Acyklické grafy se proto velmi často používá k modelování hierarchií, závislostí mezi úkoly (např. při plánování projektů, kde jeden úkol musí skončit před začátkem dalšího), nebo k reprezentaci genealogických stromů a verzovacích systémů (např. Git).



Obrázek 7: Ukázka orientovaného acyklického grafu. Černé hrany modelují platné závislosti (graf neobsahuje cyklus). Úkoly B a C mohou běžet paralelně, ale úkol D čeká na dokončení obou. Červená přerušovaná hrana ukazuje situaci, která by definici acyklického grafu porušila (vznikl by cyklus $C \rightarrow D \rightarrow E \rightarrow C$, takže by nastal deadlock – nekonečné čekání).

Algoritmus pro detekci acykličnosti:

- **DFS s detekcí zpětných hran** – Složitost: $O(V + E)$.

1. Provedeme prohledávání do hloubky (DFS) a při průchodu sledujeme stav každého vrcholu:
 - *neprozkoumaný* (unvisited)
 - *prozkoumávající* (exploring) – právě procházíme tento vrchol a jeho potomky
 - *prozkoumaný* (explored) – již jsme dokončili průchod tímto vrcholem a všemi jeho potomky
2. Pokud během DFS narazíme na hranu, která vede do vrcholu ve stavu "prozkoumávající", znamená to, že jsme našli zpětnou hranu, což indikuje přítomnost cyklu. V takovém případě graf není acyklický.
3. Pokud DFS dokončí bez nalezení zpětné hrany, graf je acyklický.

5.3 Topologické uspořádání grafu

Topologické uspořádání orientovaného acyklického grafu je lineární uspořádání jeho vrcholů takové, že pro každou hranu (u, v) z u do v platí, že u se v uspořádání nachází před v . Každý orientovaný acyklický graf má alespoň jedno topologické uspořádání, ale může jich mít i více. Topologické uspořádání existuje pouze pro orientované acyklické grafy a je klíčové pro plánování úkolů, kompilaci zdrojového kódu, analýzu závislostí a mnoho dalších aplikací.

Algoritmy pro topologické uspořádání:

- **Kahnův algoritmus** – Složitost: $O(V + E)$.
 1. Vypočítáme vstupní stupeň (počet příchozích hran) pro každý vrchol v grafu.
 2. Vytvoříme frontu a vložíme do ní všechny vrcholy s nulovým vstupním stupněm (vrcholy bez závislostí).
 3. Inicializujeme prázdný seznam pro uložení topologického uspořádání.
 4. Dokud fronta není prázdná:
 - (a) Odebereme vrchol u z fronty a přidáme jej do topologického uspořádání.
 - (b) Pro každou hranu (u, v) vedoucí z u do v snížíme vstupní stupeň vrcholu v o 1.
 - (c) Pokud se vstupní stupeň vrcholu v stane nulovým, vložíme v do fronty.
 5. Po dokončení algoritmu bude seznam obsahovat topologické uspořádání všech vrcholů. Pokud počet zpracovaných vrcholů není roven počtu vrcholů v grafu, znamená to, že graf obsahuje cyklus a topologické uspořádání neexistuje.
- **DFS s ukládáním pořadí dokončení** – Složitost: $O(V + E)$.

1. Provedeme prohlédávání do hloubky (DFS) a při návratu z rekurze každého vrcholu u vložíme u na zásobník a na konci ho vyprázdníme (vyřeší obrácené pořadí).
2. Po dokončení DFS bude seznam obsahovat topologické uspořádání vrcholů. Pokud během DFS narazíme na zpětnou hranu, znamená to, že graf obsahuje cyklus a topologické uspořádání neexistuje.

5.4 Extrémní cesty v acyklickém grafu

Extrémní cesty v orientovaném acyklickém grafu jsou nejkratší nebo nejdelší cesty mezi dvěma vrcholy. Využívají se například v plánování projektů (nejdelší cesta určuje kritickou cestu metodou CPM) nebo v analýze závislostí. V běžných grafech je problém hledání nejdelší cesty NP-těžký, ale v acyklických grafech lze tento problém řešit efektivně v polynomiálním čase pomocí dynamického programování.

Algoritmus pro hledání nejdelší cesty v acyklickém grafu:

- **Dynamické programování pomocí topologického uspořádání** – Složitost: $O(V + E)$.
 1. Nejprve provedeme topologické uspořádání vrcholů grafu.
 2. Inicializujeme pole $dist$ pro všechny vrcholy s hodnotou $-\infty$, kromě počátečního (zdrojového) vrcholu, který nastavíme na 0 (nebo na váhu počátečního vrcholu, pokud mají uzly vlastní ohodnocení).
 3. Pro každý vrchol u v přesném pořadí podle topologického uspořádání:
 - Pokud je $dist[u] \neq -\infty$, pro každou hranu (u, v) vedoucí z u do v provedeme tzv. relaxaci hrany (aktualizaci vzdálenosti):

$$dist[v] = \max(dist[v], dist[u] + w(u, v))$$

kde $w(u, v)$ je váha hrany (u, v) .

- *Vysvětlení mechanismu na příkladu projektu:* Představte si, že jste právě dokončili milník u (např. „Hrubá stavba hotova“) v čase $dist[u]$. Nyní se díváte na úkol, který na něj navazuje (např. „Montáž střechy“), což je hrana do milníku v .
 - * **Výpočet ($dist[u] + w(u, v)$):** Říkáte si: „Hrubá stavba skončila 10. den a střecha trvá 5 dní. Takže přes tuhle cestu by mohlo být hotovo nejdříve 15. den.“
 - * **Porovnání (max):** Jenže u milníku v („Střecha hotova“) už může být v deníku napsaný jiný termín (např. 17. den), protože střecha závisí i na jiném úkolu (třeba „Dodávka materiálu“), který trvá déle. ($dist[v]$ je již zapsaná hodnota ve vrcholu, popřípadě $-\infty$ pokud jde o první nsavštění)

- * **Výsledek:** Funkce *max* zajistí, že v deníku u milníku *v* zůstane ten **pozdější** čas. Tím algoritmus v podstatě říká: „Milník *v* nastane až tehdy, kdy doběhne úplně poslední (nejdelší) z úkolů, které k němu vedou.“
- 4. Po zpracování všech vrcholů obsahuje pole *dist* délku nejdelší cesty z počátečního vrcholu do každého dosažitelného vrcholu. Hodnota pro konkrétní cíl bude uložena v *dist[cíl]*.

Poznámka: Algoritmus pro hledání **nejkratší cesty** v orientovaném acyklickém grafu je naprosto identický. Stačí pouze pole *dist* inicializovat hodnotou $+\infty$ a místo funkce *max* použít funkci *min*. Díky topologickému uspořádání tak acyklické grafy nepotřebují složitější Dijkstrův ani Bellman-Fordův algoritmus.

5.5 Metody síťové analýzy

Síťová analýza je metoda používaná především pro plánování a řízení projektů. Vychází ze struktury sítí činností a milníků, kde:

- **Uzly** mohou představovat milníky (události) nebo činnosti v závislosti na typu modelu.
- **Hrany** reprezentují vazby mezi činnostmi, často s uvedenou délkou nebo trváním.
- **Cesta** znamená posloupnost činností vedoucí od začátku projektu k jeho ukončení.

Základními úlohami síťové analýzy jsou :

- výpočet *nejdřívějších* a *nejpozdějších* časů zahájení a ukončení činností
- určení **rezerv** činností
- nalezení **kritické cesty**, tj. nejdelší cesty v acyklické síti, která určuje dobu trvání celého projektu

Činnost je na kritické cestě tehdy, když její **rezerva** je nulová. Rezerva se obvykle počítá jako rozdíl mezi nejpozdějším a nejdřívějším možným časem zahájení nebo ukončení. Činnosti bez rezervy nemohou být zpožděny, jinak se prodlouží celý projekt.

V praktickém použití se síťová analýza často kombinuje s algoritmy na orientovaných acyklických grafech. Díky topologickému uspořádání lze efektivně spočítat nejdřívější časy a najít kritické cesty v čase $O(V + E)$, což je přímá aplikace dříve uvedeného postupu pro nejdelší cesty v acyklických orientovaných grafech.

CPM (Critical Path Method): Používá pevná trvání činností a spočítá nejdřívejší a nejpozdější časy pomocí dvou průchodů sítí:

1. Přední průchod: spočítají se nejdřívejší časy pro každou událost na základě maximálních hodnot z předchozích uzlů. (ES a EF)
2. Zadní průchod: spočítají se nejpozdější časy tak, aby bylo dodrženo celkové datum ukončení projektu. (LS a LF)

Video na vysvětlení.

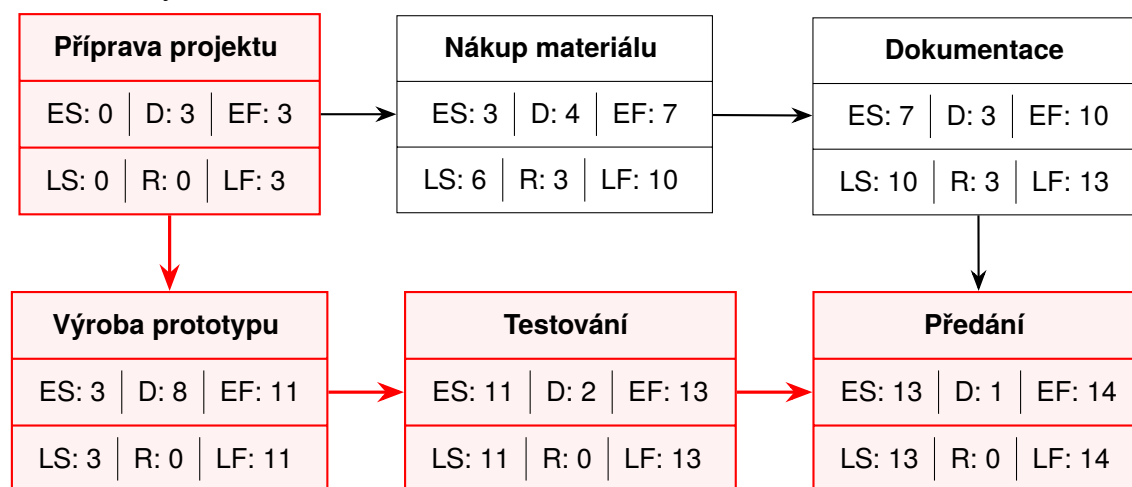


Diagram metody kritické cesty (CPM). Význam hodnot v uzlech: **ES** (Earliest Start) = nejdříve možný začátek, **D** (Duration) = doba trvání, **EF** (Earliest Finish) = nejdříve možný konec, **LS** (Latest Start) = nejpozději přípustný začátek, **R** (Reserve) = celková časová rezerva, **LF** (Latest Finish) = nejpozději přípustný konec. Uzly a vazby vyznačené **červeně** leží na kritické cestě ($R = 0$).

PERT (Program Evaluation and Review Technique): PERT pracuje s neurčitostí trvání činností. Pro každou činnost se používají tři odhady: optimistický t_o , nejpravděpodobnější t_m a pesimistický t_p . Odhadnuté očekávané trvání se potom počítá jako

$$t_e = \frac{t_o + 4t_m + t_p}{6}$$

a rozptyl jako

$$\sigma^2 = \left(\frac{t_p - t_o}{6} \right)^2.$$

Na základě těchto hodnot lze určit nejdřívejší a nejpozdější časy podobně jako v CPM a odhadnout pravděpodobnost dokončení projektu do určitého termínu.

5.6 Toky a maximální tok v sítích

Síť toku (transportní síť) je orientovaný graf $G = (V, E)$, kde je každé hraně (u, v) přiřazena nezáporná **kapacita** $c(u, v) \geq 0$. Pokud hrana mezi dvěma uzly neexistuje, je její kapacita 0.

Základní prvky sítě:

- **Zdroj (Source, s):** Vrchol, ve kterém tok do sítě vstupuje (vzniká). Z hlediska analýzy z něj tok celkově pouze odtéká.
- **Ústí / Spotřebič (Sink, t):** Vrchol, ve kterém tok ze sítě vystupuje (zaniká). Představuje cíl transportu.
- **Vnitřní uzly:** Všechny ostatní uzly v grafu $(V \setminus \{s, t\})$. Slouží jako překladiště, křižovatky či routery. Hrany do nich mohou vstupovat i z nich vystupovat.

Definice toku: Tok v síti je funkce f , která každé orientované hraně přiřadí reálné číslo (množství protékajícího materiálu, dat apod.). Aby byl tok **přípustný**, musí splňovat dvě základní podmínky:

1. **Kapacitní omezení:** Tok hranou nesmí překročit její kapacitu a nesmí být záporný. Pro každou hranu platí: $0 \leq f(u, v) \leq c(u, v)$.
2. **Zákon zachování toku (Kirchhoffův zákon):** Pro každý vnitřní uzel platí, že součet toků do něj vstupujících se musí přesně rovnat součtu toků z něj vystupujících (materiál se uvnitř uzlu neztrácí ani nehromadí):

$$\sum_{\text{vstupující } u} f(u, v) = \sum_{\text{vystupující } w} f(v, w)$$

Problém maximálního toku: Cílem je najít takový tok, jehož **velikost** (celkový součet toků vycházejících ze zdroje s , což je to samé jako součet toků vstupujících do ústí t) je co největší.

Klíčové pojmy pro algoritmické řešení:

- **Reziduální síť:** Pomocný orientovaný graf, který ukazuje, *kolik kapacity ještě zbývá*. Skládá se ze dvou typů hran:
 - *Dopředné hrany:* Ukazují zbývající volnou kapacitu $(c(u, v) - f(u, v))$.

- *Zpětné hrany*: Vznikají proti směru aktuálního toku a mají kapacitu rovnou $f(u, v)$. Umožňují algoritmu dříve poslaný tok „odvolat“ (vrátit zpět) a poslat ho jinou cestou, pokud to pomůže najít lepší globální řešení.
- **Zlepšující cesta**: Jakákoliv orientovaná cesta ze zdroje s do ústí t nalezená v *reziduální síti*. Pokud existuje, znamená to, že velikost toku lze ještě zvětšit.

Algoritmy pro hledání maximálního toku:

- **Ford-Fulkersonova metoda**
 1. Začneme s nulovým tokem na všech hranách ($f(u, v) = 0$).
 2. Zkonstruujeme reziduální síť.
 3. Dokud existuje zlepšující cesta v reziduální síti (hledáme např. pomocí DFS nebo BFS):
 - Najdeme na této cestě hranu s nejmenší reziduální kapacitou c_f (tzv. úzké hrdlo cesty).
 - Zvětšíme celkový tok podél této cesty o hodnotu úzkého hrdla c_f .
 - Aktualizujeme kapacity v reziduální síti (snížíme kapacity dopředných hran a zvýšíme kapacity zpětných hran na dané cestě).
 4. Algoritmus končí, když nelze najít žádnou další zlepšující cestu. Získaný tok je maximální.
- **Edmonds-Karpův algoritmus** – Složitost: $O(VE^2)$. Jde o konkrétní, optimalizovanou implementaci Ford-Fulkersonovy metody. Její specifikum spočívá v tom, že pro hledání zlepšující cesty používá výhradně **prohledávání do šířky (BFS)**. Díky tomu v každém kroku najde cestu s *nejmenším počtem hran*. To zaručuje polynomiální časovou složitost a zabraňuje nekonečnému zacyklení u grafů s iracionálními kapacitami.

Věta o maximálním toku a minimálním řezu (Max-flow Min-cut Theorem): Tato fundamentální věta říká, že **velikost maximálního toku v síti je přesně rovna kapacitě minimálního $s - t$ řezu**. Minimální řez je nejlevnější možné rozdělení grafu na dvě části (jedna obsahuje zdroj s , druhá ústí t) tak, že po přerušení těchto hran nemůže do ústí natéct žádný tok. Hrany, které toto „úzké hrdlo celé sítě“ tvoří, jsou právě ty, které jsou při maximálním toku plně nasycené (reziduální kapacita je 0).

6 NP úplné problémy z oblasti diskrétní matematiky

Problém obarvení grafu, problém čtyř barev, problém existence a nalezení kliky a nezávislé množiny v grafu, problém nalezení Hamiltonských kružnic v grafu, problém obchodního cestujícího.

NP-úplné problémy jsou problémy které nelze řešit polynomiálně.

6.1 Problém obarvení grafu

Obarvení grafu G je ohodnocení vrcholů hodnotami z množiny B (barev), a to takové, že žádné dva sousední vrcholy nejsou ohodnoceny (obarveny) stejnou barvou. V roli barev můžeme brát libovolné prvky z libovolné množiny, často používáme např. čísla. Graf nazýváme r - barevným, jestliže existuje jeho obarvení r barvami. Barevnost grafu (chromatické číslo) je nejmenší počet barev, který je potřebný k obarvení grafu, značíme $\chi(G)$.

Chromatické číslo je rovno minimu počtu barev potřebných k obarvení grafu. Chromatické číslo je rovné největší klice. Kromě speciálních grafů, např. bipartitních, patří tento problém mezi NP-úplné.

Problém čtyř barev Problém čtyř barev byl původně formulován pro politické mapy států: Výrobci map chtěli státy barevně odlišit, aby každé dva sousední státy měly různé barvy. Přitom chtěli mapy tisknout co nejlevněji, tedy s nejmenším možným počtem barev. Praxe brzy ukázala, že vždy stačí 4 barvy. Problém barvení map lze převést na barevnost rovinných grafů (graf je rovinný, když se dá nakreslit do roviny bez přetínání hran) a platí následující věta.

Věta (Problém čtyř barev): Barevnost každého rovinného grafu je ≤ 4 .

Algoritmy pro zjišťování barevnosti:

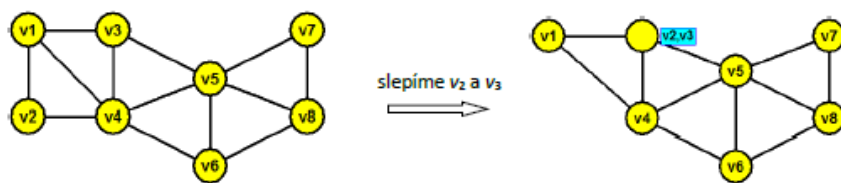
- **Barvení grafu pomocí nezávislých množin** – Tento algoritmus je založen na faktu, že množina vrcholů, které jsou obarveny stejnou barvou, tvoří nezávislou množinu grafu. Algoritmus používá heuristický algoritmus hledání maximálních nezávislých množin uvedený předtím.

Označení použita v algoritmu:

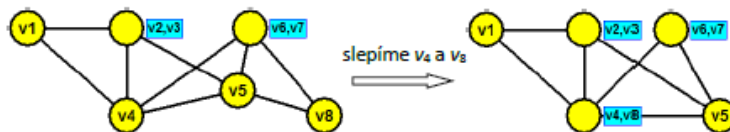
- b - proměnná označující aktuální barvu.
- G' - podgraf grafu G indukovaný doposud neobarvenými vrcholy.

1. Počáteční podmínky.

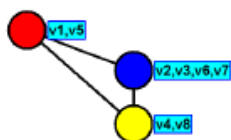
- a. $b = 1$, nastavíme první barvu.
 - b. $G' = G$, na začátku obsahuje podgraf G' celý výchozí graf G , tj. žádný vrchol není zatím obarven.
 2. V podgrafu G' nalezneme maximální nezávislou množinu vrcholů M , obarvíme ji aktuální barvou a následně odstraníme z G' .
 3. Pokud podgraf G' není ještě prázdný, pak:
 - a. $b = b + 1$
 - b. opakujeme 2.krok, obarvení další barvou.
- **Barvení grafu slepováním vrcholů** – Algoritmus provádí postupné spojování vrcholů, které lze obarvit stejnou barvou, tedy nesousedních vrcholů. Výsledkem je úplný graf K_v , jehož každý vrchol $w_i = v_{i_1}v_{i_2} \dots v_{i_r}$, $i = 1, 2, \dots, p$; i_r značí počet vrcholů slepených do vrcholu w_i , obarvíme jinou barvou. Vrcholům $v_{i_1}v_{i_2}, \dots, v_{i_r}$ přiřadíme tutéž barvu, jakou byl obarven vrchol w_i , $i = 1, 2, \dots, p$.
 1. Vytvoření pomocného úplného grafu.
 - a. Určení slepovaných vrcholů. V grafu G určíme dva navzájem různé vrcholy v_1, v_2 nespojené hranou.
 - b. Slepění vrcholů. Provedeme spojení těchto vrcholů tak, že je nahradíme jedním vrcholem v_0 . Nový vrchol v_0 spojíme hranami se všemi vrcholy, s kterými byl spojen vrchol v_1 a taktéž se všemi vrcholy, se kterými byl spojen vrchol v_2 . Tento krok opakujeme, dokud lze spojovat nějaké dvojice vrcholů, tedy dokud graf G není zredukován na úplný graf.
 2. Obarvení grafu. Vytvořený úplný graf obarvíme. Každý vrchol úplného grafu je obarven jinou barvou. Následně začneme zpětně rozdělovat spojené vrcholy, přičemž vždy stejnou barvu jakou měl vrchol v_0 , dáme rovněž vrcholům v_1, v_2 , jejichž spojením vrchol vznikl. Proces opakujeme tak dlouho, dokud nezískáme původní graf.



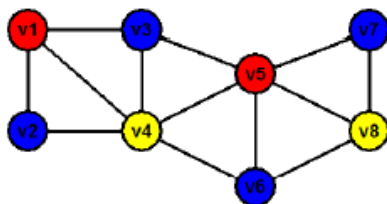
slepíme v_6 a v_7



Graf obarvíme:



Nyní obarvíme i původní graf. Červenou barvou obarvíme vrcholy v_1 a v_5 . Modrou barvou vrcholy v_3 , v_2 , v_7 a v_6 a žlutou barvou vrcholy v_4 a v_8 .



Obrázek 8: Barvení grafu slepováním vrcholů

6.2 Problém existence a nalezení kliky a nezávislé množiny v grafu

Definice a vzájemný vztah:

- **Klika** v grafu G je podmnožina vrcholů taková, že každý vrchol z této množiny je spojen hranou s každým jiným vrcholem z této množiny (jde o úplný podgraf).
- **Nezávislá množina** je podmnožina vrcholů taková, že mezi žádnými dvěma vrcholy z této množiny neexistuje hrana.
- **Dualita:** Vrcholy tvořící kliku v grafu G tvoří nezávislou množinu v doplňkovém grafu $\overline{G}$ (graf se stejnými vrcholy, kde hrany existují právě tam, kde v původním grafu chyběly). Díky této vlastnosti lze jakýkoliv algoritmus pro hledání kliky použít i pro hledání nezávislé množiny a naopak.

Problém existence (Rozhodovací úloha): Rozhodovací verze problému zní: „Existuje v grafu G klika (nebo nezávislá množina) o velikosti alespoň k ?“

Algoritmy pro nalezení největší kliky: Hledání **největší (maximum)** kliky je **NP-těžký** problém. V praxi se používají dva přístupy:

1. Exaktní algoritmy (např. Bron-Kerboschův algoritmus):

- Jde o algoritmus typu *backtracking* (zpětné prohledávání).
- Rekurzivně prohledává všechny kandidáty na kliku a využívá ořezávání větví (pruning), které prokazatelně nemohou vést k lepším výsledkům.
- V nejhorším případě má exponenciální složitost, ale pro mnoho reálných grafů je překvapivě rychlý.

2. Heuristické a aproximační algoritmy:

- *Hladový algoritmus (Greedy):* Začneme s jedním vrcholem a postupně přidáváme další vrcholy, které jsou spojené se všemi již vybranými. Algoritmus skončí rychle ($O(V^2)$), ale nezaručuje nalezení *největší* kliky, pouze *maximální* (takovou, kterou už nelze dále rozšířit).
- Používá se v aplikacích, kde nepotřebujeme absolutní optimum, ale rychlý výsledek (např. analýza sociálních sítí, bioinformatika).

Praktické aplikace:

- **Klika:** Identifikace komunit v sociálních sítích (skupiny lidí, kde se každý zná s každým), vyhledávání podobných vzorců v proteinech.
- **Nezávislá množina:** Rozvrhování (úkoly, které spolu kolidují, jsou spojeny hranou; nezávislá množina pak představuje úkoly, které lze provést současně), alokace registrů v kompilátorech.

6.3 Problém nalezení Hamiltonských kružnic v grafu

Hamiltonovská kružnice v grafu G je taková kružnice, která obsahuje každý vrchol grafu právě jednou (s výjimkou počátečního a koncového vrcholu, které splývají, tedy začíná a končí ve stejném vrcholu). Pokud v grafu existuje alespoň jedna taková kružnice, nazýváme graf **hamiltonovským**.

Složitost problému: Na rozdíl od hledání Eulerovských tahů (kde stačí kontrolovat sudost stupňů vrcholů v čase $O(E)$), je problém rozhodnutí, zda je obecný graf hamiltonovský, **NP-úplný**. Neexistuje žádná jednoduchá nutná a postačující podmínka (ekvivalence), kterou by bylo možné snadno ověřit pro všechny grafy.

Postačující podmínky (Věty o existenci): I když neznáme obecné pravidlo, existují věty, které zaručují existenci Hamiltonovské kružnice v grafech s dostatečně velkým počtem hran:

- **Diracova věta:** Má-li graf G s n vrcholy ($n \geq 3$) každý vrchol stupně alespoň $n/2$, pak je graf hamiltonovský.
- **Oreho věta:** Pokud pro každou dvojici nesusousedních vrcholů u, v v grafu o n vrcholech platí $\deg(u) + \deg(v) \geq n$, pak je graf hamiltonovský.

Algoritmy pro nalezení Hamiltonovské kružnice: Vzhledem k NP-úplnosti se pro hledání používají algoritmy s exponenciální časovou složitostí nebo heuristiky:

1. Backtracking (Hledání hrubou silou):

- Algoritmus postupně buduje cestu přidáváním sousedních a dosud nenavštívených vrcholů.
- Pokud narazí na slepou uličku, vrátí se o krok zpět (backtrack) a zkusí jinou větev.
- Složitost je v nejhorším případě $O(n!)$.

2. Dynamické programování (Held-Karpův algoritmus):

- Snižuje složitost na $O(n^2 2^n)$. To je stále exponenciální, ale pro středně velké grafy (do cca 20–25 vrcholů) mnohem rychlejší než hrubá síla.

3. Heuristiky (např. Bondyho-Chvátalův algoritmus):

- Využívají sestrojení tzv. *uzávěru grafu*. Postupně spojujeme nesousední vrcholy, jejichž součet stupňů je alespoň n . Pokud je uzávěr grafu úplný graf K_n , pak je původní graf hamiltonovský.

Poznámka: Praktickým příkladem hledání Hamiltonovské cesty je tzv. **Problém jezdce** na šachovnici, kde figurka jezdce musí navštívit každé pole šachovnice právě jednou.

6.4 Problém obchodního cestujícího (TSP)

Problém obchodního cestujícího je definován na hranově ohodnoceném grafu a cílem je najít Hamiltonovskou kružnici s minimálním součtem vah hran.

Formální definice: Mějme úplný graf $G = (V, E)$ a funkci $w : E \rightarrow \mathbb{R}^+$, která každé hraně přiřazuje cenu (vzdálenost). Hledáme permutaci vrcholů $(v_1, v_2, \dots, v_n)$ takovou, aby součet:

$$W = \sum_{i=1}^{n-1} w(v_i, v_{i+1}) + w(v_n, v_1)$$

byl minimální.

Složitost a varianty:

- **NP-těžkost:** TSP patří mezi NP-těžké problémy. Rozhodovací verze („Existuje okružní cesta s cenou $\leq K$ “) je NP-úplná.
- **Symetrický TSP:** vzdálenost z A do B je stejná jako z B do A .
- **Asymetrický TSP:** Cesty mohou mít v různých směrech různé ceny (např. vliv jednosměrných ulic nebo převýšení).
- **Metrický TSP:** Splňuje trojúhelníkovou nerovnost ($w(a, c) \leq w(a, b) + w(b, c)$). Tento případ je v praxi nejčastější a umožňuje použití efektivních aproximačních algoritmů.

Algoritmy pro řešení: Vzhledem k výpočetní náročnosti se volba algoritmu odvíjí od velikosti grafu:

1. Exaktní algoritmy (přesné řešení):

- **Hrubá síla:** Prozkoumání všech $(n - 1)!/2$ možných kružnic. Použitelné jen pro velmi malé grafy ($n < 12$).
- **Held-Karpův algoritmus (DP):** Využívá dynamické programování se složitostí $O(n^2 2^n)$.
- **Metoda větví a mezí (Branch and Bound):** Systematicky prohledává stavový prostor a odřezává větve, u kterých lze dokázat, že neobsahují optimální řešení.

2. Heuristiky (rychlé, ale ne nutně optimální):

- **Metoda nejbližšího souseda (Greedy):** Z aktuálního vrcholu pokračujeme vždy do nejbližšího dosud nenavštíveného vrcholu. Je velmi rychlá, ale může vytvořit velmi špatné řešení, pokud je nucena na konci použít dlouhou hranu pro uzavření kružnice.
- **Lokální vyhledávání (2-opt):** Vezmeme hotovou kružnici a zkusíme prohodit dvě hrany, zda se tím celková délka nezkrátí.

3. Aproximační algoritmy (garantovaná chyba):

- **Algoritmus využívající minimální kostru:** Pro metrický TSP zaručuje, že výsledek nebude horší než dvojnásobek optima (2-aproximace).
- **Christofidův algoritmus:** Pokročilejší metoda kombinující minimální kostru a párování, poskytuje 1,5-aproximaci.

Praktické aplikace: TSP se používá například v logistice při rozvozu zboží, v genetice při sestavování řetězců DNA nebo při návrhu mikročipů.

7 Problematika numerických metod a řešení nelineárních rovnic

Zdroje chyb pro numerické výpočty, systém čísel s pohyblivou řádovou čárkou a reprezentace reálných čísel v počítači, chyby v numerických výpočtech (absolutní chyba, relativní chyba, vlastnosti numerických algoritmů a asymptotická přesnost). Nelineární funkce, její nulový bod a nelineární rovnice, separace kořenů nelineárních rovnic, základní metody hledání kořenů, odhady přesnosti a podmínky konvergence.

Numerická matematika zkoumá metody, které umožňují přibližně řešit matematické úlohy aritmetickými prostředky. Aritmetickými prostředky se rozumí konečná posloupnost základních aritmetických operací včetně porovnávání reálných čísel.

7.1 Zdroje chyb pro numerické výpočty

Numerické výpočty nejsou nikdy zcela přesné. Při jejich realizaci vznikají různé typy chyb, které ovlivňují spolehlivost výsledku. Celkovou chybu výpočtu lze rozdělit do následujících kategorií:

Druhy chyb:

- **Chyby matematického modelu:** Vznikají již při popisu reálného problému pomocí matematických rovnic. Jsou způsobeny zjednodušením reality, kdy zanedbáváme určité vlivy (např. odpor vzduchu při volném pádu, předpoklad dokonalé tuhosti tělesa). Patří sem i chyby ve vstupních datech, která jsou často získána měřením s omezenou přesností.
- **Chyby numerické metody (chyby uříznutím, diskretizační chyby):** Vznikají nahrazením exaktní matematické úlohy její numerickou aproximací. Většina numerických metod nahrazuje nekonečné procesy konečnými.
 - Příkladem je nahrazení nekonečného součtu Taylorovy řady pouze jejími prvními n členy.
 - Nahrazení spojitě derivace nebo integrálu konečným součtem (diskretizace).
 - Ukončení iteračního procesu po dosažení určitého počtu kroků, nikoliv v absolutním nekonečnu.
- **Zaokrouhlovací chyby:** Jsou důsledkem práce s počítačem, který má omezenou paměť pro uložení čísel. Reálná čísla jsou v počítači reprezentována v pohyblivé řádové čárce (standard IEEE 754) s konečným počtem platných cifer.
 - Čísla jako π , e nebo i zdánlivě jednoduchá čísla (např. 0.1, které má v binární soustavě nekonečný rozvoj) nelze uložit přesně.

- Tyto chyby se mohou během milionů operací kumulovat. Nebezpečný je zejména jev *odečítání dvou blízkých čísel* (cancellation), kdy dojde k drastické ztrátě platných cifer.

Šíření chyb: Při výpočtech vstupují chyby z předchozích kroků do kroků následujících. Pokud algoritmus chyby v průběhu výpočtu zvětšuje, nazýváme jej **numericky nestabilní**. Cílem numerické matematiky je navrhovat algoritmy **stabilní**, u kterých zůstává vliv počátečních a zaokrouhlovacích chyb pod kontrolou.

7.2 Systém čísel s pohyblivou řádovou čárkou a reprezentace reálných čísel v počítači

V počítači nelze uchovávat reálná čísla s nekonečnou přesností, protože paměť je omezená. Množina čísel, která lze v počítači reprezentovat, je **konečná a diskrétní** (čísla netvoří spojitou osu, ale jsou mezi nimi mezery, které se směrem od nuly zvětšují).

Pro uložení reálných čísel se dnes téměř výhradně používá **systém s pohyblivou řádovou čárkou** (Floating-Point Number System), který je standardizován normou **IEEE 754**.

Matematický model pohyblivé řádové čárky: Každé reprezentovatelné číslo x je uloženo ve tvaru:

$$x = (-1)^s \cdot m \cdot \beta^e$$

kde:

- **s znaménko (sign):** Určuje kladné ($s = 0$) nebo záporné ($s = 1$) číslo.
- **β základ (base):** Dnes prakticky vždy $\beta = 2$ (dvojková soustava).
- **m mantisa (significand):** Určuje platné číslice a přesnost čísla. Většinou se ukládá v *normalizovaném tvaru* $1, d_1 d_2 \dots d_p$. Protože první číslice před čárkou je v binární soustavě vždy 1, v paměti se neukládá (tzv. skrytý bit - ušetří se 1 bit paměti).
- **e exponent:** Určuje řád čísla (posouvá desetinnou čárku). Je uložen s tzv. posunutím (bias), aby mohl nabývat kladných i záporných hodnot bez nutnosti ukládat znaménko exponentu.

Standard IEEE 754:

- **Single precision (32 bitů, typ float):** 1 bit znaménko, 8 bitů exponent, 23 bitů mantisa.
- **Double precision (64 bitů, typ double):** 1 bit znaménko, 11 bitů exponent, 52 bitů mantisa.

Strojové epsilon (ε_m): Strojové epsilon je definováno jako nejmenší možné kladné číslo, pro které v počítačové aritmetice platí:

$$1 + \varepsilon_m > 1$$

Určuje maximální relativní zaokrouhlovací chybu, která může vzniknout při reprezentaci reálného čísla v počítači. Cokoliv menšího než ε_m přičteno k jedničce se kvůli omezené mantise „ztratí“ a počítač to vyhodnotí jako 1.

Důsledky pro numerické výpočty (Praktický dopad):

1. **Neplatí asociativita sčítání:** Z matematiky víme, že $(a + b) + c = a + (b + c)$. V počítači to **neplatí**. Pokud sečteme obrovské číslo s malinkým, malé se ztratí. Na pořadí operací tedy záleží.
2. **Ztráta platných číslic (Cancellation):** Při odčítání dvou velmi blízkých čísel se odečtou jejich nejdůležitější platné číslice a zbydou jen ty nejméně významné (často ovlivněné zaokrouhlovací chybou). Relativní chyba výsledku v tu chvíli vzroste.

7.3 Chyby v numerických výpočtech

Měření chyby: Pro kvantifikaci nepřesnosti definujeme dva základní pojmy. Nechť x je přesná hodnota a $\tilde{x}$ je její aproximace:

1. **Absolutní chyba ($E(\tilde{x})$):** Rozdíl mezi přesnou hodnotou a aproximací. Udává se ve stejných jednotkách jako měřená veličina.

$$E(\tilde{x}) = \tilde{x} - x$$

2. **Relativní chyba ($R(\tilde{x})$):** Poměr absolutní chyby k velikosti přesné hodnoty (často vyjádřen v procentech). Poskytuje lepší představu o závažnosti chyby vzhledem k měřítku problému.

$$R(\tilde{x}) = \frac{\tilde{x} - x}{x}, \quad x \neq 0$$

Vlastnosti numerických úloh a algoritmů:

- **Podmíněnost úlohy:** Vlastnost samotného matematického problému (nezávisí na algoritmu). Vyjadřuje citlivost výstupu na malé změny ve vstupních datech.
 - *Dobře podmíněná úloha:* Malá chyba na vstupu způsobí jen malou chybu na výstupu.
 - *Špatně podmíněná úloha:* I drobná nepřesnost vstupu vede k obrovské chybě výsledku.

- **Stabilita algoritmu:** Vlastnost numerické metody. Stabilní algoritmus zaručuje, že se zaokrouhlovací chyby vzniklé během výpočtu nebudou nekontrolovatelně hromadit a zvětšovat (algoritmus je tlumí nebo alespoň nezveličuje).
- **Konvergence:** Zaručuje, že pokud zjemňujeme dělení (např. krok $h \rightarrow 0$) nebo zvyšujeme počet iterací ($k \rightarrow \infty$), numerické řešení $\tilde{x}$ se limitně blíží přesnému řešení x .

Asymptotická přesnost (Řád metody): Asymptotická přesnost popisuje **rychlost konvergence** numerické metody, tedy jak rychle klesá chyba při zmenšování kroku výpočtu (např. kroku h při integrování nebo derivování).

Vyjadřuje se pomocí tzv. *Velkého O* zápisu: $O(h^p)$, kde p je **řád metody**.

- Pokud má metoda chybu úměrnou $O(h^p)$, říkáme, že je **metodou řádu p** .
- *Praktický důsledek:* Pokud krok h zmenšíme na polovinu ($h/2$), chyba u metody 1. řádu ($p = 1$) klesne zhruba na polovinu. U metody 2. řádu ($p = 2$) chyba klesne zhruba na **čtvrtinu** ($2^2 = 4$). Vyšší řád tedy znamená mnohem rychlejší dosažení přesného výsledku.

7.4 Nelineární funkce, její nulový bod a nelineární rovnice, separace kořenů nelineárních rovnic, základní metody hledání kořenů, odhady přesnosti a podmínky konvergence

Nelineární rovnice jedné proměnné je rovnice tvaru:

$$f(x) = 0$$

kde $f(x)$ je nelineární reálná funkce (např. polynom vyššího stupně, goniometrická, exponenciální či logaritmická funkce). Číslo $\xi \in \mathbb{R}$, pro které platí $f(\xi) = 0$, se nazývá **kořen rovnice** nebo **nulový bod funkce**.

U většiny nelineárních rovnic nelze najít kořeny analyticky (přesným vzorcem). Proto se využívají numerické (iterační) metody, které k přesnému řešení ξ konvergují postupným přibližováním. Proces řešení se obvykle dělí do dvou fází:

1. **Separace kořenů** (hrubý odhad polohy kořenů).
2. **Iterační zpřesňování** vybranou numerickou metodou.

1. Separace kořenů nelineárních rovnic: Separovat kořeny znamená najít disjunktní (nepřekrývající se) intervaly $I = [a, b]$ takové, že v každém z nich leží **právě jeden kořen**.

Bolzanova věta:

- *Podmínka existence:* Pokud je funkce $f(x)$ na intervalu $[a, b]$ spojitá a v krajních bodech má opačná znaménka, tj. $f(a) \cdot f(b) < 0$, pak v otevřeném intervalu (a, b) existuje alespoň jeden kořen.
- *Podmínka jednoznačnosti:* Pokud je funkce $f(x)$ na tomto intervalu navíc *ryze monotónní* (např. první derivace $f'(x)$ nemění znaménko a je nenulová), pak se v intervalu nachází **právě jeden kořen**.

V praxi se separace provádí *graficky* (nakreslením grafu funkce nebo průsečíku dvou jednodušších funkcí) nebo *tabelací* (krokujeme po ose x a hledáme změnu znaménka funkční hodnoty).

2. Odhady přesnosti (Ukončovací kritéria): Protože iterační metody generují nekonečnou posloupnost aproximací $x_0, x_1, x_2, \dots \rightarrow \xi$, musíme výpočet v určitém okamžiku zastavit. Nechť $\varepsilon > 0$ je požadovaná přesnost (tolerance). Nejčastější kritéria jsou:

1. **Kritérium absolutní odchylky v x :** Zastavíme, pokud je vzdálenost dvou po sobě jdoucích iterací dostatečně malá: $|x_k - x_{k-1}| < \varepsilon$.
2. **Kritérium relativní odchylky:** Vhodnější pro velmi velká nebo velmi malá čísla: $\frac{|x_k - x_{k-1}|}{|x_k|} < \varepsilon$.
3. **Reziduální kritérium (Odchylka v y):** Hodnota funkce je dostatečně blízko nule: $|f(x_k)| < \varepsilon$.

3. Základní metody hledání kořenů a podmínky konvergence:

A. Metoda půlení intervalu (Bisekce): Vychází ze separovaného intervalu $[a, b]$, kde $f(a) \cdot f(b) < 0$.

Střed intervalu $c = \frac{a+b}{2}$ rozdělí interval na dvě poloviny. Podle toho, ve které polovině došlo ke změně znaménka (zda $f(a) \cdot f(c) < 0$ nebo $f(c) \cdot f(b) < 0$), vybereme nový, poloviční interval a postup opakujeme.

- **Konvergence:** Metoda **vždy konverguje** pro jakoukoliv spojitou funkci. Je to nejjistější metoda.
- **Odhad přesnosti:** Po k krocích je absolutní chyba menší nebo rovna polovině délky aktuálního intervalu: $|\xi - x_k| \leq \frac{b-a}{2^{k+1}}$.
- **Rychlost:** Lineární (velmi pomalá). Získáme zhruba jeden platný bit přesnosti s každým krokem.

B. Metoda prosté iterace: Původní rovnici $f(x) = 0$ algebraicky upravíme na ekvivalentní tvar $x = \varphi(x)$. Hledání kořene $f(x)$ se tak mění na hledání tzv. *pevného bodu* funkce φ . Funkce $\varphi(x)$ by měla být injektivní a kontraktivní.

- **Vzorec pro iteraci:**

$$x_{k+1} = \varphi(x_k)$$

- **Podmínky konvergence (Banachova věta o pevném bodě):** Iterační proces bude konvergovat pro libovolný počáteční bod $x_0 \in [a, b]$, pokud je funkce φ na tomto intervalu kontrakcí. Prakticky to znamená, že musí platit:

$$|\varphi'(x)| \leq L < 1 \quad \text{pro všechna } x \in [a, b]$$

Čím menší je derivace (čím blíže k nule), tím rychleji metoda konverguje. Pokud je derivace větší než 1, metoda diverguje a rovnici $f(x) = 0$ je nutné přepsat do jiného tvaru $\varphi(x)$.

C. Newtonova metoda (Metoda tečen): Z počátečního odhadu x_0 vedeme ke grafu funkce tečnu. Průsečík této tečny s osou x se stává novým odhadem x_1 . Využívá se Taylorova rozvoje.

- **Vzorec pro iteraci:**

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

- **Podmínky konvergence (Fourierovy podmínky):**

1. Funkce je na intervalu $[a, b]$ dvakrát spojitě diferencovatelná. (Spojitě diferencovatelná funkce je taková funkce, která má v každém bodě svého definičního oboru derivaci a tato její derivace je zároveň spojitou funkcí.)
2. První derivace nemění znaménko (funkce roste nebo klesá).
3. Druhá derivace nemění znaménko (funkce je stále konvexní nebo konkávní).
4. Výběr počátečního bodu x_0 : Musí platit $f(x_0) \cdot f''(x_0) > 0$. $\implies$ Začínáme na tom okraji intervalu, kde má funkce a její druhá derivace stejné znaménko.

- **Rychlost:** Pokud konverguje, je kvadratická (počet přesných cifer se v každém kroku zdvojnásobí). Metoda je extrémně rychlá, ale lokální (musíme začít blízko kořene, jinak může divergovat).

Další: Např. metoda sečen.

8 Aproximace funkcí, numerický výpočet derivace a integrálu funkcí

Interpolační polynom (charakteristika a metody výpočtu), interpolační splajny, aproximace funkce metodou nejmenších čtverců. Numerické derivování, základní formule, chyba výpočtu. Numerické integrování, základní a složené formule, chyba výpočtu.

8.1 Interpolace

Interpolace je metoda nalezení funkce (tzv. interpolační funkce), která **přesně prochází** zadanou množinou diskrétních bodů $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$.

Interpolační polynom

Interpolační polynom je polynom nejvýše n -tého stupně, který prochází zadanými $n + 1$ body.

Charakteristika:

- **Věta o jednoznačnosti:** Pro $n + 1$ vzájemně různých bodů existuje **právě jeden** interpolační polynom stupně nejvýše n .
- Ať použijeme jakoukoliv metodu výpočtu, vždy získáme algebraicky identický polynom (pouze v jiném tvaru zápisu).
- **Rungeho jev:** Při použití vysokého stupně polynomu (mnoho bodů) má polynom tendenci na okrajích intervalu **oscilovat**. Proto se pro velký počet bodů nedoporučuje.

Metody výpočtu:

1. **Vandermondova matice:** Ze zadaných bodů se vytvoří soustava rovnic a ta se pak řeší klasicky pomocí metod jako Gaussova eliminace.
 - *Vandermondova matice:* Matice této soustavy obsahuje mocniny bodů ($V_{i,j} = x_i^j$).
 - *Nevýhoda:* Matice je **velmi špatně podmíněná** (s rostoucím n extrémně rychle roste citlivost na zaokrouhlovací chyby). Výpočet je navíc pomalý ($O(n^3)$).
2. **Lagrangeův interpolační polynom:** Sestavuje se jako lineární kombinace tzv. fundamentálních polynomů $l_i(x)$, které mají vlastnost, že v bodě x_i jsou rovny 1 a ve všech ostatních bodech 0.
 - *Výhoda:* Snadná teoretická konstrukce a matematické důkazy.

- *Nevýhoda:* Při přidání nového bodu se musí celý výpočet dělat odznovu.

3. Newtonův interpolační polynom:

Konstruuje se pomocí tzv. **poměrných diferencí**.

- *Výhoda:* Má rekurzivní charakter. Pokud přidáme nový bod, stačí dopočítat jeden nový člen (jednu diferenci) a zbytek polynomu zůstává.

2. Interpolační splajny (Spline)

Splajny řeší problém Rungeho jevu. Místo abychom jimi prokládali jeden polynom vysokého stupně, proložíme jimi křivku po částech.

- **Definice:** Splajn stupně k je funkce, která je na každém podintervalu $[x_{i-1}, x_i]$ tvořena polynomem stupně k , a v bodech na sebe tyto polynomy plynule navazují (až do $(k - 1)$ -ní derivace).
- **Kubický splajn:** Využívá polynomy 3. stupně.
 - Zaručuje **spojitost funkce, její 1. derivace (tečny) i 2. derivace (křivosti)** ve všech bodech.
 - Výsledkem je na pohled dokonale hladká křivka, která neosciluje, a to i pro tisíce bodů.

8.2 Aproximace funkce metodou nejmenších čtverců

Na rozdíl od interpolace, aproximační funkce neprochází zadanými body přesně, ale snaží se trend dat co nejlépe napodobit. Používá se především pro zpracování nepřesných (zašuměných) experimentálních dat, kde by přesná interpolace vedla k nesmyslným oscilacím. Aproximovat lze různými funkcemi (lineární, kvadratická,...).

Princip metody:

- Cílem je najít zadaný typ funkce $f(x)$ (např. přímku, parabolu), která minimalizuje celkovou odchylku od naměřených bodů (x_i, y_i) .
- Kritérium optimality spočívá v **minimalizaci součtu druhých mocnin** těchto odchylek (reziduí):

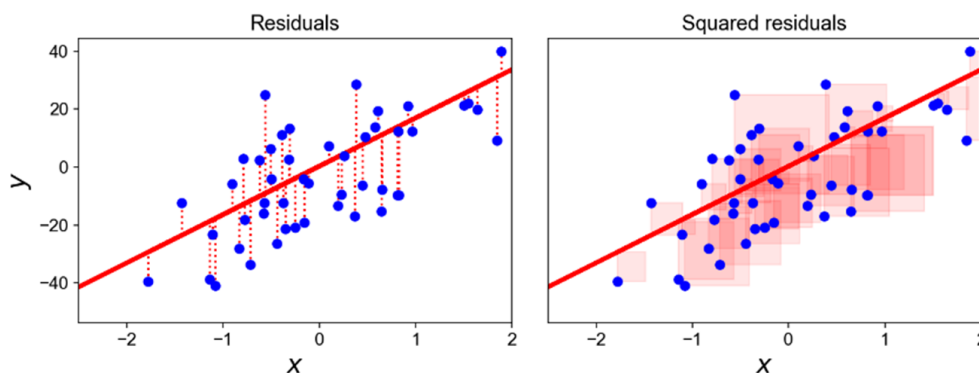
$$S = \sum_{i=1}^n (y_i - f(x_i))^2 \longrightarrow \min$$

- *Proč čtverce?* Druhá mocnina řeší problém kladných a záporných odchylek (aby se navzájem neodečetly), silněji penalizuje velké odchýlení a z matematického hlediska se funkce S velmi snadno derivuje.

Metoda výpočtu:

- Pro nalezení minima funkce S spočítáme její parciální derivace podle hledaných koeficientů funkce $f(x)$ a položíme je rovny nule.
- Tím vznikne soustava lineárních rovnic, tzv. **normální rovnice**. Vyřešením této soustavy získáme jednoznačné optimální koeficienty aproximační funkce.

$$A \times \vec{\beta} \approx \vec{y}$$



Obrázek 9: Metoda nejmenších čtverců

8.3 Numerické derivování

Numerické derivování se používá v případech, kdy známe funkci pouze v diskrétních bodech (např. z měření), nebo je její analytická derivace příliš složitá. Vychází z definice derivace, kde limitu $h \rightarrow 0$ nahradíme konečným malým krokem h .

Základní formule pro 1. derivaci: Formule se odvozují z Taylorova rozvoje funkce:

- **Dopředná diference:** Využívá bod x a následující bod $x + h$.

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} \quad (\text{Chyba metody je } O(h))$$

- **Zpětná diference:** Využívá bod x a předchozí bod $x - h$.

$$f'(x) \approx \frac{f(x) - f(x-h)}{h} \quad (\text{Chyba metody je } O(h))$$

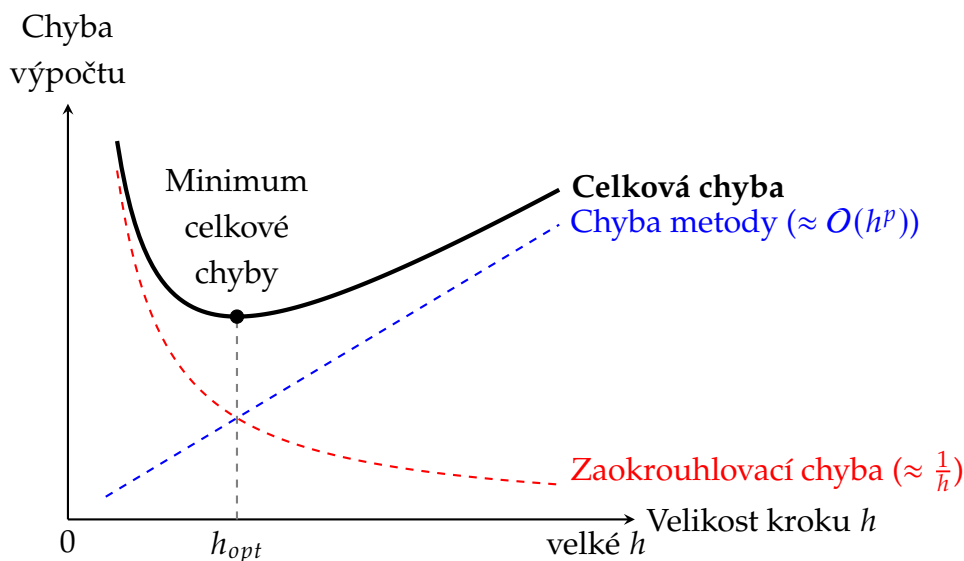
- **Centrální diference:** Využívá body před a za x . Je symetrická a **výrazně přesnější**.

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h} \quad (\text{Chyba metody je } O(h^2))$$

Chyba výpočtu a problém stability: Numerické derivování je klasickým příkladem **špatně podmíněné (nestabilní) úlohy**. Celková chyba se skládá ze dvou protichůdných složek:

1. **Chyba metody (uříznutí):** Vzniká zanedbáním vyšších členů Taylorovy řady. Tato chyba se **zmenšuje**, čím menší je krok h .
2. **Zaokrouhlovací chyba:** Vzniká při odečítání dvou velmi blízkých čísel v čitateli ($f(x+h) - f(x)$) a následném dělení velmi malým číslem h . Tato chyba se naopak **drasticky zvětšuje**, čím menší je krok h (tzv. *cancellation error*).

Praktický důsledek: Na rozdíl od matematické analýzy nemůžeme jít s krokem h libovolně blízko k nule. Při zmenšování h se nejprve přesnost zlepšuje, ale od určitého bodu začnou převládat zaokrouhlovací chyby a výsledek se naprosto znehodnotí. Existuje tedy určité **optimální h** , při kterém je celková chyba minimální.



Obrázek 10: Závislost celkové chyby numerického derivování na velikosti kroku h . Při příliš malém kroku dominuje zaokrouhlovací chyba počítače (červená křivka), při příliš velkém kroku je metoda matematicky nepřesná (modrá křivka). Naším cílem je najít optimální krok h_{opt} .

Zpřesnění výpočtu: Richardsonova extrapolace Vzhledem k nemožnosti libovolně zmenšovat krok h se pro dosažení vysoké přesnosti využívá **Richardsonova extrapolace**.

8.4 Numerické integrování

Numerické integrování se používá k výpočtu určitého integrálu $\int_a^b f(x)dx$, pokud analytický výpočet neexistuje, je příliš složitý, nebo známe-li funkci pouze v diskrétních bodech.

Princip: Základní myšlenka spočívá v nahrazení složité funkce $f(x)$ interpolačním polynomem, který lze snadno zintegrovat analyticky (tzv. Newton-Cotesovy vzorce).

- **Základní formule:** Aplikují aproximaci polynomu rovnou na celý interval $[a, b]$. Jsou velmi nepřesné pro široké intervaly.
- **Složené formule:** Interval $[a, b]$ se rozdělí na n stejných podintervalů o šířce $h = \frac{b-a}{n}$. Základní formule se aplikuje na každý podinterval zvlášť a výsledky se sečtou. Zvyšováním počtu dělení n (zmenšováním h) dosahujeme vysoké přesnosti.

Tři hlavní metody (Složené verze):

1. Obdélníková metoda (Středová)

- *Princip:* Funkci na podintervalu nahradíme konstantou (polynom 0. stupně), konkrétně hodnotou funkce **v jeho středu**. Plocha je aproximována obdélníky.
- *Vzorec:* $I \approx h \sum_{i=0}^{n-1} f(x_i + \frac{h}{2})$
- *Chyba:* $|E_M(f, h)| \leq \frac{b-a}{24} M_2 h^2 (O(h^2))$

2. Lichoběžníková metoda

- *Princip:* Body funkce spojíme úsečkami (polynom 1. stupně). Původní plocha je nahrazena součtem ploch jednotlivých lichoběžníků.
- *Vzorec:* Krajní body se započítají jednou, všechny vnitřní body dvakrát:

$$I \approx h \left(\frac{f(x_0)}{2} + \sum_{i=1}^{n-1} f(x_i) + \frac{f(x_n)}{2} \right)$$

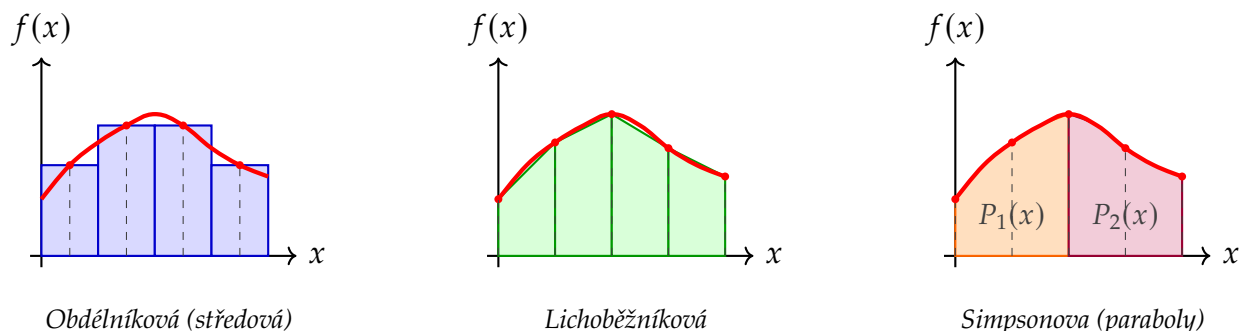
- *Chyba:* $|E_T(f, h)| \leq \frac{b-a}{12} M_2 h^2 (O(h^2))$

3. Simpsonova metoda

- *Princip:* Funkci nahrazujeme po částech **parabolami** (polynom 2. stupně). Každá parabola je proložena přes 3 sousední body, proto metoda **vyžaduje sudý počet podintervalů n** .
- *Vzorec:* Střídají se váhy 4 (pro liché body) a 2 (pro sudé body):

$$I \approx \frac{h}{3} \left(f(x_0) + 2 \sum_{i=1}^{n/2-1} f(x_{2i}) + 4 \sum_{i=1}^{n/2} f(x_{2i-1}) + f(x_n) \right)$$

- *Chyba:* $|E_S(f, h)| \leq \frac{b-a}{180} M_4 h^4 (O(h^4))$



Obrázek 11: Geometrická interpretace tří základních složených kvadraturních vzorců pro krok $h = 1$. Červená křivka reprezentuje přesnou funkci $f(x)$. Obdélníková metoda využívá funkční hodnotu uprostřed podintervalu. Lichoběžníková metoda spojuje hranice úsečkou. Simpsonova metoda prokládá vždy trojici bodů (dvěma podintervaly) parabolou (P_1, P_2).

Zlepšení výsledků:

- Rekurzivní výpočet pro dvojnásobný počet bodů
- Rombergova metoda

Chyba výpočtu a stabilita: Na rozdíl od numerického derivování je numerické integrování **vysoce stabilní proces**. Integrál je ze své podstaty suma (součet ploch), takže náhodné zao-krouhlovací chyby (nebo šum v měřených datech) se často navzájem vyruší (vyhladí).

Protože analytický výpočet chyby závisí na derivacích integrované funkce (které většinou neznáme), používá se v praxi pro odhad chyby tzv. **Rungeho princip polovičního kroku**:

1. Spočítáme integrál s krokem h (označme I_h).
2. Spočítáme integrál s polovičním krokem $h/2$ (označme $I_{h/2}$).
3. Pokud je rozdíl $|I_h - I_{h/2}|$ menší než naše požadovaná přesnost ε , výpočet ukončíme a prohlásíme výsledek za dostatečně přesný. Pokud ne, krok znovu rozpůlíme.

9 Numerické řešení soustav lineárních algebraických rovnic - přímé a nepřímé metody.

Lineární algebraická rovnice a jejich soustavy, matice soustavy, Gaussova eliminační metoda, výběr hlavního prvku, vliv zaokrouhlovacích chyb, podmíněnost úlohy, LU rozklad matice, výpočet inverzní matice, výpočet determinantu. Iterační metody a kritéria jejich konvergence.

9.1 Lineární algebraická rovnice a jejich soustavy

Rovnice, ve které se neznámé vyskytují pouze v 1. mocnině a nejsou mezi sebou násobeny. Soustava lineárních rovnic obsahuje více rovnic, které sdílí stejné neznámé.

Základní pojmy a maticový zápis: Soustava m lineárních algebraických rovnic o n neznámých $x_1, x_2, \dots, x_n$ má obecný tvar:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n &= b_2 \\ &\vdots \\ a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n &= b_m \end{aligned}$$

Tento rozsáhlý zápis se v lineární algebře zkracuje do **maticového tvaru**:

$$A\vec{x} = \vec{b}$$

kde definujeme následující prvky:

- $A \in \mathbb{R}^{m \times n}$ je **matice soustavy** tvořená koeficienty a_{ij} .
- $\vec{x} \in \mathbb{R}^n$ je **vektor neznámých** (sloupcový vektor).
- $\vec{b} \in \mathbb{R}^m$ je **vektor pravých stran** (absolutní členy).

Spojením matice A a vektoru $\vec{b}$ vzniká tzv. **rozšířená matice soustavy** $[A|\vec{b}]$, se kterou pracují základní vyhodnocovací a řešící algoritmy.

Řešitelnost soustavy (Frobeniova věta): Z čistě algebraického hlediska rozhoduje o existenci řešení **Frobeniova věta**. Zkoumáme hodnotu matice $h(A)$ (počet jejích lineárně nezávislých řádků):

1. **Nemá řešení:** Pokud se hodnost matice soustavy liší od hodnosti rozšířené matice ($h(A) \neq h(A|\vec{b})$). Soustava obsahuje spor.
2. **Má právě 1 řešení:** Pokud se hodnosti rovnají a jsou rovny počtu neznámých ($h(A) = h(A|\vec{b}) = n$). Matice A je v tomto případě **regulární** (je čtvercová a její determinant je nenulový, $\det(A) \neq 0$).
3. **Má nekonečně mnoho řešení:** Pokud se hodnosti rovnají, ale jsou menší než počet neznámých ($h(A) = h(A|\vec{b}) < n$). Řešení pak závisí na $(n - h)$ volných parametrech.

Poznámka k hodnosti matice ($h(A)$): Hodnost matice udává maximální počet jejích **lineárně nezávislých** řádků (nebo sloupců). Pokud má čtvercová matice o rozměrech $n \times n$ hodnost přesně n , říkáme, že má **plnou hodnost**. Taková matice je regulární a má nenulový determinant.

9.2 Gaussova eliminační metoda

Gaussova eliminační metoda (GEM) patří mezi přímé metody řešení soustav lineárních algebraických rovnic. Cílem je nalézt vektor řešení $\vec{x} \in \mathbb{R}^n$ splňující rovnici

$$A\vec{x} = \vec{b}$$

kde $A \in \mathbb{R}^{n \times n}$ je regulární čtvercová matice soustavy a $\vec{b} \in \mathbb{R}^n$ je vektor pravé strany.

Algoritmus Gaussovy eliminační metody

Algoritmus GEM se skládá ze dvou základních, po sobě jdoucích fází:

1. **Přímý chod (eliminance):** Rozšířená matice soustavy $[A|\vec{b}]$ je pomocí elementárních řádkových úprav (záměna řádků, přičtení lineární kombinace jiných řádků) postupně převedena na horní trojúhelníkový tvar $[U|\vec{c}]$, kde U je horní trojúhelníková matice s nulovými prvky pod hlavní diagonálou.
2. **Zpětný chod (substituce):** Získaná soustava s horní trojúhelníkovou maticí $U\vec{x} = \vec{c}$ se řeší postupně od poslední neznámé po první zpětnou substitucí.

Výběr hlavního prvku (Pivoting)

V počítačové aritmetice s omezenou přesností nastává problém tehdy, je-li právě řešený prvek na diagonále ≈ 0 . Dělení velmi malým číslem vede ke vzniku extrémně velkých multiplikátorů

m_{ik} , což způsobuje šíření a zvětšování zaokrouhlovacích chyb v následujících úpravách matice. Tento jev se eliminuje *výběrem hlavního prvku* (pivotingem):

- **Částečný (sloupcový) výběr prvku:** V k -tém kroku hledáme v k -tém sloupci na diagonále a pod ní prvek s maximální absolutní hodnotou, tedy index r takový, že:

$$|a_{rk}^{(k-1)}| = \max_{k \leq i \leq n} |a_{ik}^{(k-1)}|.$$

Následně se prohodí celý k -tý řádek s řádkem r . Tím je zaručeno, že pro všechny multiplikátory platí $|m_{ik}| \leq 1$, což stabilizuje výpočet. Tedy vybereme z aktuálního sloupce největší číslo (podle absolutní hodnoty) a vyměníme řádky, aby to číslo bylo nahoře = hlavní prvek.

- **Úplný výběr prvku:** Maximum se hledá v celé zbývající podmatici o rozměrech $(n - k + 1) \times (n - k + 1)$. Tento postup vyžaduje jak prohození řádků, tak prohození sloupců (což znamená přeuspořádání složek vektoru neznámých $\vec{x}$). Úplný výběr je numericky nejstabilnější, ale kvůli vysoké vyhledávací náročnosti se v praxi využívá zřídka. Částečný výběr obvykle poskytuje dostatečnou přesnost.

Vliv zaokrouhlovacích chyb

Každá aritmetická operace zavádí drobnou relativní chybu. U rozsáhlých soustav rovnic dochází ke kumulaci těchto chyb. Hlavním rizikem v GEM je tzv. *odčítání blízkých čísel* (cancellation) a následné násobení velkými čísly. Pokud není implementován výběr hlavního prvku, mohou chyby zcela znehodnotit výsledek, což znamená, že algoritmus je *numericky nestabilní*. Použitím částečného pivotingu se GEM stává zpětně stabilním algoritmem.

Podmíněnost úlohy

Mírou této podmíněnosti GEM je **číslo podmíněnosti matice** $\kappa(A)$, definované vzhledem k přidružené maticové normě. Číslo podmíněnosti se vypočítá jako součin normy matice a normy její inverzní matice:

$$\kappa(A) = \|A\| \cdot \|A^{-1}\|.$$

Pro libovolnou matici platí $\kappa(A) \geq 1$. Podle velikosti tohoto čísla rozlišujeme úlohy na:

- **Dobře podmíněné** ($\kappa(A) \approx 1$): Malé relativní změny na vstupu vyvolají adekvátně malé relativní změny v řešení.
- **Špatně podmíněné** ($\kappa(A) \gg 1$): I nepatrná chyba ve vstupních datech (např. zaokrouhlení při zadávání koeficientů) může způsobit obrovské, řádové změny ve výsledném vektoru $\vec{x}$. Matice s extrémně vysokým číslem podmíněnosti se blíží maticím singulárním.

Je-li matice soustavy špatně podmíněná, žádný numericky stabilní algoritmus (včetně GEM s úplným výběrem prvků) nedokáže zajistit přesné řešení, neboť velká chyba výstupu je přirozeným důsledkem vlastností zadání úlohy.

9.3 LU rozklad matice

LU rozklad je přímá metoda řešení soustavy $A\vec{x} = \vec{b}$, která spočívá v rozkladu regulární čtvercové matice A na součin dvou trojúhelníkových matic:

$$A = LU,$$

kde L je dolní (*Lower*) trojúhelníková matice s jednotkami na hlavní diagonále a U je horní (*Upper*) trojúhelníková matice, která odpovídá výsledku matice A po přímém chodu Gaussovy eliminace.

Postup řešení

Jakmile je znám rozklad $A = LU$, původní soustava $LU\vec{x} = \vec{b}$ se řeší ve dvou krocích:

1. **Dopředná substitute:** Řešíme soustavu $L\vec{y} = \vec{b}$ pro pomocný vektor $\vec{y}$.
2. **Zpětná substitute:** Řešíme soustavu $U\vec{x} = \vec{y}$ pro cílový vektor řešení $\vec{x}$.

Výhody a složitost

- **Efektivita:** LU rozklad je výhodný zejména v případech, kdy řešíme více soustav se stejnou maticí A , ale různými pravými stranami $\vec{b}$. Rozklad se provede pouze jednou a pro každou novou pravou stranu se provádí pouze substitute.
- **Vztah k determinantu:** Determinant matice lze po rozkladu snadno spočítat jako součin diagonálních prvků matice U , tedy $\det(A) = \prod_{i=1}^n u_{ii}$.

9.4 Výpočet inverzní matice

Inverzní matice A^{-1} k regulární čtvercové matici A řádu n je definována vztahem

$$AA^{-1} = I,$$

kde I je jednotková matice řádu n . Inverzní matice existuje, jen pokud $\det(A) \neq 0$. Pro praktický výpočet se využívají dva hlavní přístupy:

- **Využití LU rozkladu:** Matice A se pouze jednou rozloží na $PA = LU$. Následně se pro každý z n sloupců $\vec{e}_i$ provede rychlá dopředná a zpětná substituce.
- **Gaussova-Jordanova eliminace:** Rozšířená matice tvaru $[A | I]$ se pomocí ekvivalentních řádkových úprav převede přímo na redukováný schodovitý tvar. Z levé poloviny se stane jednotková matice a v pravé polovině vznikne hledaná matice inverzní: $[A | I] \sim [I | A^{-1}]$.

9.5 Výpočet determinantu

Výpočet determinantu $\det(A)$ podle definice nebo pomocí Laplaceova rozvoje je výpočetně neúnosný pro matice větších řádů, neboť vyžaduje asymptoticky $O(n!)$ operací. V numerické matematice se proto determinant počítá výhradně pomocí Gaussovy eliminace nebo LU rozkladu, což redukuje složitost na $O(n^3)$.

Z vlastností determinantu plyne, že determinant horní i dolní trojúhelníkové matice je roven součinu prvků na její hlavní diagonále. Máme-li k dispozici LU rozklad s částečným výběrem prvku $PA = LU$, platí podle Cauchyovy věty o determinantu součinu:

$$\det(P) \cdot \det(A) = \det(L) \cdot \det(U).$$

Z vlastností jednotlivých matic rozkladu odvodíme:

- Matice L má na diagonále pouze jedničky, tedy $\det(L) = 1$.
- Matice U je horní trojúhelníková, její determinant je součin diagonálních prvků u_{ii} .
- Permutační matice P vznikla výměnou řádků. Každá výměna řádků mění znaménko determinantu. Proto platí $\det(P) = (-1)^s$, kde s je celkový počet výměn řádků provedených během eliminace.

Výsledný vzorec pro efektivní výpočet determinantu je tedy dán jako:

$$\det(A) = (-1)^s \prod_{i=1}^n u_{ii} = (-1)^s \cdot u_{11} \cdot u_{22} \cdots u_{nn}.$$

Pokud matice A není regulární, na diagonále matice U se objeví alespoň jedna nula a součin automaticky vyjde nulový, což správně odpovídá singulární matici.

Sarrusovo pravidlo Zatímco pro velké matice se využívá výhradně Gaussova eliminace nebo LU rozklad, pro ruční výpočet determinantu malých matic řádu $n = 3$ se často používá přímý vzorec známý jako **Sarrusovo pravidlo**.

Pro matici $A \in \mathbb{R}^{3 \times 3}$ ve tvaru:

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix}$$

je Sarrusovo pravidlo mnemotechnickou pomůckou. Vypočte se jako součet součinů tří prvků ve směru hlavní diagonály (zleva nahoře doprava dolů), od kterého se odečte součet součinů tří prvků ve směru vedlejší diagonály (zprava nahoře doleva dolů):

$$\det(A) = a_{11}a_{22}a_{33} + a_{12}a_{23}a_{31} + a_{13}a_{21}a_{32} - (a_{13}a_{22}a_{31} + a_{11}a_{23}a_{32} + a_{12}a_{21}a_{33}).$$

9.6 Iterační metody a kritéria jejich konvergence

Na rozdíl od přímých metod (jako GEM či LU rozklad), které naleznou přesné řešení v konečném počtu kroků, generují iterační metody posloupnost aproximací $\vec{x}^{(0)}, \vec{x}^{(1)}, \dots, \vec{x}^{(k)}$, která konverguje k přesnému řešení $\vec{x}$. Tyto metody se využívají především pro řešení obrovských a řídkých soustav, u kterých by přímé metody byly příliš paměťově a časově náročné.

Obecný tvar lineární iterační metody pro řešení soustavy $A\vec{x} = \vec{b}$ je:

$$\vec{x}^{(k+1)} = T\vec{x}^{(k)} + \vec{c},$$

kde T je tzv. iterační matice a $\vec{c}$ je konstantní vektor.

Kritéria konvergence

Jakobiho i Gauss-Seidelova metoda konvergují, pokud matice soustavy A je:

- **Ryze řádkově (či sloupcově) diagonálně dominantní:** Absolutní hodnota prvku na diagonále je větší než součet absolutních hodnot ostatních prvků v daném řádku (resp. sloupci).
- Gauss-Seidelova metoda konverguje navíc i tehdy, pokud je matice A **symetrická a pozitivně definitní**.

Pozitivní definitnost: Symetrická matice A řádu n je podle Sylvesterova kritéria pozitivně definitní právě tehdy, když jsou všechny její vedoucí hlavní minory (determinanty levých horních podmatic) ostře kladné. Základní vlastností pozitivně definitní matice, že všechna její vlastní čísla jsou kladná.

Jakobiho metoda

Matice soustavy se rozloží na tvar $A = D + L + U$, kde D je diagonální matice a L, U jsou ostře dolní a horní trojúhelníkové matice (mají na diagonále nuly). Iterační předpis je odvozen ze vztahu $D\vec{x} = \vec{b} - (L + U)\vec{x}$ a zní:

$$\vec{x}^{(k+1)} = D^{-1}(\vec{b} - (L + U)\vec{x}^{(k)}).$$

Po složkách to znamená, že nová aproximace i -té složky se počítá **výhradně z hodnot předchozího kroku** k . Matice D musí mít nenulové diagonální prvky.

Gauss-Seidelova metoda

Jedná se o vylepšení Jakobiho metody. Při výpočtu i -té složky vektoru $\vec{x}^{(k+1)}$ se okamžitě využijí již vypočtené novější hodnoty složek s indexy 1 až $i - 1$. Iterační předpis v maticovém tvaru je:

$$\vec{x}^{(k+1)} = (D + L)^{-1}(\vec{b} - U\vec{x}^{(k)}).$$

Díky průběžnému využívání zpřesněných hodnot konverguje Gauss-Seidelova metoda obvykle rychleji než Jakobiho metoda.

Ritzova iterační metoda

Tato metoda se principiálně liší od předchozích a vychází z **variačního počtu**. Je určena pro soustavy, kde matice A je symetrická a pozitivně definitní.

10 Popis datového souboru

Typy dat a měrné stupnice. Charakteristika polohy (střední hodnota, modus, medián), variability (směrodatná odchylka, rozptyl), relativní četnosti. Rozdělení hodnot (kvantily, kumulativní relativní četnosti). Vizualizace (histogram, sloupkový, kruhový a krabicový graf). Specifika popisu dat nenumernických.

10.1 Typy dat a měrné stupnice

Typy dat (Základní rozdělení): Data obecně dělíme do dvou hlavních skupin:

- **Kvalitativní (Kategoriální) data:** Popisují vlastnosti, stavy nebo kategorie. I když jsou někdy kódována čísla (např. 1 = žena, 2 = muž), nemají matematický význam (nelze je sčítat, porovnávat, atd.).

- **Kvantitativní (Numerická) data:** Vyjadřují množství nebo velikost a lze s nimi provádět matematické operace. Dělí se dále na:
 - *Diskrétní:* Nabývají pouze oddělených, většinou celočíselných hodnot (např. počet dětí v rodině, počet defektů na výrobku). Vznikají počítáním.
 - *Spojité:* Mohou nabývat libovolné reálné hodnoty v určitém intervalu (např. výška, hmotnost, čas). Vznikají měřením.

Měrné stupnice: Podle toho, jaké matematické operace s hodnotami dávají logický smysl, rozdělujeme data do čtyř měrných stupnic (od nejchudší po nejbohatší):

1. Nominální stupnice:

- *Charakteristika:* Pouze pojmenovává kategorie. Neexistuje mezi nimi žádné logické pořadí.
- *Operace:* Lze testovat pouze rovnost ($=$ a $\neq$).
- *Příklady:* Barva očí, krevní skupina, rodné číslo, PSČ.

2. Ordinální stupnice:

- *Charakteristika:* Kategorie lze logicky seřadit (víme, co je "víc" a "míň"), ale neznáme přesnou vzdálenost mezi nimi.
- *Operace:* Lze navíc porovnávat velikost ($<$, $>$).
- *Příklady:* Stupeň vzdělání (ZŠ, SŠ, VŠ), vojenské hodnosti, hodnocení spokojenosti (1 až 5 hvězdiček).

3. Intervalová stupnice:

- *Charakteristika:* Hodnoty lze seřadit a víme, o kolik se liší (intervaly jsou stejně velké). Chybí zde však absolutní nula (nula je jen dohodnutý bod, neznamená "nic").
- *Operace:* Lze sčítat a odčítat ($+$, $-$). Nelze dělit (nemá smysl počítat násobky).
- *Příklady:* Teplota ve stupních Celsia (20°C není dvakrát teplejší než 10°C), kalendářní roky.

4. Poměrová stupnice:

- *Charakteristika:* Má všechny vlastnosti intervalové stupnice a navíc má přirozenou (absolutní) nulu, která znamená úplnou absenci měřené vlastnosti.
- *Operace:* Lze provádět všechny matematické operace včetně násobení a dělení ($\cdot$, $/$).
- *Příklady:* Věk, mzda, hmotnost, délka (10 metrů je přesně dvakrát delší než 5 metrů).

10.2 Charakteristiky polohy

Ukazují, kolem jaké hodnoty se data koncentrují (tzv. míry střední tendence).

- **Střední hodnota (Aritmetický průměr, $\bar{x}$ nebo μ (mý)):** Součet všech hodnot vydělený jejich počtem.
 - *Výhoda:* Využívá všechny hodnoty v souboru.
 - *Nevýhoda:* Je extrémně citlivý na odlehlé hodnoty. Jeden miliardář v místnosti drasticky zkreslí průměrný plat všech přítomných.
- **Medián ($\tilde{x}$):** Prostřední hodnota v seřazeném souboru dat (50 % hodnot je menších, 50 % větších).
 - *Výhoda:* Je robustní vůči odlehlým hodnotám. Používá se typicky pro platy nebo ceny nemovitostí, kde průměr dává zkreslený obraz.
- **Modus ($\hat{x}$):** Nejčastěji se vyskytující hodnota v souboru.
 - *Výhoda:* Jako jediná míra polohy se dá použít i pro nominální data (např. nejčastější barva očí, nejprodávanější značka auta).

10.3 Charakteristiky variability

Určuje odchylky hodnot v datovém souboru (průměrná teplota v poušti může být příjemných 20°C, ale přes den je 50°C a v noci -10°C).

- **Rozptyl (s^2 pro výběr, σ^2 (sigma) pro populaci):** Průměr ze čtverců odchylek jednotlivých hodnot od aritmetického průměru.
 - *Populační rozptyl* (počítáme z úplně všech prvků, N je velikost populace, μ je populační průměr):

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2$$

- *Výběrový rozptyl* (počítáme jen ze vzorku, n je velikost vzorku, $\bar{x}$ je výběrový průměr). Dělí se $n - 1$ (tzv. Besselova korekce), aby byl odhad nezkreslený:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

kde $(x_i - \bar{x})^2$ je čtverec vzdáleností bodů od průměru.

- **Směrodatná odchylka (s nebo σ):** Druhá odmocnina z rozptylu.
 - *Populační:* $\sigma = \sqrt{\sigma^2}$
 - *Výběrová:* $s = \sqrt{s^2}$
 - *Výhoda:* Vrací variabilitu zpět do původních jednotek (např. metry, koruny). Udává, o kolik se hodnoty typicky odchylují od průměru.

10.4 Absolutní a relativní četnost

Používají se především pro kvalitativní a diskrétní data k popisu jejich rozložení.

- **Absolutní četnost (n_i):** Udává, *kolikrát* se daná varianta vyskytla v souboru (např. 15 studentů dostalo jedničku). Součet absolutních četností je roven celkovému rozsahu souboru n .
- **Relativní četnost (p_i):** Poměr absolutní četnosti k celkovému počtu měření ($p_i = \frac{n_i}{n}$). Udává se jako desetinné číslo (od 0 do 1) nebo v procentech (např. 30 % studentů).
 - *Využití:* Je nezbytná pro porovnávání různě velkých souborů (absolutní počty nelze srovnávat, pokud má jeden vzorek 100 lidí a druhý 1000 lidí).

10.5 Rozdělení hodnot

Kvantily Kvantil je hodnota, která rozděluje seřazený datový soubor na části v určitém procentuálním poměru.

- **Percentily (P_k):** Rozdělují data na 100 dílů (po 1 %). Například P_{90} (90. percentil) znamená, že 90 % hodnot je menších (nebo rovných) a 10 % hodnot je větších než tento bod.
- **Kvartily (Q):** Rozdělují data na 4 čtvrtiny (po 25 %). Jsou nejpoužívanější:
 - Q_1 (**Dolní kvartil**): 25 % hodnot je pod ním.
 - Q_2 (**Medián**): 50 % hodnot je pod ním (střed).
 - Q_3 (**Horní kvartil**): 75 % hodnot je pod ním.

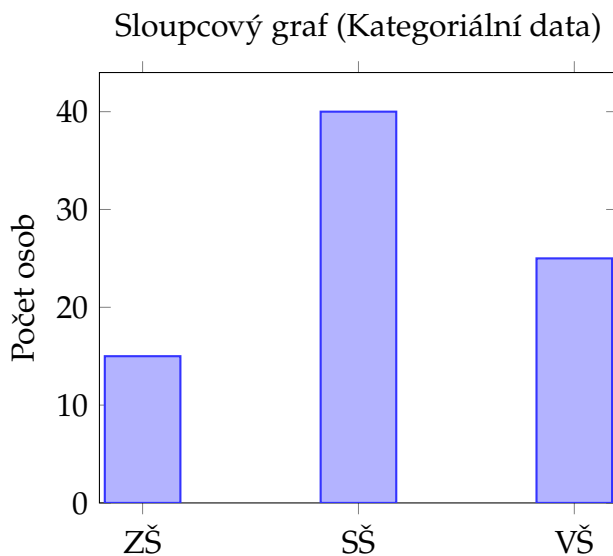
Kumulativní relativní četnosti Zatímco obyčejná relativní četnost říká, jaká část dat spadá do jedné konkrétní kategorie, kumulativní četnost hodnoty postupně sčítá (kumuluje).

- **Význam:** Odpovídá na otázku: „Kolik procent dat je menších nebo rovných dané hodnotě?“
- **Vlastnosti:** Začíná na 0 a s každou další hodnotou roste, až u maximální hodnoty v souboru dosáhne přesně 1 (neboli 100 %).

10.6 Vizualizace dat

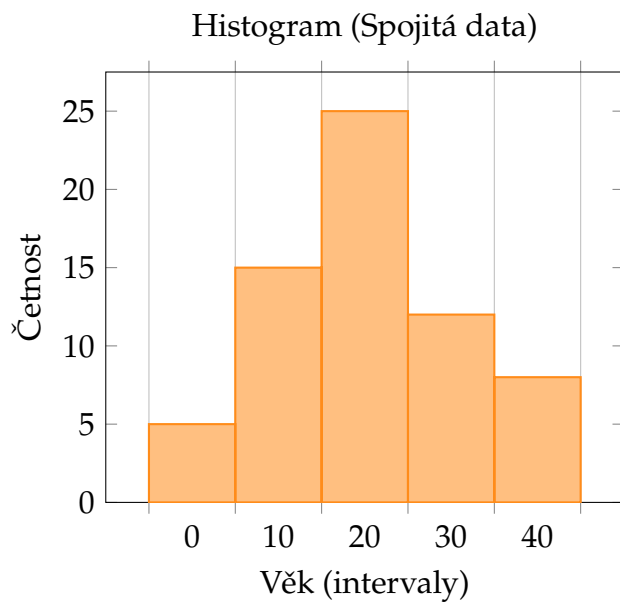
1. Sloupcový graf (Bar chart)

- **Použití:** Pro kvalitativní (kategoriální) nebo diskrétní data (např. barva očí, vzdělání).
- **Vzhled:** Mezi sloupci jsou **viditelné mezery**, protože kategorie spolu číselně nesouvisí.



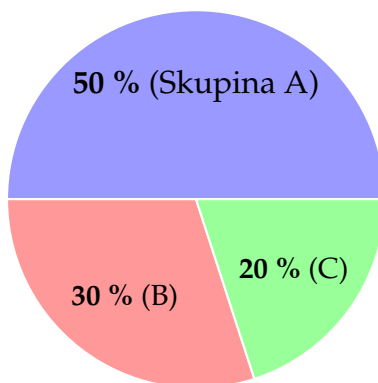
2. Histogram

- **Použití:** Pro spojitá kvantitativní data (např. výška, hmotnost, čas).
- **Vzhled:** Osa x představuje číselnou osu rozdělenou do intervalů. Sloupce na sebe těsně navazují (bez mezer), což znázorňuje spojitost dat.



3. Kruhový graf (Pie chart)

- **Použití:** Pro zobrazení **relativních četností (podílů z celku)**.
- **Pravidlo:** Vhodný pouze pro malý počet kategorií (ideálně do 5). Všechny výseče musí dát dohromady 100

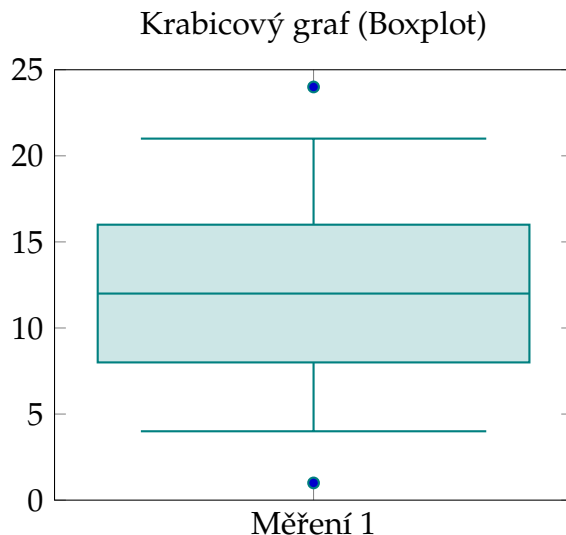


Obrázek 12: Kruhový graf (Podíly z celku)

4. Krabicový graf (Boxplot)

- **Použití:** Ideální pro vizualizaci **kvartilů** a detekci **odlehklých hodnot**. Skvěle se hodí pro porovnávání více skupin dat vedle sebe.

- **Vzhled:** Krabice vymezuje prostředních 50% dat (od Q_1 do Q_3), čára uvnitř je medián. Hmatníky (ty věci nad a pod krabicí) sahají k minimu a maximum. Tečky mimo vousy jsou odlehlé hodnoty. (Občas je znažena i průměra hodnota.)



10.7 Specifika popisu dat nenumерických

Nenumерická (kvalitativní či kategoriální) data vyžadují specifický přístup, protože jejich hodnoty reprezentují vlastnosti, názvy nebo stavy, u kterých nelze provádět běžné aritmetické operace (nelze je sčítat, dělit, atd.).

Omezení klasických popisných statistik:

- **Míry polohy:** Aritmetický průměr nelze použít vůbec. U nominálních dat lze určit pouze **modus** (nejčetnější kategorii). U ordinálních dat (kategorie lze logicky seřadit) má smysl hledat navíc i **medián** a **kvartily**.
- **Míry variability:** Klasický rozptyl či směrodatná odchylka nemají smysl. Variabilita se popisuje počtem unikátních kategorií a tím, jak rovnoměrně jsou mezi ně data rozdělena (zda převažuje jedna kategorie, nebo je datový soubor velmi heterogenní).

Hlavní nástroje pro popis:

- **Tabulky četností:** Základem je vždy spočítání absolutních a relativních (procentuálních) četností jednotlivých kategorií.

- **Kontingenční tabulky:** Slouží ke zkoumání závislosti mezi dvěma kategoriálními proměnnými (např. vztah mezi stupněm vzdělání a místem bydliště). Ukazují četnosti průniků jednotlivých kategorií.

11 Princip inference

Populace, vzorek (výběr), bodový odhad parametru, výběrová chyba, zobecnění výsledku. Pro střední hodnotu a relativní četnost: směrodatná chyba a konstrukce intervalového odhadu. Statistické hypotézy o střední hodnotě a podílu (hypotézy, rozhodovací pravidla, chyba I. a II. druhu).

11.1 Populace a vzorek

- **Populace:** Kompletní množina všech objektů, osob nebo jevů, o kterých chceme zjistit nějakou informaci (např. všichni oprávnění voliči v ČR, všechny vyrobené žárovky v továrně).
 - Číselné charakteristiky celé populace nazýváme **parametry** (značíme řecky: μ (mý) pro průměr, σ^2 (sigma) pro rozptyl). V praxi jsou většinou neznámé.
- **Vzorek / Výběr:** Část populace, kterou reálně změříme.
 - Číselné charakteristiky spočítané ze vzorku nazýváme **statistiky** (značíme latinkou: $\bar{x}$ pro průměr, s^2 pro rozptyl).
 - **Reprezentativnost:** Aby mělo zkoumání vzorku smysl, musí vzorek věrně odrážet složení populace. Toho se nejlépe dosahuje prostým **náhodným výběrem**.

11.2 Bodový odhad parametru

Bodový odhad znamená, že neznámý parametr populace (např. skutečný průměrný plat v ČR) je nahrazen jedním konkrétním číslem spočítaným ze vzorku (např. průměrný plat tisíce dotázaných lidí, tedy odhadem μ je $\bar{x}$).

Správný bodový odhad by měl splňovat dvě klíčové matematické vlastnosti:

- **Nestrannost (Nevychýlenost):** Odhad systematicky nenadceňuje ani nepodceňuje skutečnost (očekávaná hodnota odhadu se rovná skutečnému parametru). *Proto u výběrového rozptylu dělíme $n - 1$, aby byl nestranný.*
- **Konzistence:** S rostoucí velikostí vzorku ($n \rightarrow \infty$) se hodnota odhadu přesněji a spolehlivěji blíží ke skutečnému parametru populace.

- **Efektivnost:** Mezi všemi nestrannými odhady má nejmenší rozptyl (varianci).

11.3 Výběrová chyba

Protože se nenaměřila celá populace, bodový odhad se bude od skutečnosti vždy o něco lišit.

- Jedná se o přirozený, nevyhnutelný, ale matematicky spočítatelný důsledek toho, že pracujeme pouze se vzorkem.
- **Směrodatná chyba (Standard Error, SE):** Vyjadřuje, jak velkou výběrovou chybu můžeme v průměru očekávat. Pro výběrový průměr se počítá jako:

$$SE = \frac{s}{\sqrt{n}}$$

kde s je směrodatná odchylka vzorku a n je počet měření.

- Čím větší je vzorek (n), tím menší je výběrová chyba a odhad je přesnější.

11.4 Zobecnění výsledku (Inference)

Konečným cílem je zobecnit výsledky na celou populaci. Protože bodový odhad sám o sobě nestačí (je zatížen výběrovou chybou), používají se ke zobecnění dva robustní nástroje:

1. **Intervalové odhady (Intervaly spolehlivosti):** Místo jednoho konkrétního bodu sestojíme kolem bodového odhadu interval. Následně můžeme prohlásit s určitou pravděpodobností, že skutečný parametr populace leží právě uvnitř tohoto intervalu.
Interval spolehlivosti pokrývá skutečnou hodnotu populačního parametru s předem zvolenou pravděpodobností (tzv. hladinou spolehlivosti $1 - \alpha$, nejčastěji 95%).
2. **Testování hypotéz:** Postup, při kterém data ze vzorku použijeme k potvrzení nebo vyvrácení teorie o celé populaci. Protože jsme nezkoumali všechny, vždy existuje riziko, že náš závěr bude chybný (tyto chyby označujeme jako [chybu 1. a 2. druhu](#)).

11.5 Směrodatná chyba a konstrukce intervalového odhadu

1. Pro střední hodnotu (Populační průměr μ) Používá se, když analyzujeme kvantitativní data (např. průměrný plat, výška, věk).

- **Bodový odhad:** Výběrový průměr $\bar{x}$.

- **Směrodatná chyba (SE):** Udává přesnost našeho průměru. („O kolik cm se může výběrový průměr lišit od skutečné výšky v populaci“)

$$SE = \frac{s}{\sqrt{n}}$$

(kde s je směrodatná odchylka vzorku a n je velikost vzorku).

- **Kritická hodnota:** Kvantil Studentova t -rozdělení $t_{1-\alpha/2, n-1}$ (pro 95% spolehlivost a velký vzorek je tato hodnota zhruba rovna 2). Používáme t -rozdělení, protože skutečnou směrodatnou odchylku populace neznáme a odhadujeme ji pomocí s .
- **Výsledný interval spolehlivosti:**

$$\mu \in \left(\bar{x} - t_{1-\alpha/2, n-1} \cdot \frac{s}{\sqrt{n}}, \bar{x} + t_{1-\alpha/2, n-1} \cdot \frac{s}{\sqrt{n}} \right)$$

2. Pro relativní četnost (Populační podíl π) Používá se, když analyzujeme **kvalitativní data** (např. procento lidí, kteří budou volit určitou stranu, procento zmetků ve výrobě).

- **Bodový odhad:** Výběrový podíl $p = \frac{x}{n}$ (počet sledovaných jevů dělený velikostí vzorku).
- **Směrodatná chyba (SE):** Vypočítá se přímo ze samotného podílu (čím blíže je podíl k 50 % (0,5), tím je chyba logicky největší, protože situace je nejméně jednoznačná).

$$SE = \sqrt{\frac{p(1-p)}{n}}$$

- **Kritická hodnota:** Kvantil standardizovaného normálního rozdělení $z_{1-\alpha/2}$ (pro 95% spolehlivost je tato hodnota přesně 1,96). Používá se díky Centrální limitní větě (platí pro dostatečně velké vzorky).
- **Výsledný interval spolehlivosti:**

$$\pi \in \left(p - z_{1-\alpha/2} \cdot \sqrt{\frac{p(1-p)}{n}}, p + z_{1-\alpha/2} \cdot \sqrt{\frac{p(1-p)}{n}} \right)$$

Centrální limitní věta: Bez ohledu na původní rozdělení dat, pokud je vzorek dostatečně velký (často se používá pravidlo $n \geq 30$), bude rozdělení výběrového průměru přibližně normální.

11.6 Statistické hypotézy o střední hodnotě a podílu

Testování hypotéz je standardizovaný postup, kterým na základě dat ze vzorku rozhodujeme o platnosti tvrzení o celé populaci.

1. Formulace hypotéz Testování vždy probíhá proti sobě stojící dvojici hypotéz:

- **Nulová hypotéza (H_0):** Předpokládá "status quo", žádný rozdíl, nebo neexistenci závislosti. Obsahuje vždy znak rovnosti ($=, \leq, \geq$).
- **Alternativní hypotéza (H_1):** Vyjadřuje změnu, rozdíl nebo závislost. Obsahuje znaky nerovnosti ($\neq, <, >$).

Konkrétní příklady pro střední hodnotu a podíl:

- *O střední hodnotě (Populační průměr μ):* Testujeme, zda je průměrný plat v ČR roven 40 000 Kč. ($H_0 : \mu = 40\,000$ proti $H_A : \mu \neq 40\,000$).
- *O podílu (Populační proporce π):* Testujeme, zda volební podíl strany překročil 5 %. ($H_0 : \pi \leq 0,05$ proti $H_A : \pi > 0,05$).

2. Chyba I. a II. druhu Protože rozhodujeme jen na základě vzorku, můžeme se splést. Existují dva druhy chyb:

1. Chyba I. druhu (Falešně pozitivní výsledek):

- Zamítneme nulovou hypotézu, i když je ve skutečnosti pravdivá (např. odsoudíme nevinného, nebo lék označíme za účinný, ačkoliv nefunguje).
- Pravděpodobnost této chyby se označuje α (tzv. **hladina významnosti**) a určuje ji předem výzkumník (nejčastěji $\alpha = 0,05$, tedy 5 %).

2. Chyba II. druhu (Falešně negativní výsledek):

- Nezamítneme nulovou hypotézu, i když je ve skutečnosti nepravdivá (např. propustíme viníka, nebo nepoznáme, že lék opravdu funguje).
- Pravděpodobnost této chyby se označuje β . Doplněk do 100 % ($1 - \beta$) se nazývá **síla testu** (schopnost testu odhalit skutečný rozdíl, pokud existuje).

Zpět na [zobecnění výsledku](#).

3. Testové statistiky Vzorec, kterým převedeme naše data na jedno číslo (testové kritérium):

- **Pro střední hodnotu (t-test):** $t = \frac{\bar{x} - \mu_0}{SE} = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}$

Pokud známe směrodatnou odchylku populace (σ), použijeme normální rozdělení a vzorec (z-test) $z = \frac{\bar{x} - \mu_0}{\sigma/\sqrt{n}}$. V praxi však většinou σ neznáme, proto používáme s a t-rozdělení.

- **Pro podíl (z-test):** $z = \frac{p - \pi_0}{SE} = \frac{p - \pi_0}{\sqrt{\frac{\pi_0(1 - \pi_0)}{n}}}$

Poznámka: Ve jmenovateli je vždy směrodatná chyba (SE). Vzorec de facto počítá, o kolik směrodatných chyb se náš naměřený výsledek liší od předpokládané hodnoty μ_0 či π_0 . (μ_0 a π_0 jsou hodnoty z nulové hypotézy, které testujeme).

4. Rozhodovací pravidla Pro samotné rozhodnutí, zda H_0 zamítneme, nebo ne, se používají dva ekvivalentní přístupy:

- **Kritický obor:** Porovnáme vypočítanou testovou statistiku s kritickou hodnotou z tabulek. Pokud naše hodnota padne do tzv. *kritického oboru* (je příliš extrémní), H_0 zamítáme.
- **p-hodnota:** Spočítáme *p-value* (p-hodnotu). To je pravděpodobnost, že bychom takto extrémní data získali čistou náhodou za předpokladu, že H_0 platí.
 - **Pravidlo:** Pokud je $p \leq \alpha$ (např. $p \leq 0,05$), H_0 **zamítáme** a přijímáme H_A (výsledek je tzv. *statisticky významný*).
 - Pokud je $p > \alpha$, H_0 **nezamítáme** (nemáme dostatek důkazů pro její zamítnutí).

12 Obousměrné a jednosměrné vztahy kvantitativních veličin

Korelace, regresní model. Koeficient korelace (vlastnosti, bodové grafy). Lineární regresní model vícerozměrný. Odhad parametrů MNČ (požadované vlastnosti dat, důsledky porušení), formální zápis modelu, intervaly spolehlivosti. Očekávané vlastnosti reziduí modelu. Kvalita modelu (vlastnosti reziduí, DW test, graf reziduí P-P nebo Q-Q).

Při zkoumání závislostí mezi dvěma kvantitativními veličinami rozlišujeme dva základní přístupy podle toho, jakou roli veličiny hrají:

- **Jednosměrný vztah (Regresní analýza):** Předpokládáme vztah příčiny a následku. Jedna veličina je nezávislá (vysvětlující) a druhá je závislá (vysvětlovaná). Zkoumáme, jak změna jedné veličiny ovlivňuje druhou.
- **Obousměrný vztah (Korelační analýza):** Zkoumáme vzájemnou vazbu mezi dvěma veličinami bez ohledu na kauzalitu. Obě veličiny mají rovnocenné postavení a ptáme se, zda se mění společně.

12.1 Korelace

Korelace vyjadřuje míru a směr vzájemné lineární závislosti dvou kvantitativních veličin. Slouží k zodpovězení otázky, zda s růstem hodnot jedné veličiny hodnoty druhé veličiny také rostou, klesají, nebo se mění zcela nezávisle.

Důležité pravidlo: Korelace neimplikuje kauzalitu! Pouhý fakt, že se dvě veličiny mění společně, neznamená, že jedna je příčinou druhé. Může na ně působit třetí, společná (skrytá) veličina.

12.2 Koeficient korelace

Pro přesné číselné vyjádření síly a směru obousměrného lineárního vztahu se nejčastěji používá **Pearsonův korelační koeficient** (značí se r). (Další např. Spearmanův nebo Kendallův)

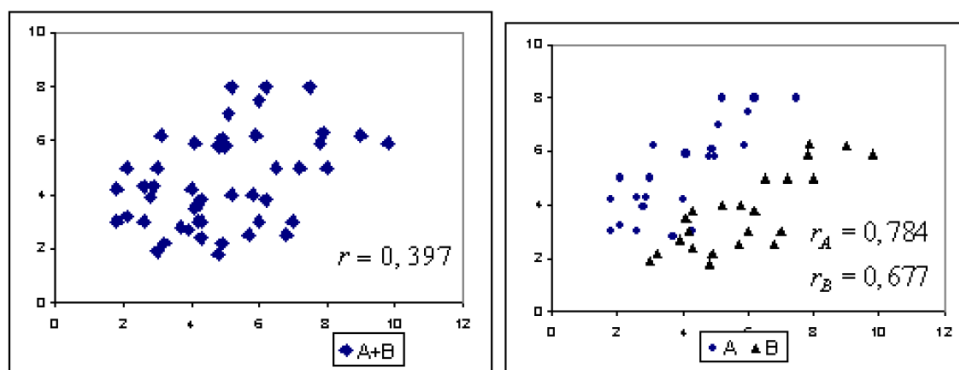
Vlastnosti korelačního koeficientu

- **Obor hodnot:** Koeficient vždy nabývá hodnot z uzavřeného intervalu $\langle -1, 1 \rangle$.
- **Směr závislosti (podle znaménka):**
 - $r > 0$: Kladná (přímá) závislost — s růstem jedné veličiny roste i druhá.
 - $r < 0$: Záporná (nepřímá) závislost — s růstem jedné veličiny druhá klesá.
- **Síla závislosti (podle absolutní hodnoty):**
 - $|r| = 1$: Dokonalá lineární závislost (všechny hodnoty leží přesně na přímce).
 - $r = 0$: Neexistuje žádná lineární závislost.
 - Čím blíže je r k hodnotám 1 nebo -1 , tím je závislost silnější.
- **Symetrie:** Vztah je obousměrný, proto korelace mezi X a Y je stejná jako korelace mezi Y a X ($r_{xy} = r_{yx}$).
- **Citlivost:** Pearsonův koeficient je velmi citlivý na odlehlé hodnoty (outliery), které mohou výsledek výrazně zkreslit.

Bodové grafy (Scatter plots)

Bodový graf je základním vizuálním nástrojem pro prvotní posouzení korelace před samotným výpočtem koeficientu. Konstruuje se v kartézské soustavě souřadnic.

- **Konstrukce:** Každá pozorovací jednotka je v grafu znázorněna jako jeden bod. Hodnota první veličiny určuje pozici na ose x , hodnota druhé veličiny pozici na ose y .
- **Interpretace shluku bodů:**
 - *Rostoucí trend (body směřují vpravo nahoru):* Indikuje kladnou korelaci.
 - *Klesající trend (body směřují vpravo dolů):* Indikuje zápornou korelaci.
 - *Těsnost shluku:* Čím více se body koncentrují kolem pomyslné přímky (jsou tvořeny do úzkého pásu), tím vyšší je absolutní hodnota korelačního koeficientu.
 - *Neuspořádaný oblak:* Body jsou náhodně rozptýlené, což značí nezávislost nebo velmi slabou korelaci ($r \approx 0$).



Obrázek 13: Příklad bodového grafu.

12.3 Regresní model

Formální (matematický) popis vztahu mezi nezávisle proměnnými (příčinami) a závisle proměnnou (následkem). Uvažujeme závisle proměnnou kvantitativní. Model má tvar matematické funkce a hodnot jejich parametrů.

Lineární regresní model jednorozměrný

Lineární regresní model jednorozměrný popisuje vztah mezi jednou nezávislou proměnnou X a jednou závislou proměnnou Y pomocí přímky.

Obecný tvar modelu:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

Význam složek:

- Y : Závisle proměnná (vysvětlovaná, následek).
- X : Nezávisle proměnná (vysvětlující, příčina, prediktor).
- β_0 : Absolutní člen – odhadovaná hodnota Y při $X = 0$.
- β_1 : Směrnice přímky – průměrná změna Y při změně X o jednu jednotku.
- ϵ : Náhodná složka (reziduum) – rozdíl mezi skutečnou a predikovanou hodnotou.

Interpretace a diagnostika:

- **Kladná směrnice** ($\beta_1 > 0$): Y roste se vzrůstem X .
- **Záporná směrnice** ($\beta_1 < 0$): Y klesá se vzrůstem X .
- **Koeficient determinace** (R^2): Udává podíl vysvětlené variability Y , který model zachycuje.
- **Reziduální analýza**: Kontrola předpokladů linearity, homoskedasticity a nezávislosti reziduí pomocí grafu reziduí proti predikcím.

Předpoklady jednoduché lineární regrese:

- Lineární vztah mezi X a Y .
- Náhodné chyby ϵ mají nulový střední průměr.
- Rezidua jsou nezávislá a mají stejnou rozptýlenost (homoskedasticita).
- Rezidua jsou přibližně normálně rozdělena.

Lineární regresní model vícerozměrný

Tento model rozšiřuje jednoduchou lineární regresi tím, že zkoumá vliv dvou a více nezávislých proměnných současně na jednu závislou proměnnou. Využívá se v situacích, kdy samotná jedna příčina nedokáže dostatečně vysvětlit chování sledovaného následku. Vysvětlující proměnné nesmí být vzájemně korelované.

Obecný tvar modelu:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_k X_k + \epsilon$$

Význam jednotlivých složek:

- Y : Závisle proměnná (vysvětlovaná, následek).

- $X_1, X_2, \dots, X_k$: Nezávisle proměnné (vysvětlující, příčiny, prediktory).
- β_0 : Absolutní člen – hodnota Y v $X = 0$.
- $\beta_1, \dots, \beta_k$: Regresní (parciální) koeficienty – udávají, o kolik se v průměru změní hodnota Y při změně příslušné proměnné X_i .
- ϵ : Náhodná složka (chyba modelu) – zachycuje vliv faktorů, které nejsou do modelu zahrnuty, a náhodné kolísání. Představuje rozdíl mezi skutečně naměřenou a modelem teoreticky odhadnutou hodnotou.

Odhad parametrů MNČ a předpoklady modelu

Metoda nejmenších čtverců (MNČ) je základním postupem pro výpočet regresních koeficientů. Její princip spočívá v nalezení takových parametrů, které **minimalizují součet čtverců reziduí** (odchylek skutečně naměřených hodnot od hodnot predikovaných modelem).

Aby byly odhady MNČ optimální (tzv. BLUE – nejlepší lineární nevychýlené odhady), musí data splňovat určité předpoklady:

- **Linearita**: Vztah mezi parametry a závislou proměnnou je lineární.
- **Absence multikolinearity**: Nezávisle proměnné nejsou mezi sebou v dokonalém nebo příliš silném lineárním vztahu. (Určuje se pomocí VIF faktoru.)
- **Homoskedasticita**: Rozptyl náhodných chyb (reziduí) je konstantní napříč všemi hodnotami nezávislých proměnných.
- **Absence autokorelace**: Náhodné chyby jednotlivých pozorování jsou na sobě nezávislé. (Lze ověřit pomocí Durbin-Watsonova testu.)
- **Normální rozdělení reziduí**: Pro korektní statistické testování a tvorbu intervalů spolehlivosti se předpokládá, že chyby mají normální rozdělení. (Určuje se pomocí Kilmogorova-Smirnova testu, Q-Q grafu, P-P grafu.)

Důsledky porušení předpokladů:

- *Nelinearita nebo vynechání důležité proměnné*: Odhady parametrů jsou vychýlené a model popisuje data chybně.
- *Silná multikolinearita*: Zvyšuje rozptyl odhadů, koeficienty jsou výpočetně nestabilní a citlivé na malé změny v datech.

- *Heteroskedasticita a autokorelace*: Odhady zůstávají nevychýlené, ale ztrácejí efektivitu. Zkreslují se standardní chyby odhadů, což vede k chybným závěrům při testování statistické významnosti.

Formální zápis modelu

Při regresní analýze striktně rozlišujeme mezi teoretickým modelem (skutečnost v celé populaci) a empirickým modelem (naš odhad na základě vzorku dat).

Pro i -té pozorování zapisujeme **odhadnutý (empirický) model** jako:

$$y_i = \hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \dots + \hat{\beta}_k x_{ik} + e_i$$

- $\hat{\beta}_0, \dots, \hat{\beta}_k$: Bodové odhady teoretických parametrů získané metodou MNČ.
- e_i (**Reziduum**): Empirická chyba. Vyjadřuje rozdíl mezi skutečně naměřenou hodnotou y_i a modelem odhadnutou (vyrovnanou) hodnotou $\hat{y}_i$. Platí tedy $e_i = y_i - \hat{y}_i$.

Intervaly spolehlivosti

Bodový odhad koeficientu (jedno konkrétní číslo) neposkytuje informaci o své přesnosti. Proto se konstruuji intervaly spolehlivosti, které s určitou předem zvolenou pravděpodobností (typicky 95 %) pokrývají skutečnou, neznámou hodnotu parametru.

- **Interval spolehlivosti pro regresní koeficient (β_j)**: Udává meze, ve kterých se s 95% pravděpodobností nachází skutečný vliv dané proměnné.
 - *Testování významnosti*: Pokud tento interval zahrnuje nulu, znamená to, že vliv dané nezávislé proměnné je statisticky nevýznamný (nelze vyloučit, že její skutečný vliv na Y je nulový).
- **Intervaly spolehlivosti pro predikci**: Slouží k odhadu budoucích nebo dosud nepozorovaných hodnot závislé proměnné.
 - *Interval pro střední hodnotu*: Odhaduje průměrnou hodnotu Y pro danou kombinaci nezávislých proměnných (je užší).
 - *Interval pro individuální hodnotu*: Odhaduje konkrétní jedinou hodnotu Y pro jedno specifické pozorování (je širší, protože zohledňuje i přirozené kolísání jednotlivých datových bodů).

Očekávané vlastnosti reziduí modelu

Rezidua ($e_i = y_i - \hat{y}_i$) představují odchylky skutečných dat od modelu. Aby byl regresní model kvalitní a jeho odhady (včetně testů) byly platné, musí rezidua splňovat tyto základní vlastnosti:

- **Nulová střední hodnota:** Součet (a průměr) všech reziduí musí být roven nule. Model data nepodhodnocuje ani nenadhodnocuje.
- **Konstantní rozptyl (Homoskedasticita):** Velikost chyb nezávisí na hodnotách prediktorů.
- **Nezávislost (Absence autokorelace):** Hodnota jednoho rezidua neobsahuje informaci o hodnotě jiného rezidua (zásadní zejména u časových řad).
- **Nekorelovanost s prediktory:** Rezidua nesmí vykazovat žádný vztah (korelaci) s nezávisle proměnnými X . Pokud ano, v modelu chybí důležitá informace.
- **Normální rozdělení:** Chyby mají symetrické rozdělení kolem nuly (Gaussova křivka). Tento předpoklad je nutný pro platnost intervalů spolehlivosti a testování hypotéz.

Kvalita modelu

Kvalita regresního modelu se neposuzuje pouze podle koeficientu determinace (R^2), ale zásadní je ověřit, zda rezidua skutečně splňují teoretické předpoklady. K tomu se využívá grafická diagnostika a statistické testy.

- **Durbin-Watsonův (DW) test:**
 - Statistický test určený k detekci **autokorelace 1. řádu** mezi rezidui (častý problém u dat s časovým vývojem).
 - Hodnota testové statistiky se pohybuje v intervalu $\langle 0, 4 \rangle$.
 - *Interpretace:* $DW \approx 2$ znamená, že autokorelace není přítomna (ideální stav). Hodnoty blíží se 0 značí silnou kladnou autokorelaci, hodnoty blíží se 4 silnou zápornou autokorelaci.
- **Grafy normality (Q-Q plot a P-P plot):**
 - Vizuální nástroje pro rychlé posouzení, zda mají rezidua **normální rozdělení** (klíčové pro platnost testů a intervalů).
 - **Q-Q plot (Quantile-Quantile):** Porovnává kvantily empirického rozdělení reziduí s teoretickými kvantily normálního rozdělení.
 - **P-P plot (Probability-Probability):** Porovnává distribuční funkce (kumulativní pravděpodobnosti).

- *Interpretace*: Pokud data pocházejí z normálního rozdělení, body v grafu se musí těsně řadit podél rovné diagonální přímky. Jakékoliv systematické odchýlení (např. tvar "S" nebo odlehle konce) indikuje porušení předpokladu normality.

13 Principy strojového učení

Rozdíl mezi klasickým programováním a strojovým učení, metody učení s učitelem a bez učitele, trénovací a testovací data, implementace v jazyce Python, vyhodnocení výsledků učení.

- **Učení**: Proces, kdy systém zlepšuje výkon na základě zkušenosti (Herbert Simon). → změna chování vedoucí k lepším výsledkům při opakovaném řešení problému.
- **Strojové učení (Machine Learning, ML)**: Obor, který dává počítači schopnost učit se bez explicitního naprogramování pravidel (Arthur Samuel). Formálně (Tom Mitchell): Počítačový program se učí ze zkušenosti E s ohledem na určitou třídu úloh T a míru výkonnosti P , pokud se jeho výkonnost v úlohách T , měřená pomocí P , zlepšuje se zkušeností E .

13.1 Rozdíl mezi klasickým programováním a strojovým učení

- **Klasické programování**: Programátor přesně definuje pravidla (algoritmus).

Data (Vstupy) + Pravidla (Program) → Odpovědi (Výstupy)

- **Strojové učení**: Algoritmus se učí pravidla sám na základě poskytnutých dat.

Data (Vstupy) + Odpovědi (Očekávané výstupy) → Pravidla (Natrénovaný model)

13.2 Metody učení s učitelem a bez učitele

Základní rozdělení metod strojového učení podle způsobu, jakým algoritmus zpracovává data:

- **Učení s učitelem (Supervised learning)**: K dispozici jsou vstupy i správné odpovědi (cíl). Řeší úlohy *regrese* a *klasifikace*.
- **Učení bez učitele (Unsupervised learning)**: K dispozici jsou pouze vstupy (bez odpovědí). Řeší úlohy *shlukování* (clustering) a *redukce dimenzionality*.
- **Učení posilováním (Reinforcement learning)**: Agent se učí komunikovat s prostředím pomocí systému odměn a trestů s cílem maximalizovat celkovou odměnu.

1. Učení s učitelem (Supervised Learning) Model se učí na tzv. označených datech (*labeled data*). Každý vstupní příklad obsahuje i správnou odpověď (např. fotografie zvířat s popisky „pes“, „kočka“ nebo parametry bytu s jeho skutečnou cenou). Cílem je naučit model mapovat vstupy na správné výstupy u dosud neviděných dat.

Základní metody a jejich principy:

1. Lineární regrese

- *Úloha:* Regrese (predikce spojité číselné hodnoty, např. ceny nemovitosti).
- *Princip:* Prokládá datovými body přímku (nebo vícerozměrnou nadrovinu) tak, aby se minimalizovala celková chyba (obvykle součet čtverců odchylek skutečných dat od této přímky, tzv. MSE).

2. Logistická regrese

- *Úloha:* Klasifikace (nejčastěji binární, např. e-mail je/není spam).
- *Princip:* Výstup z lineární rovnice se transformuje pomocí *sigmoidální funkce*, která výsledek převede do intervalu $(0, 1)$. Hodnota pak reprezentuje pravděpodobnost, že vstup patří do dané třídy.

3. Rozhodovací stromy (Decision Trees)

- *Úloha:* Klasifikace i regrese.
- *Princip:* Datový prostor se rekurzivně štěpí na menší a menší části na základě podmínek (např. "je věk > 30 ?"). Uzly se štěpí tak, aby nově vzniklé skupiny byly co nejvíce jednorodé. Používají se metriky jako Giniho index nebo Informační zisk.

4. Metoda podpůrných vektorů (SVM - Support Vector Machines)

- *Úloha:* Klasifikace.
- *Princip:* Algoritmus hledá optimální hraniční nadrovinu, která od sebe odděluje třídy s maximální vzdáleností mezi skupinami (*margin*). Důležité jsou pouze body ležící nejbližší této hranici, tzv. *podpůrné vektory*. Pro nelineárně separabilní data využívá tzv. *kernel trick* (transformaci dat do vyšší dimenze).

2. Učení bez učitele (Unsupervised Learning) Model dostává neoznačená data (bez správných odpovědí). Cílem je najít v datech skryté struktury, vzorce, nebo je seskupit na základě jejich vnitřní podobnosti.

Základní metody a jejich principy:

1. K-Means (K-středů)

- *Úloha:* Shlukování (Clustering) – segmentace zákazníků, detekce anomálií.
- *Princip:* Uživatel předem zvolí počet shluků K . Algoritmus iterativně střídá dva kroky:
 - 1) Přiřadí každý datový bod k nejbližšímu těžišti (*centroidu*).
 - 2) Přepočítá polohu těžišť jako průměr bodů v daném shluku. Proces končí, když se těžiště přestanou hýbat.

2. Hierarchické shlukování

- *Úloha:* Shlukování (Clustering) – např. taxonomie v biologii, analýza podobnosti dokumentů.
- *Princip:* Algoritmus postupně vytváří stromovou hierarchii shluků (tzv. dendrogram). Nejčastěji se používá *aglomerativní* (zdola nahoru) přístup: na začátku je každý bod považován za samostatný shluk a v každém kroku se dva nejbližší shluky spojí do jednoho. Obrovskou výhodou je, že nemusíme předem znát počet shluků K . Ten si můžeme zvolit vizuálně až dodatečně, odříznutím dendrogramu v požadované výšce.

3. DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

- *Úloha:* Shlukování na základě hustoty a detekce odlehlých hodnot (anomálií).
- *Princip:* Na rozdíl od K-Means, který tvoří pouze kulovité shluky, DBSCAN definuje shluk jako souvislou oblast s vysokou hustotou bodů, oddělenou oblastmi s nízkou hustotou. Algoritmus vyžaduje dva parametry: poloměr okolí (ϵ) a minimální počet bodů v tomto okolí (*MinPts*). Dokáže najít shluky libovolného (i nelineárního) tvaru a izolované body, které nejsou blízko žádného shluku, automaticky označí jako šum (noise / outliers).

Analýza hlavních komponent (PCA - Principal Component Analysis):

- *Úloha:* Redukce dimenzionality (snížení počtu vlastností dat pro vizualizaci nebo zrychlení jiných algoritmů).
- *Princip:* Jedná se o lineární transformaci. Algoritmus hledá nové, vzájemně kolmé osy (hlavní komponenty) tak, aby první osa zachycovala největší možný rozptyl (informaci) v datech, druhá největší zbylý rozptyl atd. Vícerozměrná data lze tak zploštit do nižší dimenze při minimální ztrátě podstatných informací.

13.3 Trénovací a testovací data

Základním pravidlem strojového učení je, že model nesmíme vyhodnocovat na stejných datech, na kterých se učil. Jinak bychom nezjistili, zda se naučil obecná pravidla, nebo se data jen "naučil nazpaměť". Proto se dostupná data rozdělují:

- **Trénovací množina (Training set):** Obvykle 70–80 % dat. Slouží k samotnému učení modelu (nastavení vnitřních vah a pravidel).
- **Validační množina (Validation set):** Občas vyčleněná z trénovacích dat. Slouží k ladění tzv. *hyperparametrů* (nastavení samotného algoritmu, např. hloubky rozhodovacího stromu). V praxi se často nahrazuje tzv. křížovou validací (Cross-validation).
- **Testovací množina (Testing set):** Obvykle 20–30 % dat. Jde o zcela neznámá data, na kterých ověříme finální výkon modelu a jeho schopnost zobecňovat (generalizovat) na nová data.

Problémy při učení:

- **Přeučení (Overfitting):** Model je příliš složitý a naučil se trénovací data nazpaměť (včetně šumu). Má vysokou přesnost na trénovacích datech, ale selhává na testovacích a reálných.
- **Nedoučení (Underfitting):** Model je příliš jednoduchý a nedokázal v datech najít hledané vztahy. Má nízkou přesnost na obou množinách.

13.4 Implementace v jazyce Python

Python je v současnosti standardním jazykem pro strojové učení díky masivnímu ekosystému knihoven.

Klíčové knihovny:

- **Scikit-learn:** Standardní knihovna pro tradiční strojové učení (obsahuje implementaci SVM, K-Means, regresí atd., a také nástroje pro dělení dat a výpočet metrik).
- **Pandas a NumPy:** Knihovny pro čištění, manipulaci a vektorové operace s datovými sadami.
- **TensorFlow a PyTorch:** Pokročilé frameworky sloužící pro návrh neuronových sítí a hluboké učení (*Deep Learning*).
- **Matplotlib, seaborn:** Knihovny na vizualizaci dat a grafy.

Typický životní cyklus v knihovně Scikit-learn: Všechny algoritmy v této knihovně sdílí stejné a velmi intuitivní API. Postup zapsaný v kódu vypadá obvykle takto:

1. `X_train, X_test, y_train, y_test = train_test_split(X, y)` (*Rozdělení dat*)
2. `model = SVC()` (*Inicializace vybraného algoritmu*)
3. `model.fit(X_train, y_train)` (*Učení modelu na trénovacích datech*)
4. `predikce = model.predict(X_test)` (*Predikce na dosud neviděných testovacích datech*)
5. `accuracy_score(y_test, predikce)` (*Vyhodnocení pomocí zvolené metriky*)

13.5 Vyhodnocení výsledků učení (Metriky)

Volba metriky k hodnocení modelu kriticky závisí na tom, jakou úlohu model řeší (regrese vs. klasifikace).

1. Metriky pro klasifikaci: Základem je **Matice záměn (Confusion Matrix)**, která porovnává skutečné třídy s predikovanými a dělí výsledky na: *True Positives (TP)*, *True Negatives (TN)*, *False Positives (FP)* a *False Negatives (FN)*. Z ní se počítají metriky:

- **Accuracy (Přesnost):** Podíl všech správných predikcí k celkovému počtu vzorků.
- **Precision (Přezicnost):** Kolik z těch, které model označil jako pozitivní, bylo skutečně pozitivních ($TP / (TP + FP)$). Důležité, pokud chceme minimalizovat falešné popluchy (např. označení neškodného e-mailu jako spam).
- **Recall (Citlivost):** Kolik ze všech skutečně pozitivních případů model odhalil ($TP / (TP + FN)$). Důležité v medicíně (nechceme pacienta s nemocí označit za zdravého).
- **F1-Score:** Harmonický průměr mezi Precision a Recall. Ideální metrika pro nevyvážené datové sady.

2. Metriky pro regresi: U regrese (predikce číselné hodnoty) hodnotíme velikost odchylky od skutečnosti.

- **MSE (Mean Squared Error - Průměrná čtvercová chyba):** Obrovské chyby (odchylky) jsou díky umocnění na druhou penalizovány mnohem více než malé chyby.
- **MAE (Mean Absolute Error - Průměrná absolutní chyba):** Ukazuje průměrnou chybu v původních jednotkách (např. model predikuje cenu bytu s průměrnou chybou $\pm 50\,000$ Kč).

- **R^2 skóre (Koeficient determinace):** Udává, jaké procento rozptylu cílové proměnné model vysvětluje. Hodnoty blízko 1 znamenají perfektní model, hodnota 0 znamená, že model nepredikuje o nic lépe, než kdybychom vždy tipnuli pouhý průměr trénovacích dat.

14 Neuronové sítě

Základní pojmy (neuron, váha, práh, aktivační funkce, bias, synapse), perceptron, vícevrstvá dopředná neuronová síť, hluboké neuronové sítě, aplikační oblasti.

14.1 Základní pojmy (neuron, váha, práh, aktivační funkce, bias, synapse)

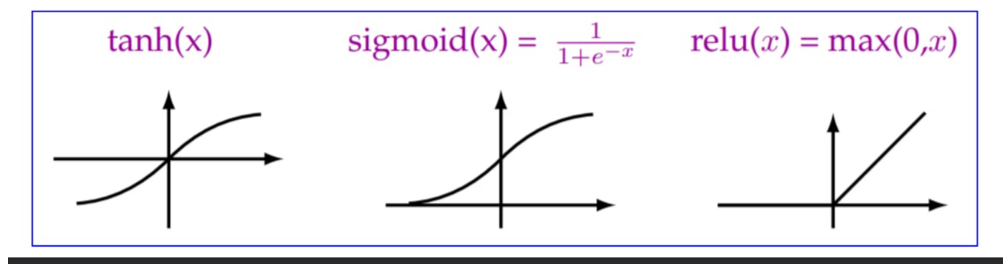
Umělé neuronové sítě jsou výpočetní modely inspirované strukturou a fungováním biologického mozku. Slouží k řešení složitých nelineárních úloh (např. rozpoznávání obrazu či zpracování přirozeného jazyka). Základem sítě je matematický model umělého neuronu.

Stavební prvky umělého neuronu:

- **Neuron (Uzel / Node):** Základní výpočetní jednotka sítě. Přijímá signály z jiných neuronů (nebo ze vstupních dat), matematicky je zpracuje a svůj výsledek pošle dál.
- **Synapse (Spojení):** Orientované propojení mezi dvěma neurony. Zajišťuje přenos signálu (číselné hodnoty) z výstupu jednoho neuronu na vstup dalšího.
- **Váha (Weight, w):** Každá synapse má přiřazenou reálnou hodnotu - váhu. Ta určuje sílu a důležitost daného spojení. Znamená to, že signál procházející synapsí se touto váhou vynásobí. Samotné učení sítě spočívá právě v neustálém upravování těchto vah.
- **Práh (Threshold):** Historický koncept z prvních modelů neuronů (McCulloch-Pitts). Určuje hraniční hodnotu - pokud vážený součet vstupních signálů překročí tento práh, neuron „se-pne“ (vydá signál 1), jinak zůstane neaktivní (vydá 0).
- **Bias (Vychýlení, b):** Moderní náhrada a zobecnění prahu. Jde o speciální vstup do neuronu, který má vždy hodnotu 1, ale má svou vlastní nastavitelnou váhu b . Z matematického hlediska umožňuje posouvat aktivační funkci doleva nebo doprava. Díky biasu může neuron generovat výstup, i když jsou všechny běžné vstupy nulové. (Poznámka: Práh a bias jsou matematicky ekvivalentní, platí $\text{bias} = -\text{práh}$).
- **Aktivační funkce (f):** Matematická funkce aplikovaná na konečný součet všech vážených vstupů a biasu. Zajišťuje transformaci tohoto součtu na požadovaný výstupní rozsah (např.

do intervalu 0 až 1). Její absolutně nejdůležitější rolí je vnesení nelinearity do sítě. Kdyby neurony neměly nelineární aktivační funkce, celá i ta nejsložitější síť by se dala zjednodušit na jedinou lineární rovnici.

- *Běžné aktivační funkce:* Skoková (Step), Sigmoida, Tanh, ReLU (dnes nejpoužívanější).



Obrázek 14: Příklad aktivačních funkcí.

14.2 Perceptron

Perceptron (navržený Frankem Rosenblattem v roce 1957) je historicky nejstarší a nejjednodušší model neuronové sítě.

- **Struktura:** Skládá se pouze ze vstupní vrstvy (přijímá data) a jednoho výstupního neuronu. K aktivaci využívá skokovou (prahovou) funkci.
- **Učení:** Perceptronové pravidlo - pokud síť udělá chybu, váhy se mírně upraví tak, aby se v dalším kroku chyba zmenšila.
- **Zásadní omezení (Problém XOR):** Perceptron dokáže řešit pouze tzv. lineárně separabilní problémy (kdy lze data v grafu oddělit jednou rovnou čarou). Nedokáže vyřešit ani jednoduchou logickou funkci XOR (exkluzivní OR). Tento matematický důkaz (Minsky a Papert, 1969) vedl k tzv. *první zimě umělé inteligence*, kdy opadl zájem o neuronové sítě, protože se zdály být slepou uličkou.

14.3 Vícevrstvá dopředná neuronová síť (MLP - Multi-Layer Perceptron)

Omezení perceptronu se podařilo překonat přidáním tzv. skrytých vrstev mezi vstup a výstup. Vstupní vrstva obsahuje tolik neuronů kolik je vstupů (např. pro každý pixel). Výstupní vrstva obsahuje tolik neuronů kolik je výstupů (např. kategorie zvířat).

- **Struktura (Dopředná síť):** Skládá se ze vstupní vrstvy, jedné nebo více skrytých vrstev a výstupní vrstvy. Termín *dopředná* (Feedforward) znamená, že signál putuje pouze jedním směrem – od vstupu k výstupu. Neexistují zde žádné cykly ani zpětné vazby. Každý neuron ve vrstvě je propojen se všemi neurony předchozí vrstvy.
- **Řešení nelinearity:** Díky skrytým vrstvám a nelineárním aktivačním funkcím dokáže MLP řešit nelineárně separabilní problémy (včetně problému XOR). Podle tzv. *Univerzální aproximační věty* dokáže MLP (s alespoň jednou dostatečně širokou skrytou vrstvou) aproximovat jakoukoliv spojitou matematickou funkci.
- **Učení (Backpropagation):** Trénování MLP probíhá pomocí algoritmu zpětného šíření chyby (*Backpropagation*). Síť vypočítá výsledek, porovná ho s očekávanou hodnotou a pomocí derivací (gradientního sestupu) pošle informaci o chybě zpět sítí. Váhy se upravují od konce směrem k začátku tak, aby se chyba minimalizovala. To vyžaduje použití hladkých (derivovatelných) aktivačních funkcí, např. Sigmoidy.

14.4 Hluboké neuronové sítě (Deep Neural Networks / Deep Learning)

Hluboká neuronová síť je v podstatě vícevrstvá dopředná síť, která má velký počet skrytých vrstev (desítky až stovky). Z této architektury vzešel obor zvaný **Hluboké učení** (Deep Learning).

- **Hierarchická extrakce příznaků:** Hlavní výhodou hloubky je, že si síť dokáže sama skládat složité koncepty z jednoduchých.
 - *První vrstvy* se učí rozpoznávat hrany a barvy.
 - *Prostřední vrstvy* skládají hrany do tvarů (kruhy, křivky, nos, oko).
 - *Poslední vrstvy* rozpoznávají komplexní objekty (konkrétní obličeje, auto, pes).
- **Problém mizejícího gradientu (Vanishing Gradient):** Historický problém hlubokých sítí. Při zpětném šíření chyby přes mnoho vrstev se chyba (gradient) neustále násobí malými čísly, až se u prvních vrstev "ztratí" (blíží se nule). První vrstvy se tak přestávaly učit.
- Rozmach hlubokých sítí po roce 2010 umožnily tři faktory:
 1. Nástup **GPU** (grafických karet), které zvládají tisíce maticových výpočtů paralelně.
 2. Dostupnost obrovského množství dat (Big Data).
 3. Využití aktivační funkce **ReLU** (Rectified Linear Unit), která efektivně řeší problém mizejícího gradientu.

14.5 Aplikační oblasti neuronových sítí

Neuronové sítě (zejména hluboké učení) se využívají v úlohách, kde je obtížné nebo nemožné vytvořit přesná pravidla pomocí klasického programování. Jejich hlavní využití je ke zpracování tzv. **nestrukturovaných dat** (obrázky, text, zvuk).

Hlavní oblasti nasazení:

- **Počítačové vidění (Computer Vision):**
 - *Typické architektury:* Konvoluční neuronové sítě (CNN - Convolutional Neural Networks).
 - *Aplikace:* Rozpoznávání obličejů (biometrie), medicínská diagnostika (automatická detekce nádorů z rentgenu či MRI), systémy autonomního řízení vozidel (detekce chodců a značek), vizuální kontrola kvality v průmyslu.
- **Zpracování přirozeného jazyka (NLP - Natural Language Processing):**
 - *Typické architektury:* Rekurentní sítě (RNN) a dnes především **Transformery** (např. GPT).
 - *Aplikace:* Strojový překlad (Google Translate, DeepL), chatboti a virtuální asistenti (Chat-GPT), analýza sentimentu (automatické hodnocení, zda je recenze produktu pozitivní či negativní), sumarizace dlouhých textů.
- **Rozpoznávání a syntéza řeči a zvuku:**
 - *Aplikace:* Automatický přepis mluveného slova na text (Speech-to-Text, tvorba titulků), hlasové ovládání (Siri, Alexa), identifikace mluvčího, generování přirozeně znějícího syntetického hlasu.
- **Doporučovací systémy (Recommender Systems):**
 - *Aplikace:* Streamovací platformy a e-commerce (Netflix, Spotify, Amazon, YouTube). Sítě analyzují obrovské množství historických vzorců chování milionů uživatelů a predikují, jaký film, písnička nebo produkt bude uživatele s největší pravděpodobností zajímat.
- **Prediktivní analytika a detekce anomálií:**
 - *Aplikace:* Detekce bankovních podvodů (vyhodnocení, zda aktuální transakce kartou nevybočuje z běžného chování klienta), prediktivní údržba v průmyslu (předvídaní poruchy stroje na základě vibrací a zvuku), analýza rizik v pojišťovnictví.
- **Generativní umělá inteligence (Generative AI):**
 - *Typické architektury:* Generativní adversariální sítě (GANs) a Difuzní modely.

- *Aplikace:* Generování realistických obrázků z textového popisu (Midjourney, DALL-E), syntéza nového designu, tvorba deepfakes, automatické generování zdrojového kódu pro programátory.

15 Principy teorie her

Základní pojmy (hráč, pravidla, strategie, výplata), klasifikace her, reprezentace hry maticí a stromem, řešení maticové hry s nulovým a nenulovým součtem, eliminace dominovaných strategií, Nashova rovnováha, řešení hry s rizikem a neurčitostí, řešení hry s neúplnou informací.

15.1 Základní pojmy

Teorie her je matematická disciplína, která analyzuje konfliktní rozhodovací situace. V těchto situacích závisí výsledek (zisk nebo ztráta) každého účastníka nejen na jeho vlastním rozhodnutí, ale také na rozhodnutích ostatních účastníků.

- **Hráč:** Racionálně uvažující entita (člověk, firma, stát, algoritmus), která činí rozhodnutí s cílem maximalizovat svou vlastní výplatu.
 - Předpoklad *racionality* je klíčový - předpokládáme, že hráč nedělá chyby a vždy volí to nejlepší pro sebe.
 - Ve specifických modelech může vystupovat i tzv. **Příroda** (pseudohráč), která nemá vlastní cíl, ale provádí náhodné tahy (rozhoduje na základě pravděpodobnosti, např. hází kostkou). Nebo **P-inteligentní** chová se inteligentně jen s nějakou pravděpodobností.
- **Pravidla:** Striktní rámec, který definuje logiku hry. Určuje:
 - Jaké tahy mají hráči k dispozici.
 - V jakém pořadí hráči svá rozhodnutí činí (současně vs. postupně).
 - Jaké informace mají v momentě rozhodování k dispozici (zda vidí, co zahrál soupeř).
 - Kdy hra končí.
 - Určuje výplatní funkce.
- **Strategie:** Kompletní plán chování hráče. Nejedná se o pouhý jeden tah, ale o strategii, která říká, jaký tah hráč provede v *absolutně každé situaci*, která může ve hře nastat. Značí se obvykle písmenem s a množina všech možných strategií hráče jako S . Dělí se na:

- *Čisté strategie*: Hráč zvolí jeden konkrétní plán a toho se stoprocentně drží.
- *Smíšené strategie*: Hráč vybírá mezi několika čistými strategiemi náhodně na základě určitého rozdělení pravděpodobnosti (např. v kámen-nůžky-papír hraje každou možnost s pravděpodobností $1/3$).
- **Výplata**: Kvantitativní ohodnocení výsledku hry pro daného hráče (peníze, zisk, čas, trestné body). Zapisuje se pomocí **výplatní funkce**, která každé možné kombinaci strategií všech hráčů přiřadí konkrétní číselnou hodnotu.

15.2 Klasifikace her

1. Podle součtu výplat

- **Hry s nulovým (konstantním) součtem**: Platí pravidlo „co jeden získá, to druhý ztratí“ (součet výplat obou hráčů je vždy 0). Jde o čistý konflikt (např. šachy, poker). Zájmy hráčů jsou přesně opačné.
- **Hry s nenulovým součtem**: Může nastat situace oboustranného zisku (win-win) nebo oboustranné ztráty (lose-lose). (např. Vězňovo dilema)

2. Podle pořadí tahů

- **Simultánní hry**: Hráči se rozhodují současně, aniž by věděli, co právě zvolil soupeř (např. kámen-nůžky-papír). K reprezentaci se obvykle používá **matice**.
- **Sekvenční hry**: Hráči se v tazích střídají a reagují na předchozí kroky soupeře. Často se reprezentují pomocí **rozhodovacího stromu**.

3. Podle dostupnosti informací

- **S dokonalou informací**: Hráč má při svém tahu absolutní přehled o celé historii všech předchozích tahů (např. piškvorky).
- **S nedokonalou informací**: Hráč nezná některé předchozí tahy soupeře (např. mariáš, nebo jakákoliv simultánní hra, kde se tahá současně).
- **S neúplnou informací**: Hráč nezná cíle (výplatní funkce) ostatních hráčů (neví přesně, proti komu hraje a co soupeř získá).

4. Podle možnosti spolupráce

- **Kooperativní hry:** Hráči spolu mohou komunikovat a uzavírat závazné (vymahatelné) dohody o rozdělení zisku.
- **Nekooperativní hry:** Závazné dohody nejsou možné. I když hráči spolupracují, činí tak jen proto, že je to pro ně v daný moment osobně výhodné.

5. Podle symetričnosti

- **Symetrická hry:** Hráči mají stejné výplatní funkce a možné strategie.
- **Asymetrická hry:** Výplaty a možnosti hráčů se mohou lišit.
- **Signální hry:** Hráči mají rozdílný přístup k informacím (např. zaměstnavatel-uchazeč).

Další dělení

- **Jednokolová / vícekolová**
- **Konečná / diskrétní / spojitá**
- **Klasická / Reverzní / Evoluční**

15.3 Reprezentace hry maticí a stromem

Pro matematický zápis a řešení her se používají dva základní modely, které se volí podle pořadí tahů a dostupnosti informací.

1. Maticová reprezentace (Hra v normálním tvaru) Používá se především pro **simultánní hry**, kde se hráči rozhodují současně, nebo bez znalosti tahu soupeře.

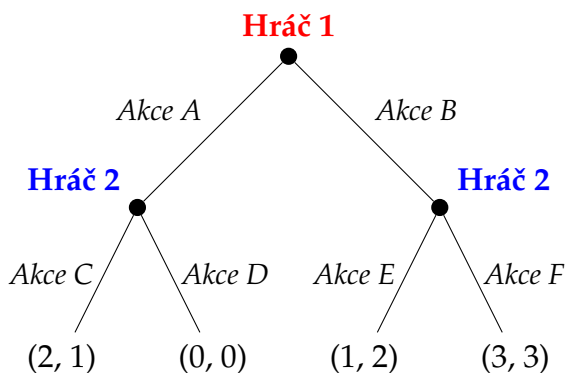
- **Řádky** matice představují všechny možné strategie **Hráče 1**.
- **Sloupce** matice představují všechny možné strategie **Hráče 2**.
- **Buňky (průsečíky)** obsahují výplaty obou hráčů ve formátu (v_1, v_2) , kde v_1 je zisk prvního a v_2 zisk druhého hráče. V případě hry s nulovým součtem je zde pouze výplata **Hráče 1**, výplata **Hráče 1** je rovna $-(\text{Výplata } H_1)$.

		HRÁČ 2	
		Mlčet	Zradit
HRÁČ 1	Mlčet	$(-1, -1)$	$(-5, 0)$
	Zradit	$(0, -5)$	$(-3, -3)$

Obrázek 15: Maticová reprezentace (Normální tvar) na příkladu Věžňova dilematu. Červená odpovídá řádkům, modrá sloupcům.

2. Stromová reprezentace (Hra v explicitním tvaru) Používá se pro **sekvenční hry**, kde se hráči v tazích střídají a reagují na sebe.

- **Uzly:** Rozhodovací body. Každý uzel patří konkrétnímu hráči, který je zrovna na tahu. (Kořen stromu je první tah hry).
- **Hrany:** Možné tahy (akce), které má hráč v daném uzlu k dispozici.
- **Listy (Koncové body):** Konec hry, u kterého jsou vypsány výplaty (v_1, v_2) pro všechny hráče.



Obrázek 16: Stromová reprezentace (Explicitní tvar). Nejdříve táhne Hráč 1 (vybírá A nebo B), na jeho volbu pak reaguje Hráč 2.

15.4 Řešení maticových her a Nashova rovnováha

1. Eliminace dominovaných strategií Než začneme hru složitě řešit, je často možné ji zjednodušit vyřazením nelogických tahů.

- **Dominovaná strategie:** Strategie A je pro hráče (striktně) dominovaná strategií B, pokud mu strategie B přináší vždy lepší výplatu bez ohledu na to, co zahraje soupeř.

- **Princip iterativní eliminace:** Protože předpokládáme, že hráči jsou racionální, nikdy nezahrají dominovanou strategii. Můžeme ji tedy z matice (vyškrtnutím řádku nebo sloupce) odstranit. Tím se hra zmenší a tento proces můžeme střídavě opakovat pro oba hráče, dokud nezbude matice, kterou už zredukovat nelze.

2. Nashova rovnováha (NE - Nash Equilibrium)

- **Definice:** Jedná se o takovou kombinaci strategií (stav hry), ve které žádný z hráčů nemůže jednostrannou změnou své strategie svůj zisk zvýšit. Hráči hrají vůči sobě vzájemně *nejlepší odpovědi*.
- **Hledání v matici (Metoda podtrhávání):**
 1. Postupně zafixujeme sloupce (představíme si, že Hráč 2 zahrál tento tah) a podtrhneme v tomto sloupci nejvyšší výplatu pro Hráče 1.
 2. Postupně zafixujeme řádky (tah Hráče 1) a podtrhneme v něm nejvyšší výplatu pro Hráče 2.
 3. Buňka, ve které jsou podtrženy obě výplaty, je Nashovou rovnováhou v *čistých strategiích*.
- **Nashova věta:** Každá konečná hra má alespoň jednu Nashovu rovnováhu. Pokud neexistuje v čistých strategiích (např. Kámen-Nůžky-Papír nemá žádnou buňku se dvěma podtržítky), garantovaně existuje ve *smíšených strategiích* (pomocí pravděpodobností).

3. Řešení hry s nulovým součtem Jelikož platí $v_1 + v_2 = 0$, zájmy hráčů jsou přesně opačné. Cokoliv Hráč 1 získá, to Hráč 2 ztratí. V matici se proto často uvádí jen výplata Hráče 1. K řešení se využívá **Minimaxový teorém** (John von Neumann):

- **Maximin Hráče 1 (Dolní cena hry):** Hráč 1 hledá tah, který mu zaručí největší možný zisk za předpokladu, že Hráč 2 bude hrát pro něj co nejhůře. (Maximalizuje své minimální zisky).
- **Minimax Hráče 2 (Horní cena hry):** Hráč 2 chce minimalizovat zisk Hráče 1. Hledá tah, kde Hráč 1 získá co nejméně v nejhorším případě. (Minimalizuje maximální ztráty).
- **Sedlový bod:** Pokud se Maximin = Minimax, hra má řešení v čistých strategiích. Tato společná hodnota se nazývá **cena hry**. (Sedlový bod je v hrách s nulovým součtem totožný s Nashovou rovnováhou).
- Pokud se Maximin $\neq$ Minimax, sedlový bod neexistuje a hra se řeší ve smíšených strategiích pomocí lineárního programování (nebo graficky pro matice $2 \times n$).

	Sloupec X	Sloupec Y	Sloupec Z	
Řádek A	2	4	-1	Min = -1
Řádek B	3	5	②	Min = 2 (MAXIMIN)
	Max = 3	Max = 5	Max = 2 (MINIMAX)	

Sedlový bod existuje (Cena hry = 2)

Obrázek 17: Řešení hry s nulovým součtem. Maximin řádkového hráče je roven Minimaxu sloupceového hráče, hra má tedy čisté řešení.

4. Řešení hry s nenulovým součtem Hráči nejsou absolutní nepřátelé, existuje zde prostor pro oboustranný zisk nebo oboustrannou ztrátu.

- Hra se řeší hledáním Nashových rovnováh (kterých může být více).
- Konflikt s Pareto efektivitou:** Zásadním problémem těchto her je, že Nashova rovnováha nemusí být globálně optimální. Klasickým příkladem je *Věžňovo dilema* - racionální postup oba hráče neomylně dovede do Nashovy rovnováhy (Zradit, Zradit), která je ale pro oba prokazatelně horší než varianta (Mlčet, Mlčet). K lepší variantě se však bez možnosti *závazné dohody* (kooperace) nedostanou, protože samostatně by pro každého bylo výhodné dohodu porušit.

		VĚZEŇ 2 (Sloupec)	
		Mlčet	Zradit
VĚZEŇ 1 (Řádek)	Mlčet	(-1, -1)	(-5, 0)
	Zradit	(0, -5)	(-3, -3)

Nashova rovnováha

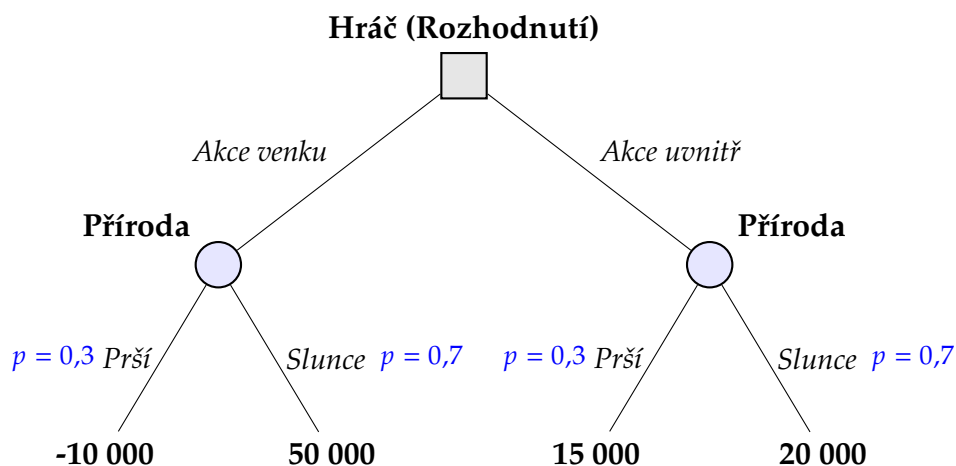
Obrázek 18: Řešení Věžňova dilematu. Podtržené hodnoty označují nejlepší odpověď hráče na tah soupeře. Červená buňka se dvěma podtržítky je stabilní Nashova rovnováha.

15.5 Řešení hry s rizikem a neurčitostí (Hry proti přírodě)

Tyto modely nepopisují standardní konflikt dvou racionálních protivníků, ale situaci, kdy jeden racionální hráč činí rozhodnutí vůči prostředí tzv. **Přírodě**. Příroda nesleduje žádný cíl a nesnaží se hráče poškodit. Její tahy jsou dány pouze náhodou.

1. Rozhodování za rizika Hráč předem zná všechny možné stavy přírody a zná i pravděpodobnosti, se kterými tyto stavy nastanou (např. pravděpodobnost, že bude zítra pršet, je 30 %).

- **Pravidlo očekávané hodnoty:** K řešení se využívá výpočet střední hodnoty. Hráč pro každou svou strategii sečte součiny možných výplat a jejich pravděpodobností. Racionální hráč volí tu strategii, která maximalizuje celkový očekávaný zisk.



Obrázek 19: Rozhodovací strom pro hru za rizika. Čtvercový uzel značí vědomé rozhodnutí hráče, kruhové uzly představují tahy Přírody (náhodné jevy se známou pravděpodobností p). Očekávaná hodnota pro akci venku je $E(V) = 0,3 \cdot (-10\,000) + 0,7 \cdot 50\,000 = 32\,000$. Očekávaná hodnota pro akci uvnitř je $E(U) = 0,3 \cdot 15\,000 + 0,7 \cdot 20\,000 = 18\,500$. Racionální hráč volí akci venku, protože maximalizuje očekávaný zisk.

2. Rozhodování za neurčitosti Hráč zná všechny možné budoucí stavy přírody, ale nezná pravděpodobnosti jejich výskytu, případně je vůbec nelze odhadnout. Hra se řeší pomocí různých rozhodovacích kritérií na základě toho, jaký postoj má hráč k riziku:

Kritérium	Postup výběru	Charakteristika
Princip maximaxu	Z nejhorších možných výsledků strategií vybere ten nejlepší.	Absolutní pesimista (minimalizuje ztráty, hraje na jistotu).
Princip optimismu racionálního hráče (Hurwiczovo)	Vážený průměr nejhoršího a nejlepšího výsledku pro každou strategii.	Kompromisní. Využívá koeficient optimismu α , kde $0 \leq \alpha \leq 1$.
Princip maximaxu ztráty (Savageovo)	Pojistí hráče proti velkým ztrátám oproti optimálnímu rozhodnutí (kdyby strategii přírody znal předem).	Minimalizuje ušlý zisk. Řeší se pomocí pomocné ztrát a v ní se hledá maximax.
Laplaceovo	Spočítá prostý aritmetický průměr výplat každé strategie a vybere nejvyšší jako u rizika.	Princip nedostatečného důvodu (předpokládá, že všechny stavy mají stejnou šanci).

15.6 Řešení hry s neúplnou informací

Ve hrách s neúplnou informací hráči neznají výplatní funkce (cíle, motivace nebo možnosti) ostatních účastníků. Hráč tedy neví přesně, proti komu hraje, nebo jak moc si soupeř cení výhry. Typickým příkladem jsou aukce, pohovory nebo utajená obchodní vyjednávání.

1. Harsanyiho transformace Americký ekonom John Harsanyi vyřešil problém neúplných informací tím, že do hry zavedl Přírodu jako úvodního hybatele.

- **Určení typů:** Hra začíná nultým tahem, ve kterém Příroda náhodně vylosuje **typ** pro každého hráče (např. agresivní konkurent vs. opatrný konkurent).
- **Změna informací:** Každý hráč následně zjistí svůj vlastní typ, ale u soupeřů zná pouze *rozdělení pravděpodobnosti*, s jakou mohou být různými typy.
- **Výsledek:** Původní hra s *neúplnou* informací se matematicky převede na řešitelnou hru s *nedokonalou* (ale už úplnou) informací.

2. Bayesovská Nashova rovnováha Protože hráči nevědí s jistotou, jaký typ soupeře proti nim stojí, nemohou hrát klasickou nejlepší odpověď na jeden konkrétní tah. Hledá se proto Bayesovská rovnováha.

- **Princip:** Hráč musí zvážit všechny možné typy svého soupeře, vynásobit jejich tahy pravděpodobností jejich výskytu a podle toho maximalizovat svůj očekávaný užitek.
- Rovnováhy je dosaženo v momentě, kdy je strategie každého hráče tou nejlepší možnou odpovědí na očekávané chování všech ostatních hráčů napříč všemi jejich myslitelnými typy.

Poznámka: Každá konečná hra s neúplnou informací má alespoň jedno Bayesovo-Nashovo rovnovážné řešení.